People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research

University of Science and Technology Houari Boumédiène

Faculty of Computer Science
Department of Artificial Intelligence and Data Science

Master's Thesis

Specialization: Intelligent Computer Systems

Prediction and Optimization of Processes in the Algiers Refinery

---

Supervised by:
- Mr. TAKDJOUT Amine
- Mrs. HEDJAZI-DELLAL Badiâa

Prepared by:
- AIRECHE Maria
- AIT AMARA Mohamed

Presented to the jury composed of:
- Jury President: Mrs. BOUGHACI Dalila
- Jury Member: Mrs. BELAGGOUN Nassima

Project No: SII_05
2024/2025

Acknowledgements

First and foremost, we thank God Almighty, who granted us the strength and determination to carry out this work.

We would like to express our sincere gratitude to all those who, directly or indirectly, contributed to the completion of this thesis.

We are deeply grateful to Mrs. HEDJAZI-DELLAL Badiâa, our academic supervisor, for her availability, methodical guidance, and valuable advice throughout this project.

We also extend our heartfelt thanks to Mr. TAKDJOUT Amine, our industrial supervisor at Sonatrach, for his insightful feedback, professional rigor, and the trust he placed in us within a demanding industrial context.

Our appreciation also goes to Mrs. LOULOU Farah, process engineer at Sonatrach, for her expertise, technical explanations, and involvement in interpreting the results.

Finally, our most sincere thoughts go to our parents, who have never stopped encouraging us, supporting us morally, and believing in us, even in the most difficult moments.

Dedications

To our parents, for their constant support throughout our journey.

To our brothers and sisters, for their unwavering support.

To our friends, for their help and motivation.

To everyone who takes interest in this academic work.

We dedicate this thesis to all of you.

Abstract

This work presents the design of an intelligent pipeline combining forecasting and anomaly detection techniques for monitoring a complex industrial process (RFCC) in the Algiers refinery. The approach relies on an LSTM model with multi-head attention for anticipating future sensor values, coupled with a BiLSTM Autoencoder to detect abnormal deviations. We also integrate domain-specific heuristics based on critical thresholds validated by process engineers to improve the reliability of the detection system.

The evaluation is conducted using standard metrics (MSE, MAE, precision, recall, F1-score) and enriched with visual tools such as reconstruction error plots and smoothed data deviation curves. The LSTM-Attention model achieved an MSE of 0.0042 and a MAE of 0.0468, while the anomaly detection pipeline reached a precision of 73%, recall of 89%, and F1-score of 80%. Results demonstrate that the pipeline can detect both known anomalies and previously unlabeled ones, sometimes before they occur. Looking ahead, this system could serve as a semi-autonomous labeling tool, contributing to smarter diagnostics in the context of Industry 4.0.

Keywords: anomaly detection, time series, LSTM autoencoder, BiLSTM, multi-head attention, RFCC, Industry 4.0, predictive maintenance.

Résumé

Ce travail présente la conception d'un pipeline intelligent combinant des techniques de prévision et de détection d'anomalies pour la surveillance d'un procédé industriel complexe (RFCC) à la raffinerie d'Alger. L'approche repose sur un modèle LSTM avec mécanisme d'attention multi-tête pour anticiper les valeurs futures des capteurs, couplé à un autoencodeur BiLSTM pour détecter les écarts anormaux. Des heuristiques spécifiques au domaine, basées sur des seuils critiques validés par les ingénieurs procédé, sont également intégrées afin d'améliorer la fiabilité du système de détection.

L'évaluation a été réalisée à l'aide de métriques standards (MSE, MAE, précision, rappel, F1-score) et complétée par des outils visuels tels que les courbes d'erreur de reconstruction et les déviations lissées. Le modèle LSTM-Attention a obtenu un MSE de 0,0042 et un MAE de 0,0468, tandis que le pipeline de détection d'anomalies a atteint une précision de 73%, un rappel de 89% et un F1-score de 80%. Les résultats montrent que le pipeline permet de détecter à la fois des anomalies connues et d'autres non étiquetées auparavant, parfois même avant qu'elles ne surviennent. À terme, ce système pourrait servir d'outil d'étiquetage semi-autonome, contribuant à un diagnostic plus intelligent dans le cadre de l'Industrie 4.0.

Mots-clés: détection d'anomalies, séries temporelles, autoencodeur LSTM, BiLSTM, attention multi-têtes, RFCC, Industrie 4.0, maintenance prédictive.

Contents

List of Figures

List of Tables

List of Algorithms

General Introduction

The RFCC Unit

The RFCC unit (Residue Fluid Catalytic Cracking), part of the new installations at the Algiers refinery, constitutes a key component of the facility.
It is dedicated to the catalytic cracking of heavy residues from distillation in order to convert them into lighter, commercially valuable products such as gasoline and diesel.
Its operation is based on chemical reactions carried out under specific conditions of temperature, pressure, etc. The efficiency of this unit directly impacts the profitability of the refinery; however, it is frequently subject to anomalies that may reduce its performance.

This document focuses on the prediction and detection of such anomalies, with the aim of improving the RFCC unit's performance and reducing the risk of malfunctions.

Objective

The objective of this work is to design an intelligent system capable of predicting and detecting anomalies related to the chemical reactions within the RFCC unit.

Such a solution holds strategic value for Sonatrach by enabling proactive and intelligent monitoring of its critical units, thereby enhancing profitability, operational reliability, and the safety of industrial facilities.

Using advanced deep learning techniques, this system will:

- Predict the future values of critical variables in the RFCC unit over a temporal horizon (e.g., 1 hour or more), based on historical time series data.

- Detect anomalies within our predictions by comparing the results with expected behaviors, in order to identify unusual or suspicious deviations.

- Deploy the system's results through an interactive web application, facilitating interpretation, visualization, and decision-making by operators and engineers.

Our contribution lies both in theory and practice, through the design of a lightweight and generalizable solution tailored to industrial environments with limited labeled data and constrained hardware resources. This approach not only enhances early anomaly detection but also helps prevent costly failures, thereby generating substantial savings for the company.

The implementation of this system aims to optimize the operation of the RFCC unit while reducing costs associated with unexpected shutdowns and maintenance.

Thesis Organization

This thesis is divided into two main parts. The first part, dedicated to the state of the art, consists of four chapters: first, we will describe the industrial context and the RFCC unit. Then, we will discuss the detection techniques commonly used in the field of industrial anomaly detection. We will follow the same approach for the prediction techniques. Finally, we will review some existing related works.

The second part, focused on design and implementation, includes two chapters: the first details the design of our solution. Then, we move on to the final chapter of this thesis, which illustrates the implementation of the solution and presents the final evaluation of the pipeline.

Part I : State of the Art

Chapter 1: Host Organization and Problem Statement

1.1 Introduction

The objective of this chapter is to contextualize our project within its real industrial environment. We first introduce the Algiers refinery, and more specifically the RFCC unit (Residue Fluid Catalytic Cracking), which is the core of the process under study. We then describe how it operates, highlight the critical thermal anomalies that can occur, and discuss the limitations of the current monitoring methods used by process engineers. Finally, we present the problem our solution aims to address: the prediction and early detection of these anomalies based on time series data.

1.2 The Algiers Refinery

Algeria has five main refineries located in Skikda, Algiers, Arzew, Adrar, and Hassi-Messaoud [1], all operated by Sonatrach, the national oil company. Among them, the Algiers refinery, located in Sidi Arcine, stands out for its historical and strategic importance. Commissioned in 1964, it was initially designed to process around 3.6 million tons of crude oil per year [2]. After several decades of operation, an ambitious modernization project was launched to upgrade its facilities and enhance its performance. Although the initial contract was awarded to Technip in 2010, it was the Chinese company CPECC that ultimately carried out the rehabilitation work starting in 2016. Less than three years later, the refinery was re-inaugurated on February 21, 2019, with a 35% increase in refining capacity and production meeting Euro 5 standards, allowing it to sustainably meet the fuel needs of the central region while reducing reliance on imports [2].

This transformation is part of a broader national program to upgrade northern refineries, aiming to increase processing capacity and improve the quality of end products, particularly gasoline and diesel. Efforts also focused on industrial safety and environmental compliance, extending the operational lifespan of the installations by at least fifteen years [1].

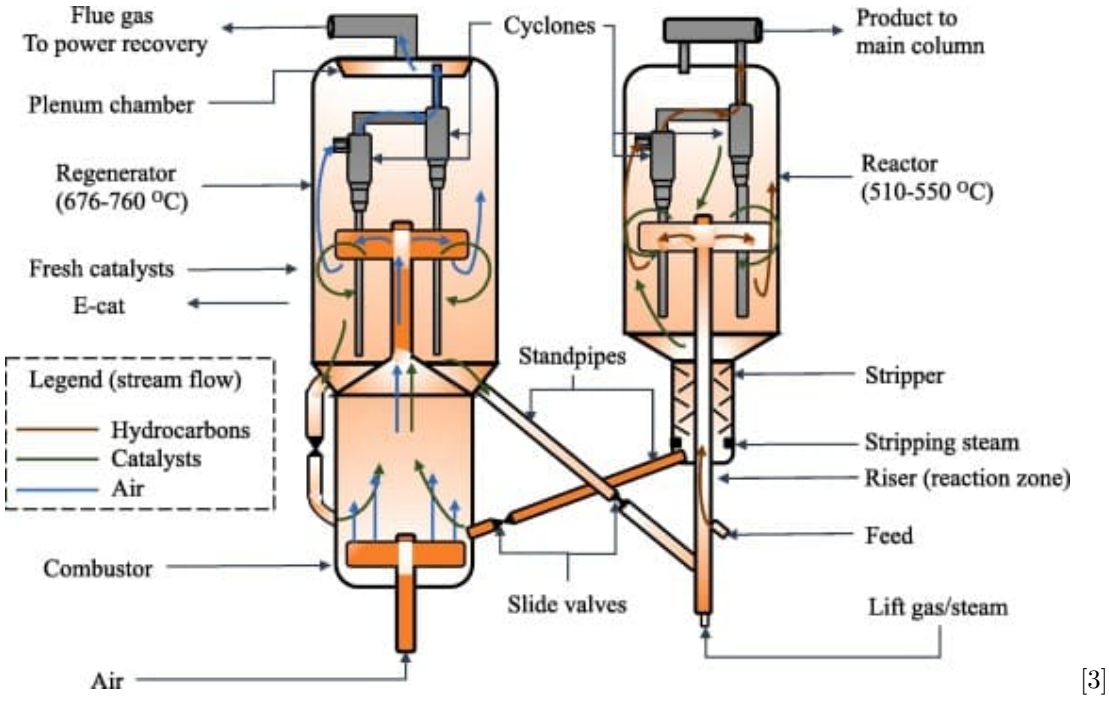
At the heart of this modernized infrastructure lies the RFCC unit (Residue Fluid Catalytic Cracking), one of the most complex and essential components of the refinery.

1.2.1 Sonatrach – RPC Division

Sonatrach is the leading energy company in Algeria, covering the exploration, production, transportation, processing, and marketing of hydrocarbons. Its RPC (Refining & Petrochemicals) division is specifically responsible for managing refining activities, including the Algiers refinery.

1.3 The RFCC Unit (Residue Fluid Catalytic Cracking)

The RFCC unit is dedicated to the fluid catalytic cracking of heavy residues to convert them into lighter, more valuable products such as gasoline and diesel.



[3]

Figure 1.1: Schematic view of the RFCC unit

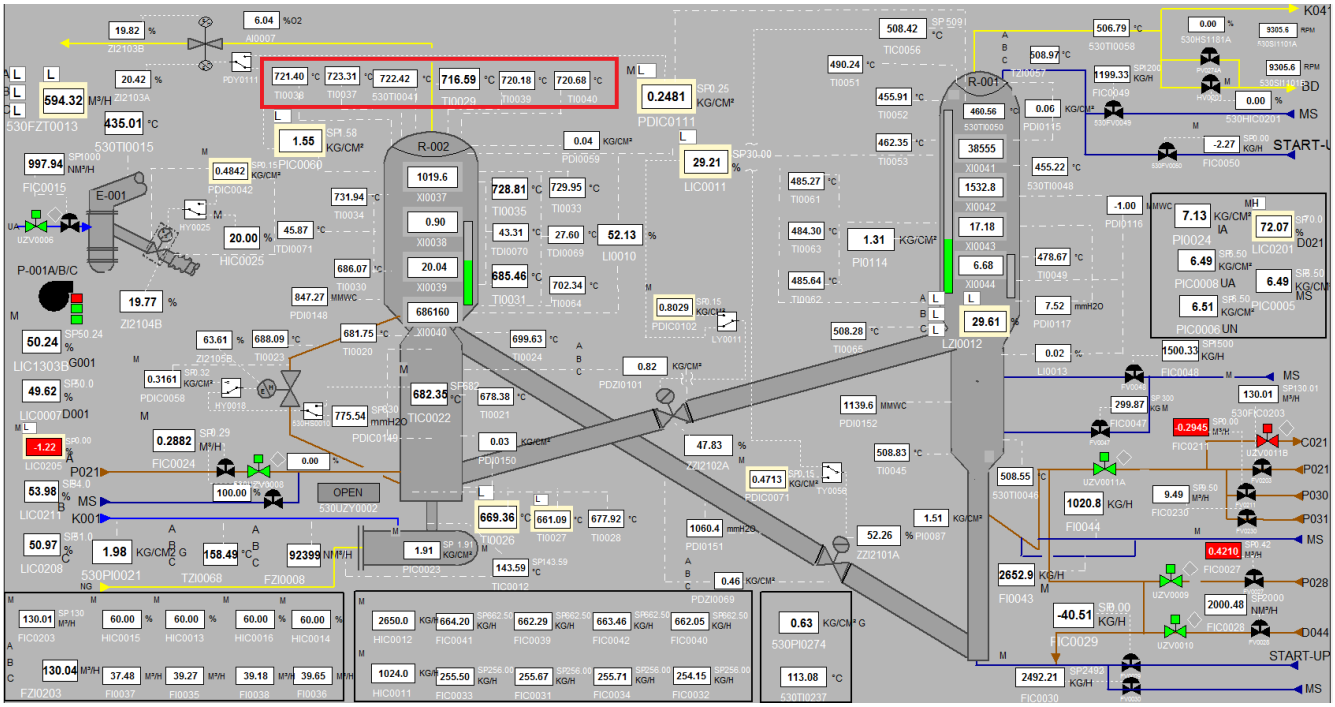
The catalytic cracking process takes place at high temperatures (approximately 508 °C) and moderate pressure, in the presence of a fluidized powdered catalyst that circulates continuously between the reactor and the regenerator. This circulation ensures optimal catalytic activity by promoting the breaking of long carbon chains. However, during the process, the catalyst becomes progressively coated with a non-volatile carbonaceous material commonly known as "coke", which, once deposited on the catalyst, deactivates its catalytic cracking function [4].

The restoration of the spent catalyst, which accumulates at the bottom of the reactor in a section called the **stripping** zone, is therefore necessary. It is directed to the regenerator and exposed to a stream of hot air, ranging from 648 °C to 746 °C [4], allowing the coke to burn off and the catalyst to regain its activity. This regeneration process must be optimized to avoid excessive energy consumption or catalyst damage, which would negatively affect the overall efficiency and cost-effectiveness of the process.

1.3.1 Anomalies in Catalyst Regeneration

The regenerator typically operates under conditions that allow for the complete combustion of coke deposited on the spent catalyst. However, the combustion temperature may vary in the case of partial combustion or post-combustion, when operating conditions lead to a heat production level below or above the nominal values [4].

To detect such anomalies, engineers rely on six temperature sensors: `TI0036, TI0037, TI0038, TI0039, TI0040, TI0041`, located at the top of the regenerator, as shown in Figure 1.2.



[4]

Figure 1.2: Monitory interface of the RFCC unit

The main anomalies are:

Afterburning (Post-combustion)

When the temperature indicated by the sensors exceeds 730 °C, a phenomenon known as **afterburning** may occur. This issue can result from:

- Excessive coke combustion caused by an over-supply of air.
- Poor distribution of air within the regenerator.

Although this phenomenon is mostly observed in the regenerator, it can also occur in the reactor section particularly if poorly regenerated catalyst particles, still carrying coke or only partially burnt, are reintroduced into the riser. In some cases, residual oxygen manages to infiltrate this zone and disrupts the balance by triggering additional combustion reactions. In a system designed for heat-absorbing (endothermic) reactions, these intrusions cause unwanted exothermic reactions. The result? A sharp, uncontrolled rise in temperature that damages the catalyst, disrupts the normal cracking process, and degrades the final product quality.

Incomplete Regeneration

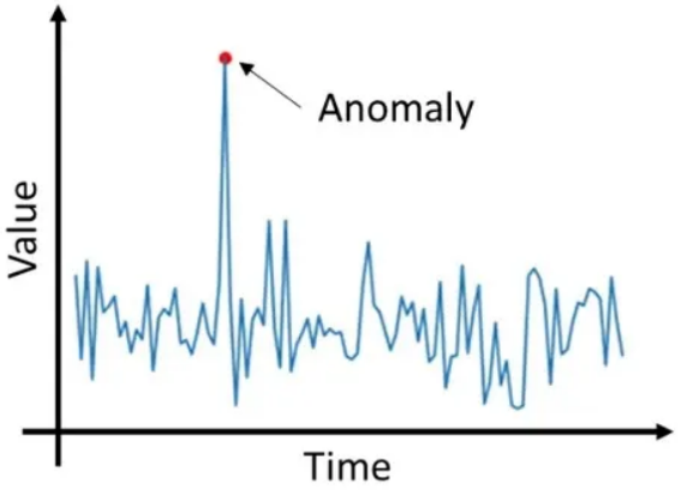
When the regenerator temperature drops below 690 °C, coke combustion becomes incomplete. As a result, the catalyst remains partially fouled, which reduces:

- Its effectiveness in the reactor.
- The overall cracking yield, with progressive buildup of carbon residues.

This regeneration failure may also lead to reactor anomalies: an insufficiently regenerated catalyst reduces cracking efficiency, lowers product selectivity, and promotes rapid coke accumulation ultimately degrading system performance.

Both anomalies: afterburning and incomplete regeneration, represent critical challenges for the RFCC unit, affecting both the reactor and the regenerator. Although they manifest differently, they share a common point: they disrupt the system's thermal stability and overall efficiency. Rather than treating them separately, we chose to group them into a single anomaly category. This simplification allows us to better predict overheating risks and detect abnormal behavior.

From an AI perspective, this unified thermal anomaly appears as outliers in the sensor data. It's like an industrial ECG: when the temperature deviates from its normal range, it's a sign that something is off. These unexpected spikes serve as alerts for a thermal imbalance that requires quick intervention.



[5]

Figure 1.3: Illustration of an anomaly in a time series

1.4 Problem Statement

In the context of monitoring the RFCC unit (Residue Fluid Catalytic Cracking), process engineers at Sonatrach currently rely on the previously listed indicators to detect potential abnormal behaviors or signs of degradation. While these indicators are effective at signaling critical situations, they only capture a partial view of the unit's overall state and often with a delay. Emerging failures, which tend to worsen over time before triggering these alarms, often go undetected in the early stages. This limits the ability to anticipate problems and increases the risk of severe failures or even production shutdowns. Relying on a single variable thus constitutes a significant limitation in terms of early detection, diagnosis, and predictive maintenance.

1.5 Conclusion

This chapter has allowed us to position our project within its industrial context by presenting the RFCC unit of the Algiers refinery, its role in the catalytic cracking process, and the various types of thermal anomalies that can occur and disrupt operations. We have highlighted the strategic

limitations of existing diagnostic methodologies and introduced the core problem that our proposed solution aims to address.

The following chapter serves as a transition and will present a state-of-the-art review of anomaly detection techniques and time series forecasting methods.

Chapter 2: Anomaly Detection

2.1 Introduction

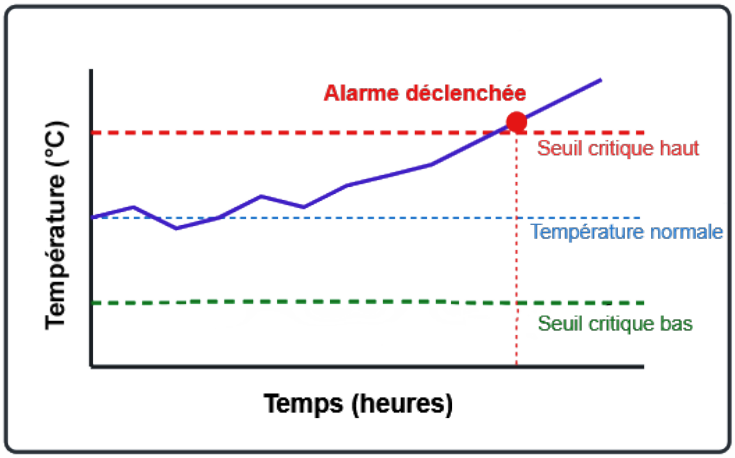
In this chapter, we present a set of widely used techniques for anomaly detection in industrial systems. We begin with classical statistical methods, which are still commonly applied due to their ease of implementation. We then discuss techniques derived from machine learning, including its two main categories: supervised and unsupervised approaches. Finally, we explore recent and increasingly popular deep learning-based methods, capable of capturing complex relationships within time series data.

2.2 Classical Statistical Methods

In our field, the earliest approaches were based on simple-to-implement statistical methods. However, they lack flexibility when dealing with complex and non-linear behaviors, such as those observed in catalytic cracking processes.
Among these methods, we mention the following:

2.2.1 Thresholding

This approach consists of defining critical thresholds. Currently, the refinery uses a similar system, known as a "setpoint", to monitor temperature variations at the top of the regenerator.



[5]

Figure 2.1: Critical Temperature Threshold Method

In Figure 2.1, we observe a delayed detection of a gradually developing issue. Moreover, determining the normal threshold values for all sensors (attributes) is sometimes very challenging, especially when the dataset is high-dimensional and the actual operating values differ from the original design specifications.

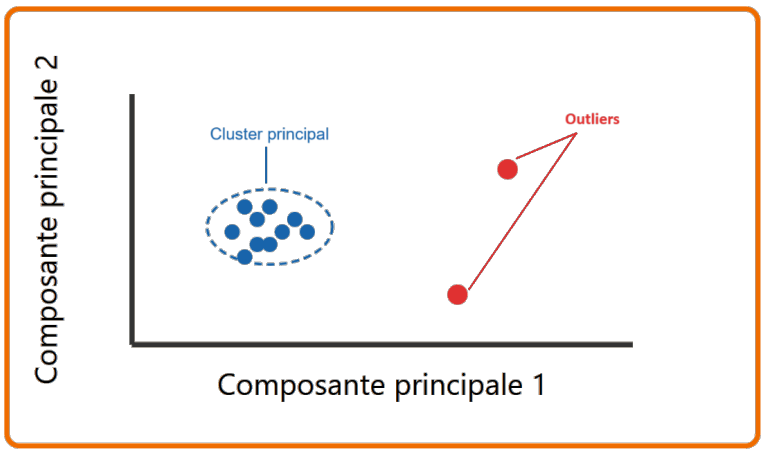
2.2.2 Principal Component Analysis (PCA)

PCA is a dimensionality reduction technique, widely used in industry for anomaly detection. It involves projecting data into a new space whose axes, known as "principal components", are defined so as to maximize the variance of the projections. In this way, the information contained in the original dataset can be captured using a limited number of relevant directions [6].

Anomaly detection using this technique relies on identifying observations that no longer follow the correlation structure learned from normal data. Specifically, one can focus on the projections of instances onto the components with low variance: while normal data typically show small projections on these components, anomalies that disrupt the system structure tend to project strongly onto them [6].

One approach is to use robust PCA in order to better withstand the presence of noise or outliers. This involves computing the principal components based on data assumed to be normal, and then calculating the distance of a new observation to this subspace. If this distance is statistically significant (i.e., beyond a threshold derived from a χ^2 distribution), the observation is flagged as anomalous [6].

Similar approaches have been successfully applied in various domains such as embedded component monitoring (in aerospace), cybersecurity (intrusion detection), and evolving graph analysis in complex systems [6].



[6]

Figure 2.2: Dimensionality reduction illustration using PCA

Despite the fact that this type of analysis presents several advantages; such as its applicability to unlabeled data, its ability to model multivariate correlations, and its usefulness as a preprocessing step or in combination with other models; it also suffers from certain limitations. These include high sensitivity to noise, lower performance in detecting non-linear anomalies, and a strong reliance on the assumption that the majority of the data is normal.

2.3 Machine Learning-Based Approaches

With the rise of machine learning, more sophisticated techniques have been developed:

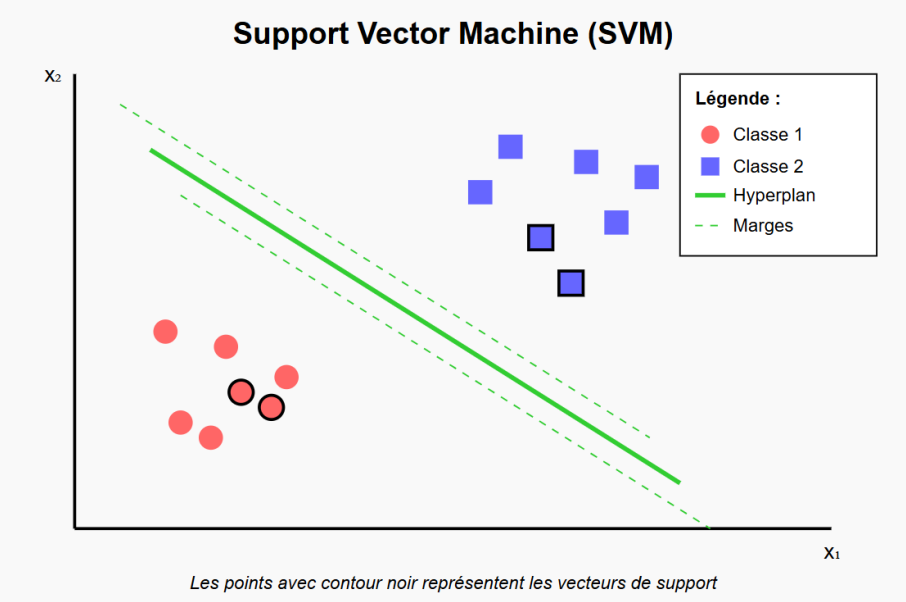
2.3.1 Support Vector Machines (SVM)

The Support Vector Machine (SVM) is a supervised learning method primarily used to solve binary classification problems. It aims to find the optimal hyperplane that separates different classes in the data space while maximizing the margin between them [7].

When the data is linearly separable, a hard-margin SVM is used. For noisy or non-separable data, a soft-margin SVM allows some tolerance for classification errors [7].

In cases where the separation is not linear, the SVM can leverage kernel methods, which project the data into a higher-dimensional feature space where linear separation becomes possible [7].

As such, the classical SVM can be seen as a prerequisite before moving on to more complex anomaly detection cases, such as the One-Class SVM.



[7]

Figure 2.3: Illustration of the working principle of an SVM in a two-dimensional space.

2.3.2 One-Class Support Vector Machines

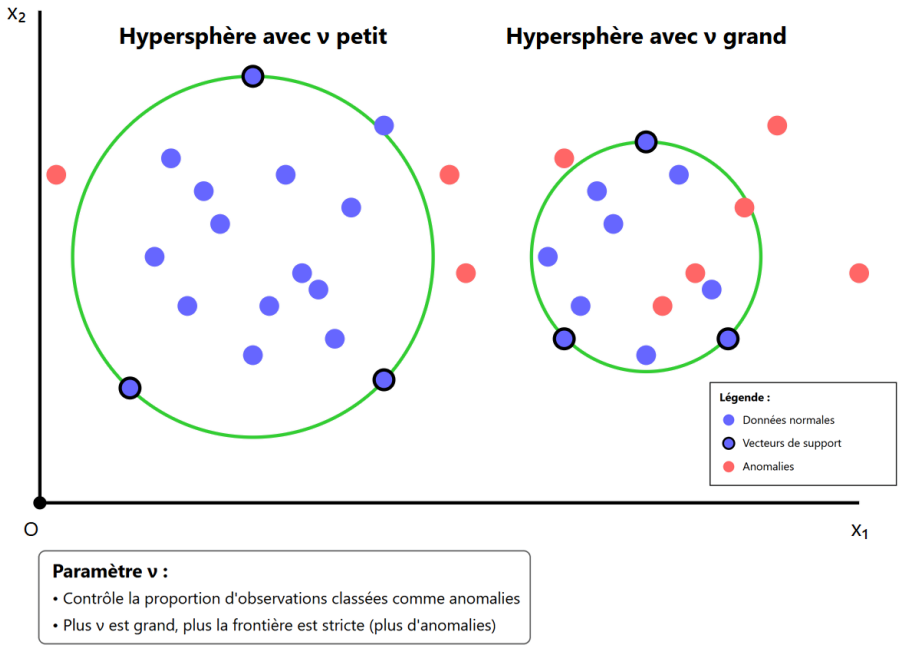
The One-Class SVM is an unsupervised extension of the standard SVM, specifically designed for anomaly detection or learning from datasets that contain only normal instances [7].

Unlike binary SVMs that separate two classes, the One-Class SVM seeks to distinguish the bulk of the normal data from the rest of the space by constructing a hyperplane that maximizes the margin from the origin.

Two main approaches exist to define this boundary:

- **Hypersphere**: encloses the majority of normal data (Tax and Duin approach).
- **Hyperplane relative to the origin**: separates normal data from the origin (Schölkopf and Müller approach) [7].

The parameter ν is important as it determines the proportion of observations considered as anomalies. Therefore, it serves as a control lever to adapt to various industrial scenarios, especially when anomalies are rare or difficult to label [7].



[7]

Figure 2.4: Illustration of the One-Class SVM using the hypersphere approach

The One-Class SVM presents several advantages. Notably, it is effective when detecting anomalies in cases where no abnormal examples are present in the training data. Additionally, through the use of kernel functions, it is capable of detecting complex decision boundaries.

However, this technique also has drawbacks. For instance, detection performance is highly sensitive to the choice of hyperparameters such as ν and γ. Moreover, the computational cost of the One-Class SVM is high ($O(n^2)$ to $O(n^3)$), making it challenging to apply to large datasets with many variables. Furthermore, unlike more interpretable techniques such as decision trees, the One-Class SVM does not provide explicit explanations for why an observation is considered anomalous.

Lastly, its main advantage can become a limitation when the training data is not clean (i.e., contains anomalies).

Despite these shortcomings, the One-Class SVM remains an essential method for identifying abnormal behavior in complex industrial processes [7].

2.3.3 Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

DBSCAN is a density-based clustering method that is particularly useful for anomaly detection, especially when anomalies form small, sparse clusters. Unlike traditional clustering methods, DBSCAN does not require the number of clusters to be specified in advance, making it a flexible approach. Furthermore, it is highly sensitive to anomalies, as it readily identifies them as isolated points [8].

However, DBSCAN also presents several drawbacks. Its algorithmic complexity can make it inefficient for large-scale time series, especially when the number of attributes is high. Additionally, its hyperparameters such as the maximum distance between two points to be considered neighbors (ϵ) and the minimum number of points to form a cluster (MinPts) can be difficult to tune optimally, often requiring experimental adjustment or fine-tuning [8].

2.3.4 Isolation Forest

Isolation Forest is an unsupervised learning approach designed to detect various types of anomalies. Unlike other methods that attempt to model normal data, Isolation Forest takes an opposite approach: it aims to isolate anomalies by actively separating them from the majority of normal data [9].

The underlying idea involves building an ensemble of randomly generated decision trees, known as isolation trees. Each tree partitions the data through a recursive process known as splitting, where features and split thresholds are randomly selected. Because anomalies are rare and different from the norm, they tend to be easier to isolate, requiring fewer splits to be separated from the rest of the data. An observation is considered anomalous if it can be isolated quickly, i.e., with a short path length in the tree [9].

Isolation Forest is particularly well-suited for high-dimensional data and large datasets, as its algorithmic complexity is linear with respect to the number of samples, making it a highly efficient method for industrial applications requiring fast anomaly detection [9].

This is a fully unsupervised method that does not require labeled anomaly examples during training. It performs well on large-scale, high-dimensional datasets and features a particularly fast runtime of $O(n \log n)$, making it suitable for industrial contexts where large-scale, real-time detection is required.

On the other hand, this technique has some limitations. It may struggle to isolate anomalies that are too close to normal points, which can negatively affect overall detection performance. Additionally, its performance deteriorates significantly when too many frequent anomalies are present in the dataset. Lastly, Isolation Forests tend to be sensitive to certain hyperparameters, such as the number of trees and the maximum tree depth, which must be carefully tuned to achieve optimal results.

2.4 Deep Learning-Based Approaches

With the rapid development of deep learning, new and powerful methods have emerged to tackle complex anomaly detection problems by leveraging the ability of deep neural networks to automatically extract representative features from data.

2.4.1 AutoEncoders

An Autoencoder (AE) is a specific type of feedforward neural network composed of interconnected artificial neurons. In a feedforward network, the connections between neurons do not form any cycles; the information flow is unidirectional, starting from an input layer, passing through one or more hidden layers, and ending at an output layer.

In each hidden layer, a neuron receives several inputs, computes the weighted sum of these inputs plus a bias, and applies an activation function to produce an output. This output is then

used as input to the next layer. The final output y of the network can thus be expressed as a function of the input x, depending on the weights, biases, and the successive activation functions.

The structure of an Autoencoder is symmetric around its central point, called the bottleneck. Unlike standard neural networks, the output layer of an AE has exactly the same number of neurons as the input layer. The Autoencoder is trained in an unsupervised manner: each input vector is used both as the input and the target (desired output). The goal is to minimize the reconstruction error, i.e., the difference between the original input and its reconstruction at the output.

The bottleneck represents a compressed encoding (or latent representation) of the input data. This encoding can be exploited as a feature extraction method or for dimensionality reduction.

Formally, an Autoencoder consists of two components:

- The encoder α, which maps the input $x \in X$ to a coded representation $y \in Y$.
- The decoder β, which reconstructs the input data from the code y to generate $x' \in X'$.

In a simple one-hidden-layer model, these operations are defined as:

$$y = \theta(Wx + b)$$

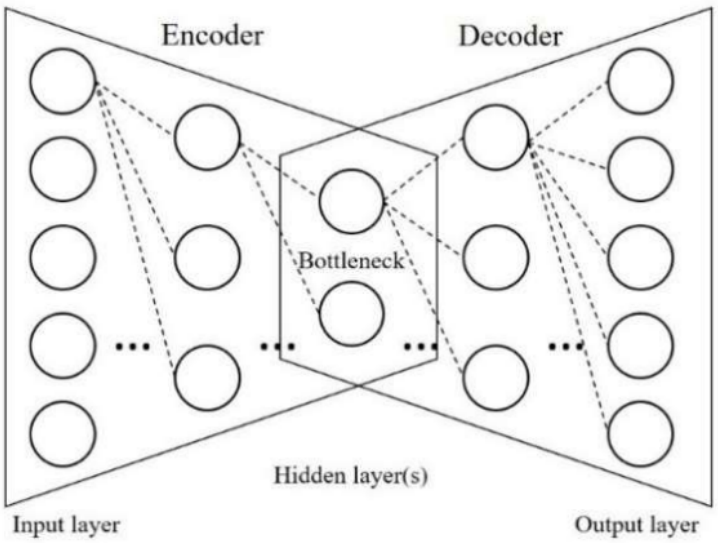
$$x' = \theta'(W'y + b')$$

where W and W' are weight matrices, b and b' are bias vectors, and θ, θ' are activation functions.

The training objective of the AE is to minimize the reconstruction loss function:

$$\text{Loss}(x, x') = \|x - x'\|^2$$

In more complex architectures with multiple hidden layers, these formulas are generalized by introducing additional intermediate activations, which are omitted here for simplicity.



[9]

Figure 2.5: Illustration of an Autoencoder model.

2.4.2 Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM) networks are an extension of recurrent neural networks (RNNs), specifically designed to handle long-term dependencies in sequential data. Unlike standard RNNs, LSTMs include control mechanisms known as "gates" (input, forget, and output gates), which regulate the flow of information within the network. This helps to mitigate the issues of vanishing or exploding gradients during training [10].

Their operation is as follows [10]:

- **Input gate equation (z):**

$$\mathbf{z_{i,j}} = \mathbf{g}(\mathbf{W^{(z)}x_{i,j}} + \mathbf{R^{(z)}h_{i,j-1}} + \mathbf{b^{(z)}})$$

This equation computes the input gate, which determines what information will be added to the cell state. The function $g(\cdot)$ is the hyperbolic tangent (tanh).

- **Forget gate equation (s):**

$$\mathbf{s_{i,j}} = \sigma(\mathbf{W^{(s)}x_{i,j}} + \mathbf{R^{(s)}h_{i,j-1}} + \mathbf{b^{(s)}})$$

This equation computes the forget gate, which determines what information will be retained from the previous cell state. The function $\sigma(\cdot)$ is the sigmoid function.

- **Control gate equation (f):**

$$\mathbf{f_{i,j}} = \sigma(\mathbf{W^{(f)}x_{i,j}} + \mathbf{R^{(f)}h_{i,j-1}} + \mathbf{b^{(f)}})$$

This equation computes the control gate, which regulates the amount of information to retain in the cell state. It produces a value between 0 and 1.

- **Cell state update equation (c):**

$$\mathbf{c_{i,j}} = \mathbf{s_{i,j}} \odot \mathbf{z_{i,j}} + \mathbf{f_{i,j}} \odot \mathbf{c_{i,j-1}}$$

This equation updates the cell state by combining new information with previous information. The operator $\odot$ denotes element-wise multiplication.

- **Output gate equation (o):**

$$\mathbf{o_{i,j}} = \sigma(\mathbf{W^{(o)}x_{i,j}} + \mathbf{R^{(o)}h_{i,j-1}} + \mathbf{b^{(o)}})$$

This equation computes the output gate, which determines what parts of the cell state will be passed to the output.

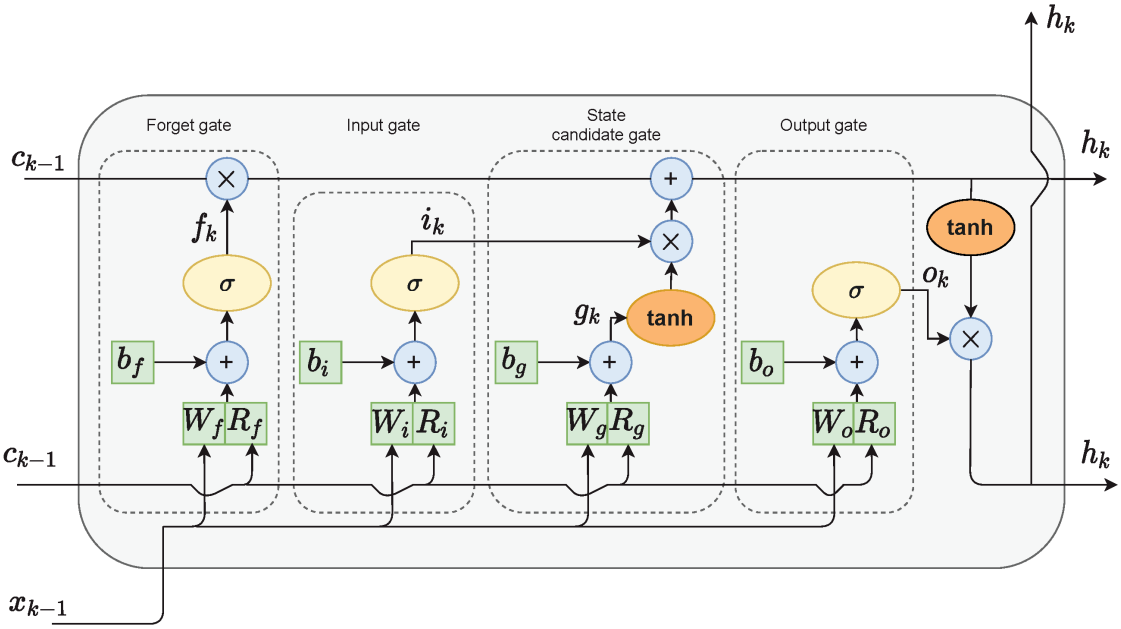
- **Hidden state output equation (h):**

$$\mathbf{h_{i,j}} = \mathbf{o_{i,j}} \odot \mathbf{g}(\mathbf{c_{i,j}})$$

This equation computes the hidden output, which is passed to the next LSTM unit.

where $c_{i,j} \in \mathbb{R}^m$ is the cell state vector, $x_{i,j} \in \mathbb{R}^p$ is the input vector, and $h_{i,j} \in \mathbb{R}^m$ is the output vector for the j-th LSTM unit. The variables $s_{i,j}$, $f_{i,j}$, and $o_{i,j}$ represent the input, forget, and output gates, respectively. The function $g(\cdot)$ is defined as the hyperbolic tangent (tanh) and is applied element-wise to the input vectors. Similarly, $\sigma(\cdot)$ refers to the sigmoid activation function. The operator $\odot$ denotes element-wise multiplication between two vectors of the same size.

The parameters $W^{(\cdot)}$, $R^{(\cdot)}$, and $b^{(\cdot)}$ represent the weights and biases of the LSTM architecture, whose dimensions are determined by the input and output vector sizes. The cell state $c_{i,j-1}$ corresponds to the memory state from the previous LSTM block, thereby ensuring a continuous flow of information across consecutive LSTM units.



[11]

Figure 2.6: Internal architecture of an LSTM cell

LSTM networks are particularly well suited for anomaly detection in time series data due to their ability to model temporal dynamics. In unsupervised learning settings, a common approach involves training the LSTM on sequences known to be normal, then detecting anomalies based on the reconstruction error. A high reconstruction error indicates an unexpected behavior, suggesting the presence of an anomaly [10].

An innovative approach has been proposed: instead of relying solely on the reconstruction error, the authors combine LSTM outputs with classification methods such as One-Class SVM or SVDD, and jointly optimize the parameters of both models. This significantly enhances detection performance, even for sequences of variable lengths [10].

In conclusion, LSTMs offer significant advantages for sequential data analysis. These include their ability to capture complex temporal dependencies, handle sequences of varying lengths

through pooling mechanisms, adapt to various learning paradigms (unsupervised, semi-supervised, and supervised), and provide superior anomaly detection performance compared to standalone methods such as OC-SVM or SVDD [10].

However, these benefits come with notable drawbacks: high training complexity due to the joint optimization of LSTM and classifier models, which requires substantial computational resources and may lead to instability; the need for careful tuning of critical hyperparameters such as cell size, learning rate, and regularization terms; lower interpretability compared to rule-based or tree-based approaches; and finally, a significant risk of overfitting if regularization mechanisms and appropriate constraints are not properly implemented and calibrated [10].

2.5 Conclusion

In this chapter, we reviewed a range of anomaly detection techniques for industrial systems, starting from traditional statistical methods to more recent approaches based on machine learning and deep learning. These techniques play a critical role in identifying abnormal behavior in high-risk industrial processes such as the RFCC unit.

However, the effectiveness of anomaly detection is closely tied to the quality of the underlying predictions. Therefore, the following chapter focuses on time series forecasting methods, where we explore predictive models that enable anticipatory analysis of industrial variables, ultimately leading to more robust anomaly detection.

Chapter 3: Timeseries Prediction

3.1 Introduction

Prediction plays a fundamental role in industrial monitoring systems, particularly for anticipating abnormal behavior. This chapter presents various approaches used in time series forecasting. These methods, stemming from different model families, are extensively explored in the literature for their ability to model the evolution of variables in complex contexts such as industrial processes.

3.2 Prediction Methods

Among the most commonly used predictive methods in industrial systems, some are based on well-established machine learning algorithms. Several approaches are recognized for their effectiveness, including Random Forests, Autoencoders, and Recurrent Neural Networks.

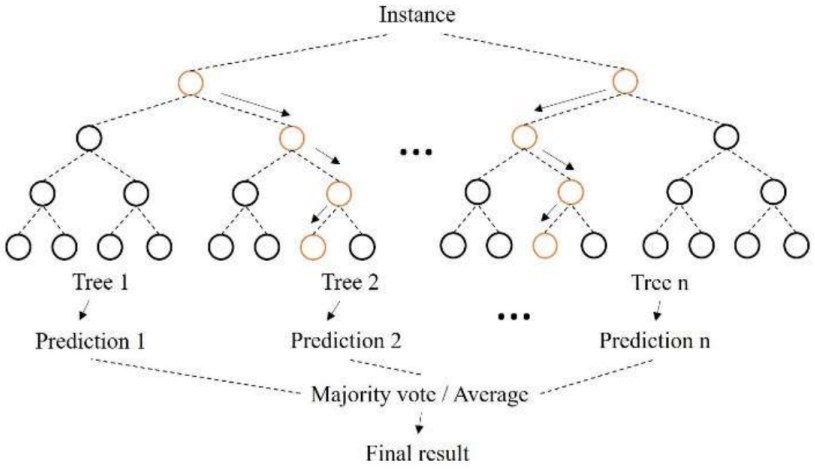
3.2.1 Random Forests

Random Forest (RF) is an ensemble learning technique used for classification, regression, and other predictive analysis tasks. This model relies on the use of a large number of decision trees and the bagging mechanism (bootstrap aggregating) to reduce the risk of overfitting, which is often associated with deep decision trees [12].

A decision tree is a tree-like structure where each internal node represents a decision rule based on a feature of the data, while each leaf corresponds to a prediction (either a class for classification or a value for regression). To determine the best split at each node, criteria such as Gini impurity, entropy, or standard deviation reduction may be used to maximize the information gain. The splitting process continues until a stopping condition is met, such as:

- All data points in a node belong to the same class.
- The number of observations in a node falls below a predefined threshold.
- The tree reaches a maximum predefined depth.

In a random forest, the bagging mechanism works as follows: from a dataset of d observations, n bootstrap samples (with replacement) are generated, each containing d randomly drawn observations. A model (tree) is trained on each bootstrap sample, resulting in n independent predictions. The final prediction is then aggregated either by majority voting (for classification) or by averaging (for regression) [12].



[12]

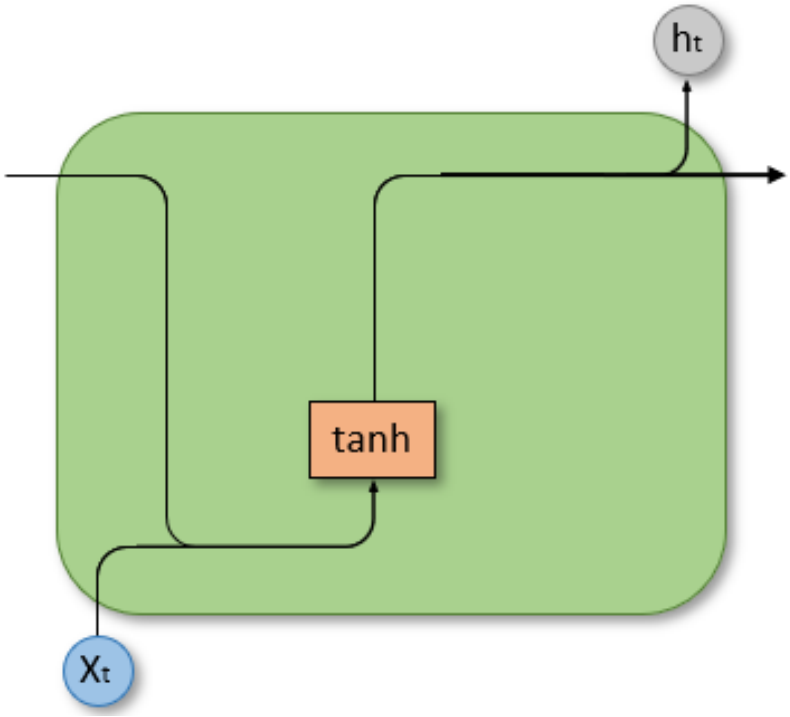
Figure 3.1: Illustration of the overall functioning of a Random Forest model.

In the context of predicting critical variables in the RFCC unit, Random Forests offer several advantages. They are capable of modeling complex relationships among input variables, even in the presence of noise or non-linear correlations, making them a robust tool for predicting, for example, temperature variations in the regenerator. Additionally, they require minimal hyperparameter tuning, adapt well to heterogeneous datasets, and provide insights into feature importance. However, Random Forests do not inherently account for the temporal dimension of the data, limiting their ability to capture sequential dynamics or gradual trends. In industrial time series contexts, it is often necessary to complement them with preprocessing techniques such as sliding windows or temporal feature extraction to preserve dependencies between observations.

3.2.2 Recurrent Neural Networks (RNNs)

Recurrent Neural Networks (RNNs) are widely used for modeling and forecasting temporal data. Thanks to their ability to exploit the sequential structure of data, they allow future values of a time series to be predicted based on its past states [13].

This makes them a suitable approach for forecasting in industrial contexts, where anomalies may manifest as gradual deviations from expected behavior. By comparing predicted values with actual observations, it becomes possible to detect significant discrepancies, which may indicate potential faults or degradations [13].



[14]

Figure 3.2: Internal structure of an RNN

To overcome certain limitations of conventional RNNs such as their difficulty in modeling long-term dependencies due to the vanishing gradient problem, and their often reduced performance on long or complex sequences; improved variants have been developed. These include LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Units), which will be discussed in more detail later in this chapter [13].

3.2.3 LSTM

LSTM networks can also be used in a purely predictive setting by directly modeling the temporal evolution of a variable. In this case, the objective is not to detect anomalies via reconstruction, but rather to forecast future values based on a history of measurements. For instance, an LSTM trained on temperature time series can predict the expected value at the next time step or across multiple future time horizons [13].

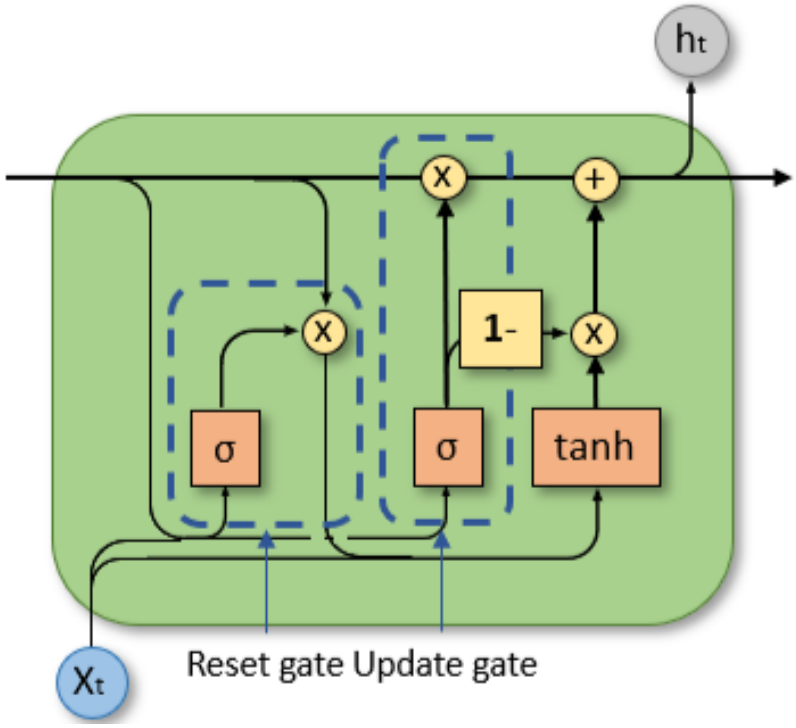
This approach is particularly relevant in industrial contexts such as the RFCC unit, where anticipating thermal drifts or pressure fluctuations is essential. Once trained, the model provides predictions that can be compared with actual measurements to identify abnormal deviations, or used directly for process regulation.

Using LSTM for prediction allows for modeling the system's normal dynamics and anticipating future behavior, making it a solid foundation for monitoring or predictive maintenance systems.

3.2.4 Gated Recurrent Units (GRU)

Gated Recurrent Units (GRUs) are a simplified variant of recurrent neural networks (RNNs), specifically designed to model temporal dependencies while reducing the structural complexity of models like LSTMs. In the context of anomaly prediction on time series, GRUs are used as

autoregressive models that learn the normal dynamics of sequences to then anticipate future values. An anomaly is thus detected when the difference between the predicted and actual value exceeds a certain threshold [13].



[14]

Figure 3.3: Internal structure of a GRU

Thanks to their lighter architecture (fewer gates than LSTMs), GRUs allow for faster training, making them well-suited for real-time or large-scale applications. Moreover, their ability to perform incremental updates makes them particularly useful in continuous data stream environments [13].

However, GRUs also have some limitations: their performance can heavily depend on the choice of the time window, they still require careful threshold calibration, and their interpretability remains limited compared to more explicit models. In summary, GRUs offer a good balance between efficiency and performance for anomaly prediction in industrial time series, especially when computational constraints are significant [13].

3.2.5 AutoRegressive Integrated Moving Average (ARIMA)

The ARIMA (AutoRegressive Integrated Moving Average) model is one of the most commonly used classical approaches for time series forecasting. It is based on three core components:

- **Autoregressive (AR) part:** This component models the dependency between an observation and several lagged observations. An AR(p) model is expressed as:

$$\text{AR}(p) : X_t = \phi_1 X_{t-1} + \phi_2 X_{t-2} + \cdots + \phi_p X_{t-p} + \epsilon_t$$

  where ϕ_i are the autoregressive coefficients, ϵ_t denotes white noise, and p is the order of the AR model, indicating how many lagged terms are used.

- **Moving Average (MA) part:** This component models the dependency between an observation and past forecast errors. An MA(q) model is expressed as:

$$\text{MA}(q) : X_t = \epsilon_t - \theta_1 \epsilon_{t-1} - \theta_2 \epsilon_{t-2} - \cdots - \theta_q \epsilon_{t-q}$$

where θ_i are the moving average coefficients, and q is the order of the MA model.

- **Integration (I) part:** This involves differencing the time series to make it stationary. For a differencing order $d = 1$, we compute:

$$\hat{X}_t = X_t - X_{t-1},$$

An ARIMA(p,d,q) model combines these three components and is formulated as:

$$\hat{y}_t = \mu + \underbrace{\phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p}}_{\text{AR}(p)} - \underbrace{\theta_1 \epsilon_{t-1} - \theta_2 \epsilon_{t-2} - \cdots - \theta_q \epsilon_{t-q}}_{\text{MA}(q)},$$

where μ is a constant and y_t represents the differenced series ($y_t = Y_t - Y_{t-1}$ for $d = 1$).

This formulation shows that each value at a specific time is influenced both by past observations (AR component) and past forecast errors (MA component). Differencing renders the series stationary, making ARIMA particularly effective for non-stationary time series.

This combination enables ARIMA to capture both trend and temporal dependency structures. As such, ARIMA is effective for univariate time series forecasting by considering both past observations and residual errors.

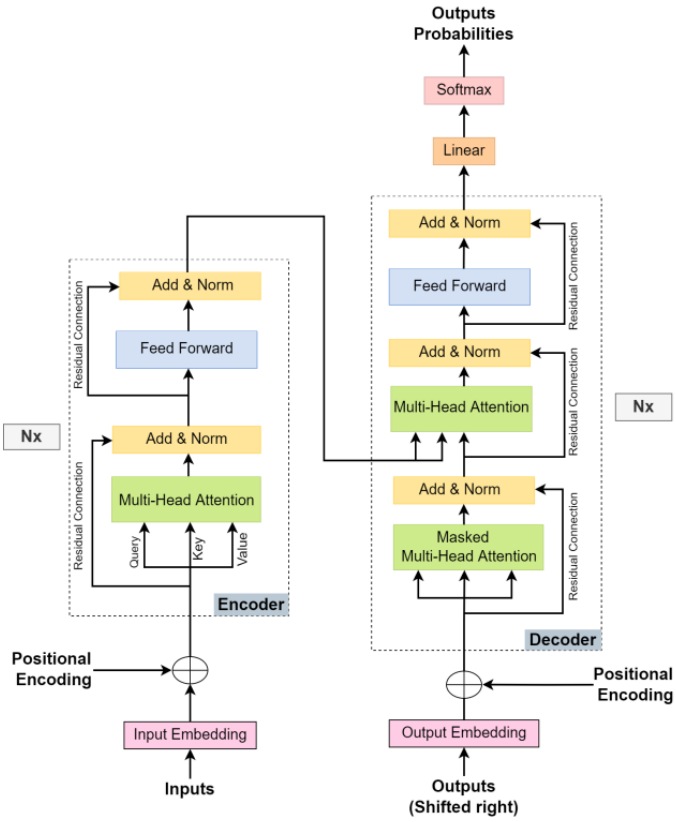
However, it has certain limitations: it assumes that the series is stationary (or made stationary through differencing) and is not inherently designed for multivariate data. In such cases, extensions like ARIMAX (which incorporates exogenous variables) or VAR (Vector AutoRegression) are employed.

Furthermore, when a time series exhibits strong seasonal patterns, the SARIMA (Seasonal ARIMA) model can be used, which introduces additional parameters to account for seasonality. Although ARIMA does not rely on deep learning techniques, it remains a robust and interpretable tool for time series modeling in industrial contexts.

3.2.6 Transformers

The Transformer, introduced in 2017 for neural machine translation tasks, has rapidly inspired numerous models adapted to various domains. Some retain the original full architecture, while others use only the encoder or decoder module, depending on the specific requirements of the task. Regardless of the variant, the **attention mechanism**, particularly the *multi-head attention*, remains the core learning component of Transformers. This mechanism allows the model to assign different weights to parts of an input sequence, effectively capturing long-range dependencies [15].

The Transformer relies on an autoregressive sequence transduction architecture composed of two main modules: the **encoder** and the **decoder**, each made up of several stacked layers [15].



[16]

Figure 3.4: Transformer Architecture

Encoder Each encoder layer includes:

- a multi-head attention layer;
- a feed-forward layer (fully connected network);
- residual connections;
- add and normalization layers (*Add & Norm*).

The input is first encoded as an embedding vector enriched with positional encoding. Then, the query, key, and value matrices are computed and processed by the attention layer. The feed-forward layer performs a non-linear transformation of the information, while the normalization layers stabilize the training process [15].

Decoder The decoder adopts the same structure as the encoder, with the addition of a **masked multi-head attention** layer to prevent access to future positions. It then applies a second attention layer, known as cross-attention, over the encoder's outputs. The resulting vectors pass through a feed-forward network, then through a linear layer and a SoftMax function to generate the outputs sequentially. This process is repeated until the final token of the sequence is produced [15].

In the context of chemical anomaly prediction within the RFCC unit, Transformer-based models offer several significant advantages. Their primary strength lies in their ability to effectively model long-term temporal dependencies, which is crucial for anticipating anomalies in a complex chemical

process. Unlike RNNs or LSTMs, which process sequences sequentially, Transformers rely on a global attention mechanism that enables them to process the entire sequence in parallel. This facilitates a better understanding of relationships between different phases of the process, even if they are temporally distant, such as the interaction between the regenerator temperature and the composition of the output stream.

Moreover, Transformers offer substantial architectural flexibility and can be easily adapted to multivariate sequences typical of sensor data in the RFCC unit. Thanks to their modular architecture, they can effectively integrate multiple types of signals (temperature, pressure, concentration, etc.), which is crucial for predicting abnormal behaviors before they become critical. The possibility of pretraining and fine-tuning on specific datasets further enhances their relevance for environments like the RFCC, where labeled data is scarce.

However, this power comes with notable limitations. First, Transformers have high computational complexity, especially for long temporal sequences often generated by industrial sensors. This can be a constraint when deploying in real-time or on resource-constrained systems. Additionally, the performance of Transformers often depends on the availability of large training datasets, which can be a barrier in the case of rare anomalies or limited operating periods.

3.3 Related Work

The detection and prediction of abnormal conditions in Fluid Catalytic Cracking (FCC) units have become major areas of research in the refining industry, due to the industrial risks, shutdown costs, and yield losses they entail. In response to the limitations of traditional approaches, many recent studies have turned to intelligent solutions, particularly those based on deep learning, hybrid models, and causal analysis. This chapter provides an overview of major contributions in this field, highlighting innovative methods that combine robustness, predictive accuracy, and interpretability. These works serve as both a reference point and a source of inspiration for the development of our own anomaly detection solution in the RFCC unit.

3.3.1 Work by Chenwei Tang et al.

Among the most promising recent contributions in the field of intelligent monitoring of FCC units is the work on the AEW-AOC framework (Attention-based Early Warning for Abnormal Operating Conditions). This innovative approach offers an early detection solution for abnormal operating conditions in FCC units, leveraging the advanced capabilities of deep neural networks. The main strength of this method lies in its ability to overcome the limitations of classical two-stage models by implementing an end-to-end architecture capable of anticipating anomalies from complex industrial time series. AEW-AOC is based on three key modules: a Self-Correlation Denoiser (SCD), which uses spatio-temporal correlations to reduce noise in data collected under extreme conditions; a Conv-LSTM capable of capturing delayed dynamic variations of critical FCC parameters; and an Anomaly Pattern Attention (APA) module that enhances the model's ability to distinguish abnormal behaviors from historical patterns. This last feature is particularly useful in highly imbalanced class scenarios, where anomalies are naturally rarer than normal conditions. The results obtained in real industrial settings show remarkable performance, with accuracy exceeding 90%, and highlight the potential of these techniques to improve the reliability, safety, and profitability of FCC units [17].

3.3.2 Work by Ge He et al.

Another recent and highly innovative approach has been proposed to improve product yield prediction in FCC units, by leveraging the combined power of deep learning and evolutionary algorithms. The authors designed a hybrid model called Differential Evolutionary Dual-Stage Attention-Based LSTM, aimed at accurately predicting yields of products such as dry gas, LPG, gasoline, and diesel. This model features two attention layers: the first identifies the most influential process parameters and catalyst properties, while the second dynamically assigns weights to the most relevant time-dependent values. To avoid the learning process becoming trapped in local minima, the model incorporates an adaptive differential evolution algorithm with a staged mutation strategy to optimize attention weights. Additionally, by applying clustering techniques to catalyst data, the authors significantly improved predictive performance, reducing Root Mean Square Errors (RMSE) by up to 95%. This study perfectly illustrates the current trend of combining LSTM networks, attention mechanisms, and heuristic optimization to accurately model highly nonlinear and dynamic industrial processes like those in FCC [18].

3.3.3 Work by Wende Tian et al.

Another noteworthy contribution in the field of anomaly prediction and early warning in FCC units is the hybrid data- and knowledge-driven approach known as DL-SDG (Deep Learning - Signed Directed Graph). This method combines the power of deep learning with the explanatory capability of signed causal graphs to anticipate abnormal conditions in highly complex industrial environments such as FCC. The approach begins with the identification of key variables using centrality measures in a complex network, thereby targeting the most critical process parameters. Dimensionality reduction is then carried out using the Spearman correlation coefficient, enabling the filtering of noisy or redundant variables and improving input data quality. The core of the model is an LSTM network enriched with attention and convolution mechanisms, designed to learn fine-grained temporal behavior of the system. A particularly original aspect of this work is the integration of the SDG model after prediction, which retraces the likely propagation path of anomalies within the system, thereby enhancing operator interpretability [19].

3.4 Conclusion

In this chapter, we explored the main prediction approaches used to model the evolution of industrial systems based on time series data. Whether based on traditional models or deep neural networks, these techniques form the foundation of modern solutions for anomaly anticipation.

Before designing our own predictive pipeline, it is essential to examine the solutions developed by the scientific community in similar contexts. This is why the following chapter presents a focused state-of-the-art review of recent works specifically applied to FCC units. This critical review will help identify current trends and best practices, as well as the most promising approaches to integrate or adapt in our own framework.

Part II: Design and Implementation

Chapter 4: Design

4.1 Introduction

Our solution must be capable of alerting engineers well before failures impact the critical indicator currently used by the Sonatrach teams.

This approach raises several questions:

Which predictive models should be selected to anticipate a distribution close to that of the actual data? Which detection models should be employed to distinguish between normal and abnormal distributions without explicit supervision? How can we capture the complex dynamics of temporal signals? How can we address the issue of strong correlations among variables? And most importantly, how can we ensure good generalization regardless of the inference data?

In this chapter, we present the different methods used to detect and predict anomalies in the time series data collected from the RFCC unit.

4.2 Data

As part of our project, we use time series data collected over a one-year period within the RFCC unit. These numerical data are obtained from industrial sensors measuring various physical quantities such as temperature, pressure, flow rate, and others. Each sensor is identified by a TAG, and the measurements are recorded at regular intervals.

We evaluated several temporal granularities before selecting a sampling step of 3 minutes. Initially, we began with data captured every 10 minutes, but this led to a significant loss of important temporal patterns and yielded too few rows to adequately train the models.
We then tested a 30-second interval to capture shorter patterns. However, this posed a technical issue: even with our lightest model, the usable sequences on GPU were limited to about twenty rows, i.e., barely 10 minutes of forecasting, which proved unprofitable and ineffective in an industrial context. Eventually, a compromise was found with a 3-minute granularity, which provided a satisfactory balance between pattern richness, dataset size, and hardware constraints.

To address the challenge of limited data volume and to enrich the training dataset, we artificially generated a large number of sequences using overlapping sliding windows, where each new sequence was shifted by one step from the previous one. This greatly increased the dataset size (30,104 * 32 = 963,328 rows) while preserving the brevity of patterns.

However, this generation method had major drawbacks. On the one hand, many sequences contained numerous identical data rows, which artificially overrepresented certain patterns. As a result, the predictive model overfitted these regions where the data distribution was relatively stationary, at the expense of its ability to generalize to other parts of the process. On the other hand, this bias induced a stationarity phenomenon in the predictions: for some sequences, the model predicted a nearly constant behavior, namely the identical repetition of the same value

across the 32 forecasting steps.

These findings led us to abandon this generation scheme in favor of a segmentation based on disjoint sequence chains, better suited to the problem's constraints and more faithful to the natural variations in industrial data.

Each data row contains a timestamp followed by dozens of measurements from various equipment in the unit. At this stage, our objective is to analyze the structure of these data in order to lay the foundation for an effective anomaly detection and prediction system.

4.2.1 Data Preprocessing

The dataset initially contained 73 variables. A preprocessing phase reduced this number to 23 variables through two complementary steps. The first involved applying statistical tools to identify and eliminate highly correlated variables, aiming to remove redundancy and reduce the dimensionality of the dataset. The second step relied on the industrial expertise of Sonatrach engineers, who identified several variables as irrelevant for diagnostic purposes: some consistently displayed negative values (indicating that the associated sensors were inactive), while others were deemed uninformative or unrelated to post-combustion or incomplete regeneration phenomena. In total, 4 variables were excluded at this stage, improving the training dataset's quality while reducing computational load.

For consistency and reliability in analysis, data from RFCC unit start-up phases were excluded. Only observations corresponding to stable operating regimes were retained, as these are considered more relevant for training models.

This reduction in the number of rows introduced discontinuities in our dataset. To overcome this issue, we identified several exploitable temporal windows. However, we deliberately selected a single window the longest one comprising 30,104 rows. This window offers the advantage of structural homogeneity and centers on typical operating conditions. Other windows, although potentially usable, included incidents related to equipment failures or transient states different from our target phenomena, which would have compromised the temporal coherence of learning. Concatenating several windows would have disrupted the integrity of sequential patterns by breaking the temporal continuity essential for proper sequential modeling. This choice ensures that our solution is applied at the right moment and in the appropriate context, specifically targeting anomalies related to post-combustion and incomplete regeneration.

Although the selected window contains only 30,104 rows, this volume proved sufficient to effectively train the proposed models. On the one hand, this choice was driven by constraints: it was the only available time window that simultaneously offered stable operating conditions, usable temporal continuity, and the absence of major external disturbances (such as hardware failures or restarts). Additionally, the end of this window (used for testing) contains enough relevant anomalies (afterburning or incomplete regeneration) to demonstrate the effectiveness of our model on unseen data. On the other hand, experimental results presented in the following chapters clearly show that this amount of data enables not only the training of high-performing models but also ensures reasonable generalization to unseen cases.

It is important to note that this dataset, although modest in size compared to some deep learning standards, holds very rich informational density: each row contains over twenty critical physical measurements, capturing the state of the industrial process at a given moment. Ultimately, the success of our approach does not lie solely in raw volume but in the relevance of the chosen window, the quality of the preprocessing, and the appropriateness of the methods used. This balance between quantity and quality was one of the key drivers of our pipeline's success, as

demonstrated by the evaluations presented in the remainder of this thesis.

We fed the training set sequences into the forecasting model to predict them entirely, sequence by sequence. These predictions were then manually smoothed by removing anomalies identified in the regenerator (using the threshold defined below), and the result was used as the training dataset for the detection model. This strategy allows us to provide the detection model with a clean dataset representative of normal situations.

The threshold applied to clean anomalies from the regenerator relies on the following key variable:

- `TI0037` with an upper threshold set at 730 °C

This threshold does not originate from the operation manual but rather from the direct expertise of process engineers. Indeed, manuals only reflect ideal conditions, which do not align with the real-world context of the Sidi Arcine unit, which has undergone multiple modifications over the years.

Furthermore, two separate MinMaxScalers were used: one for prediction and another for detection. This decision is justified by the fact that the dataset used for detection is a predicted (and smoothed) version of the original dataset. Since the statistical distributions are altered by forecasting, a separate normalization is necessary to ensure consistent scaling and processing.

Finally, the test sets were defined differently depending on the task. The prediction test set corresponds to the most recent 20% of the selected time window. For detection, a set of 1,442 rows was extracted from the test set and manually labeled by process engineers using their weekly reports (see the *Evaluation* chapter for more details). Although partial, this manual labeling provides a solid foundation for assessing the quality of the detections generated by our system.

4.3 Design of Our Solution

In this section, we present the design approach adopted to address our problem statement. This methodology is summarized in the pipeline illustrated in Figure 4.1.

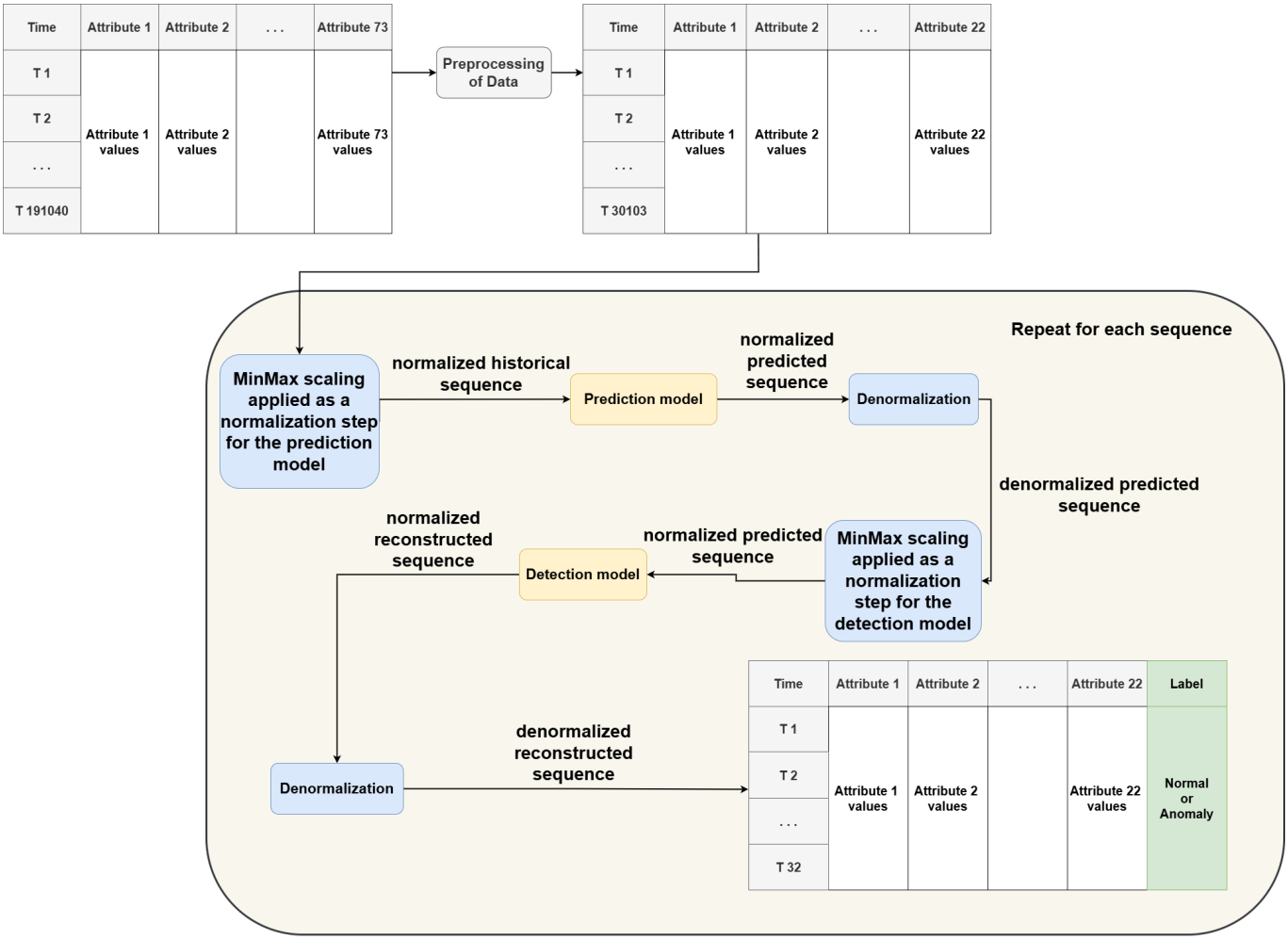


Figure 4.1: Overall architecture of our system

4.3.1 Forecasting

The notion of forecasting is central to our approach. It consists of estimating how the unit will evolve based on the observation of its past behavior, thus anticipating the occurrence of potential malfunctions. This capability is particularly important in contexts where anomalies must be detected before they manifest through noticeable deviations in traditionally monitored metrics.

In this part, we present the various forecasting models that we explored during our work.

It is also worth noting that the data used for training and validating the models were structured into sliding time windows. Each input sequence (*input_steps*) corresponds to a history of 32 consecutive time steps, from which the model is tasked with predicting the next 32 steps (*forecast_steps*).

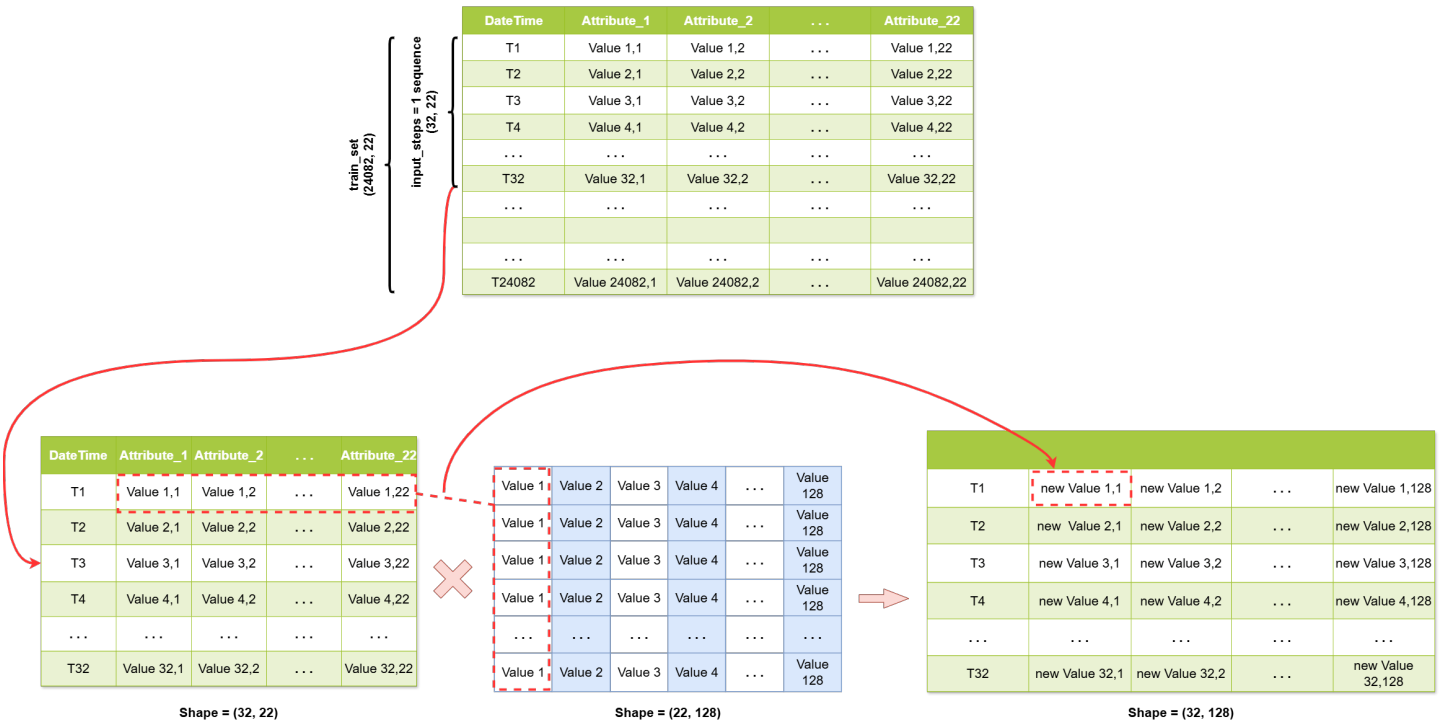
4.3.1.1 Transformer Model

In our case, we chose an encoder-only architecture without a decoder, a configuration particularly well-suited for time series data. This setup differs from the classical *Transformer* architecture, illustrated in Figure 3.4, originally designed for natural language processing (NLP) tasks. The idea is to leverage the Transformer's ability to model long-range dependencies between different sensor variables over time, while remaining flexible and effective on data that are not strictly sequentially

structured, as is often the case in industrial processes.

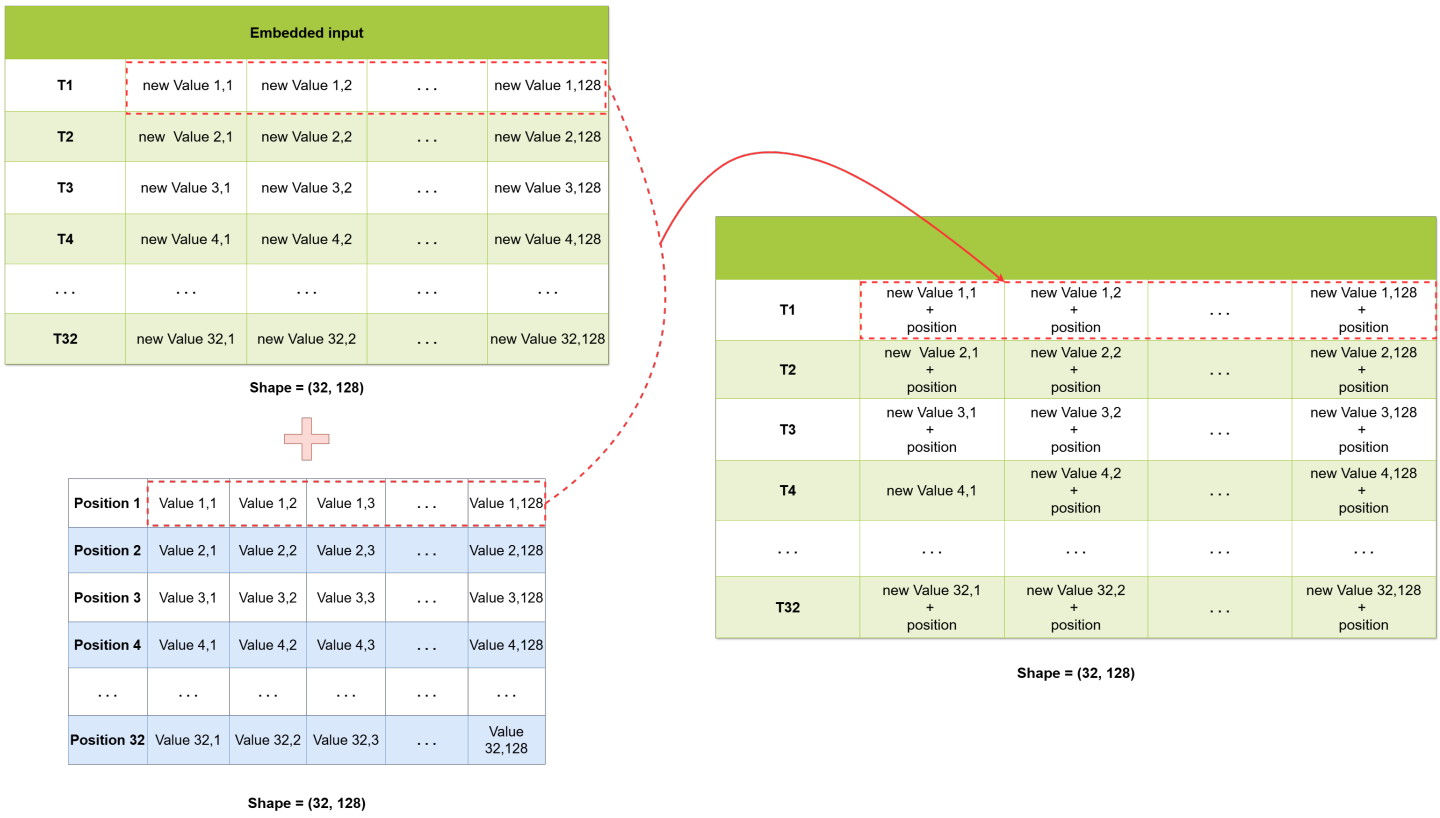
1. **Model Architecture**
   Our model is structured as follows:

   - **Before the encoder:**
     For each time step, a multivariate vector is projected into a fixed-dimensional space using an embedding dimension. For example, a 2-dimensional vector ([val1, val2]) is projected into a space of dimension "embed_dim" ([val1, val2, ..., val_embed_dim]).
     To this representation, we add a non-trainable positional encoding, allowing the model to incorporate the notion of temporal position (e.g., does the data come from day 1 or day 15?), which is crucial in a time series context.
     This process is applied independently and in parallel to each time step of the input sequence.



[16]

Figure 4.2: Illustration of linear projection



[16]

Figure 4.3: Illustration of positional encoding

- **Encoder:**
  We use two successive encoders, each being a transformer block composed of three main layers:

  – **Multi-Head Attention:**
  Multi-head attention extends the standard attention mechanism. Instead of having a single view over the sequence, the model simultaneously attends to multiple views via distinct "heads." Each head learns to focus on a specific type of relationship by applying its own transformation to the query (Q), key (K), and value (V) vectors using dedicated matrices $\left(W_Q^i, W_K^i, W_V^i\right)$ [16].
  The outputs from the heads are concatenated to form a global representation, which is then projected again, yielding an enriched view of the sequence. This enables the model to remain both flexible and powerful without significantly increasing computational complexity [16].
  The general formula is expressed as:

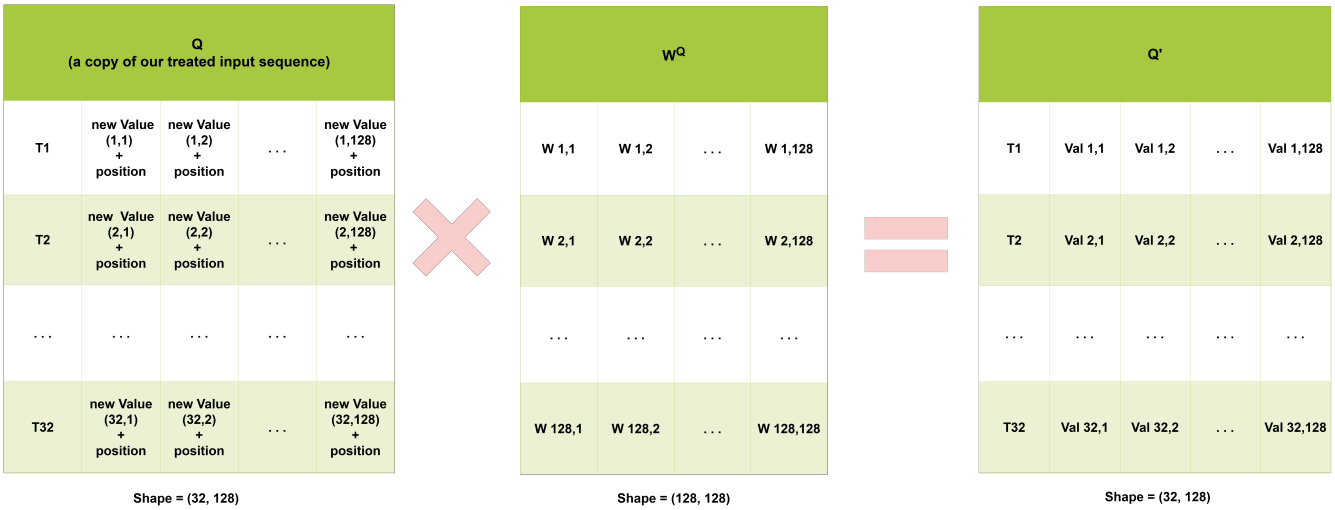
$$MultiHead(Q, K, V) = Concat(head_1, head_2, ..., head_d).W_O$$

  where

$$head_i = Attention(QW_Q^i, KW_K^i, VW_V^i)$$

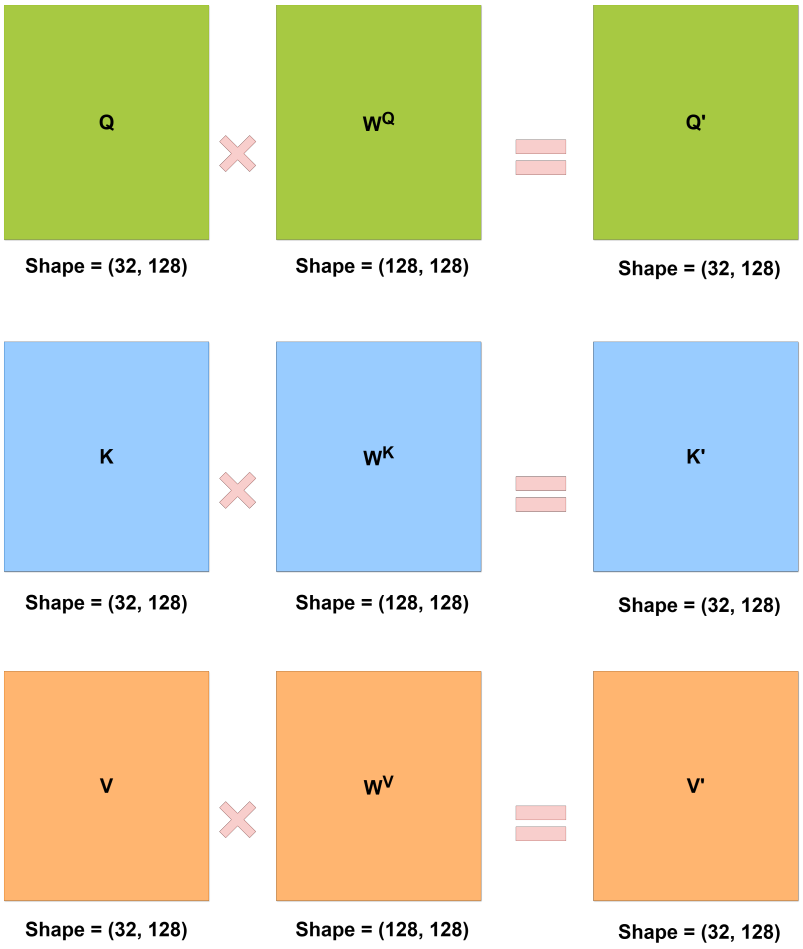
  In other words, this block enables each time step to "attend" to all others using the *self-attention* mechanism, which learns to assign different weights to contributions

from other moments. For example, when analyzing behavior at time step $t = 10$, the model might determine that $t = 2$ contains highly relevant information [16].



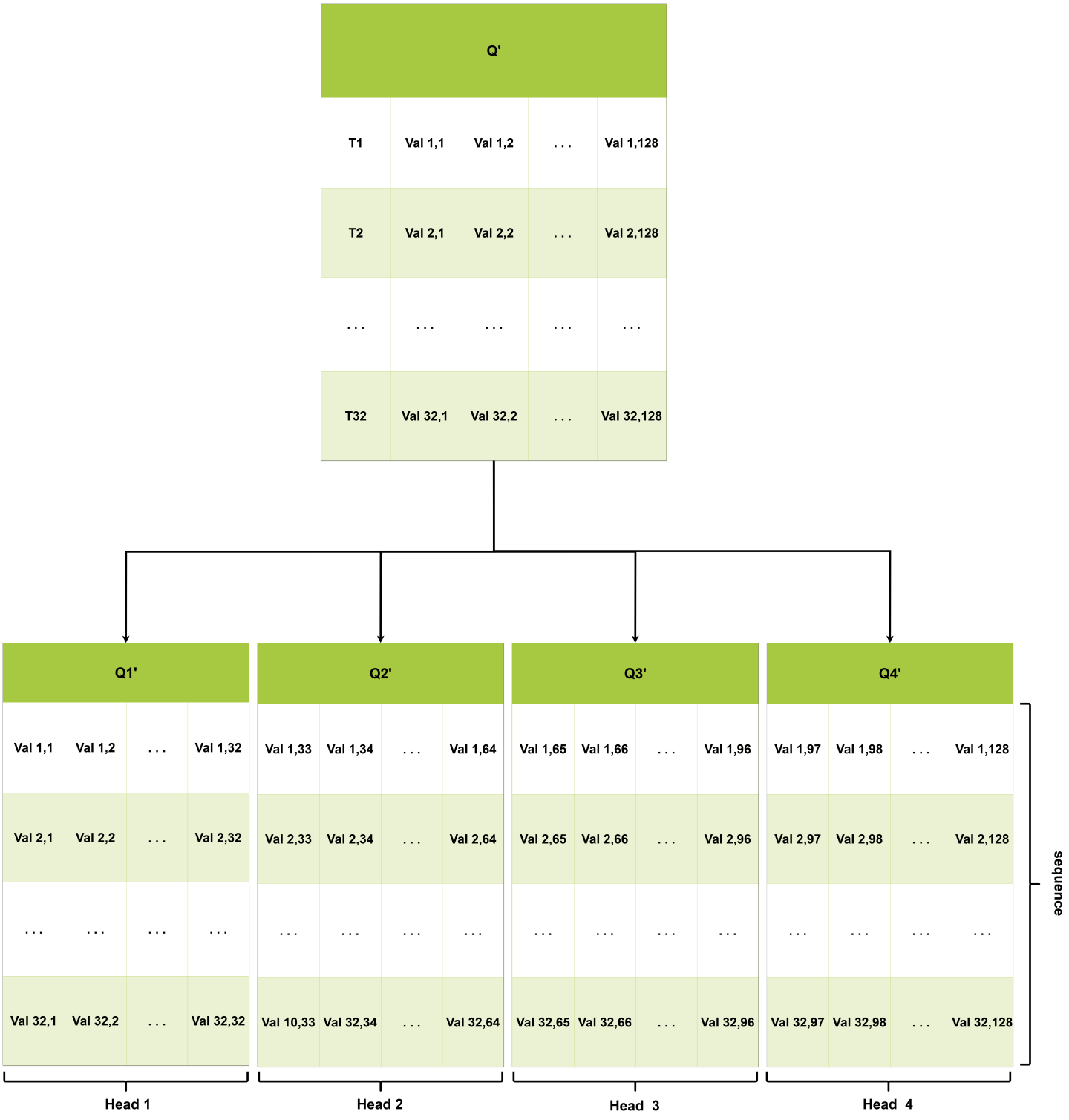
[16]

Figure 4.4: Linear projection of matrix Q



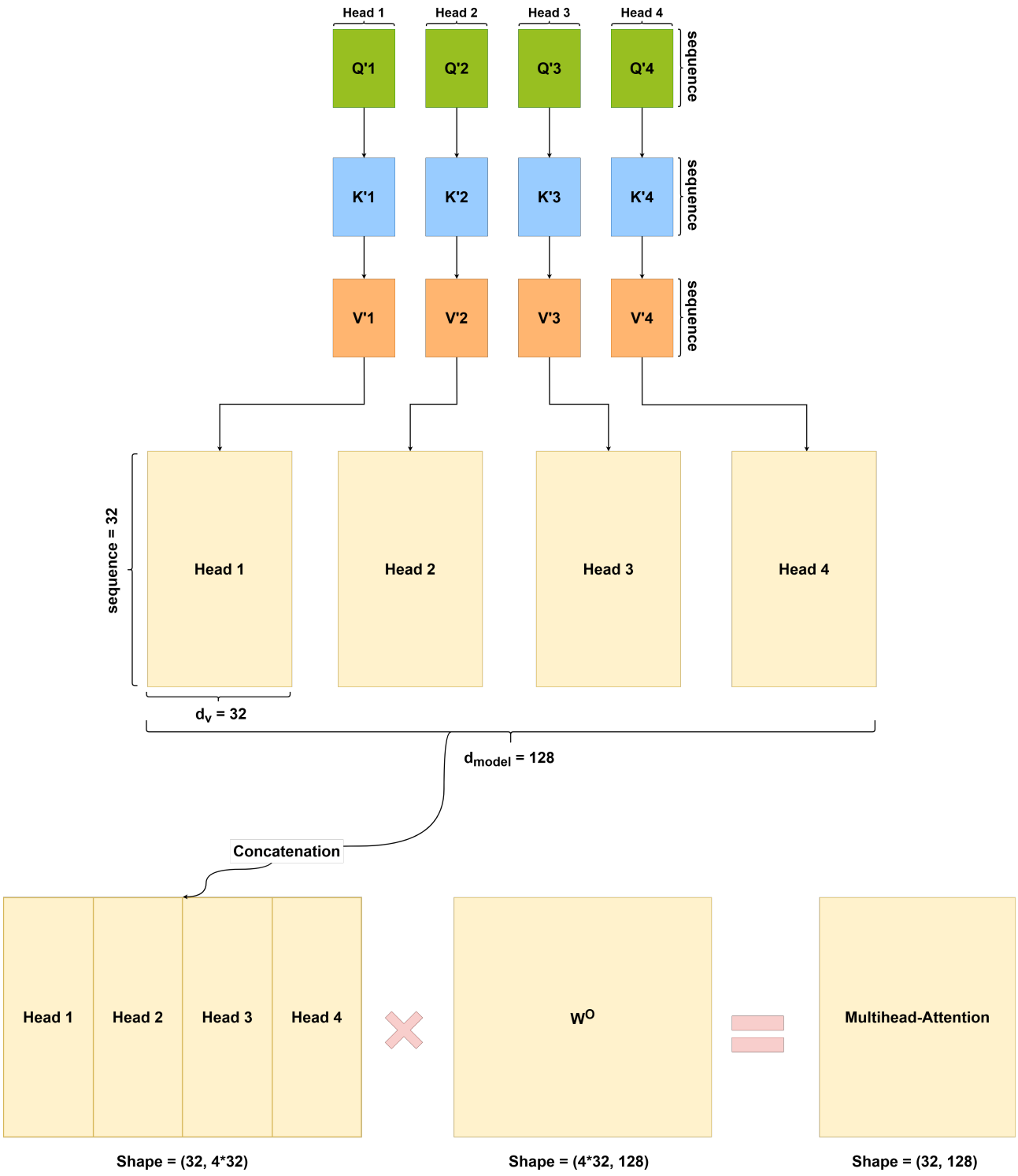
[16]

Figure 4.5: Linear projection of matrices Q, K, and V
Note: The operation shown above is replicated identically for the K and V matrices.



[16]

Figure 4.6: Splitting of Q representation into subspaces
Note: This example uses input_steps=32, nb_heads=4, and encoding=128.

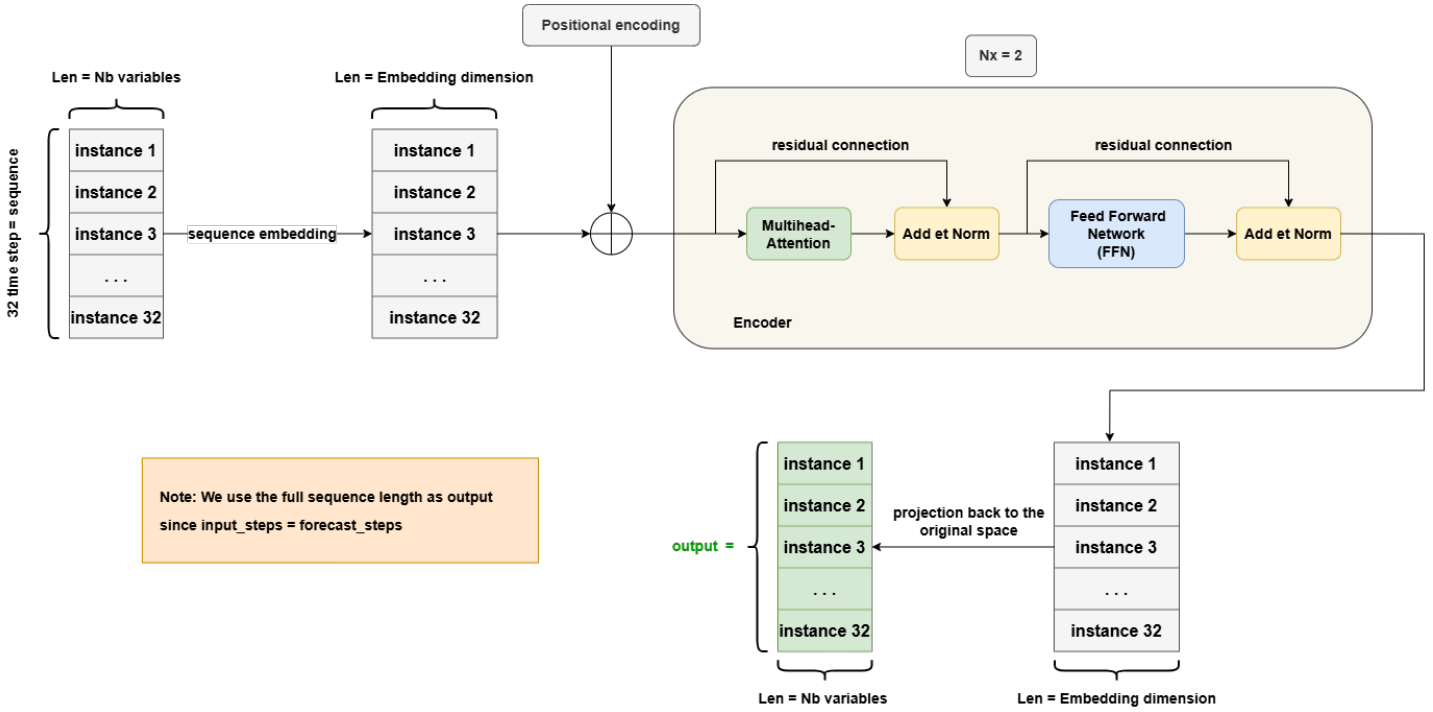


[16]

Figure 4.7: Multi-head attention mechanism inside a transformer encoder

– **Normalization:**
This layer receives the output of the previous layer, adds the residual (skip) connection, and then applies normalization to stabilize and speed up the learning process.

– **Feed Forward Network:**
Each time step is then processed independently by a small fully connected neural network (Dense → ReLU → Dense), identical for each step. The output of this network is added to a residual connection, followed by another normalization step.



[16]

Figure 4.8: Encoder-only Transformer architecture

Algorithm 1 Train_Transformer_Model

function INITIALIZE_TRANSFORMER_MODEL
Require: input sequence length L, output sequence length S, input dimension D, embedding dimension E, number of heads H, feed-forward dimension F
Ensure: Initialized model $\mathcal{M}$
    $inputs \leftarrow$ input sequence of shape (L, D);
    $x \leftarrow$ positional projection of $inputs$, shape (L, D, E);
    $x \leftarrow$ transformer encoder of shape (E, H, F);
    $x \leftarrow$ second transformer encoder of shape (E, H, F);
    $x \leftarrow$ dense layer with E units, ReLU activation;
    $x \leftarrow$ second dense layer with E units;
    $outputs \leftarrow x[:, -S:, :]$; /*Selection of forecast steps*/
    Compile $\mathcal{M}$ using `Adam` optimizer and `MSE` loss;
    return $\mathcal{M}$;
end function

Require: Training data $\mathcal{D}_{\text{train}}$, input sequence length $input\_steps$, output sequence length $forecast\_steps$, input dimension $input\_dim$, LSTM units $units$
Ensure: Trained model $\mathcal{M}$

Start
$X \leftarrow$ create sequences from $\mathcal{D}_{\text{train}}$ with window size $input\_steps$;
// starting index of $X = i$
$Y \leftarrow$ create sequences from $\mathcal{D}_{\text{train}}$ with window size $forecast\_steps$;
// starting index of $Y =$ starting index of $X + forecast\_steps$
$\mathcal{M} \leftarrow$ INITIALIZE_TRANSFORMER_MODEL($input\_steps$, $forecast\_steps$, $\dim(X)$, $units$);
Initialize `EarlyStopping` with patience 20 and restore best weights;
Train $\mathcal{M}$ on (X, Y) with 10% validation, E epochs and batch size B, without shuffle;
End

4.3.1.2 Informer Model

In this study, we rely on an architecture inspired by *Informers*, a model derived from the *Transformer*, specifically optimized for modeling long time series. The Informer encoder essentially follows the Transformer structure with its sophisticated attention mechanisms. However, our approach adopts a decoder based on an *LSTM*, forming an original hybrid that combines global attention to capture long-range dependencies with sequential memory for prediction.

1. **Model Architecture**
   Our model is structured as follows:

   - **Before encoding**
     The input is processed similarly to a *Transformer*: the sequence undergoes a *Data Embedding* phase that combines both value embedding and positional encoding.

- **Encoder**
  We use a single encoder whose internal structure is very close to that of a *Transformer*, with a few key differences outlined below:

  – **ProbSparse Self-Attention:**
  This layer allows each time step to attend to others by focusing only on the most relevant parts of the sequence, while ignoring or approximating the rest. More precisely, instead of treating all queries equally, only the most informative ones are retained. To determine which queries are most useful, a sparsity probability is used, based on the *Kullback-Leibler divergence* (KL divergence) between each query vector and a uniform distribution. Queries that deviate most from this distribution (i.e., those carrying the most information) are preserved [16].
  This technique reduces the computational complexity of the Attention mechanism while maintaining high performance [16].

Step	**Operation**	**Tensor Shape**
1	Inputs Q, K, V	(B, L, D_{model})
2	Linear projections: $Q' = QW^Q$, $K' = KW^K$, $V' = VW^V$	(B, L, D_{model})
3	Head splitting	(B, H, L, D_h) where $D_h = D_{\text{model}}/H$
4	Dot products $QK^\top/\sqrt{D_h}$	(B, H, L, L)
5	Sparsity measure (max - mean)	(B, H, L)
6	Top-k query selection based on sparsity	(B, H, k)
7	Masking of unselected queries	(B, H, L, L)
8	Softmax on filtered scores	(B, H, L, L)
9	Attention $\times$ V	(B, H, L, D_h)
10	Head concatenation	(B, L, D_{model})
11	Final projection (Dense layer)	(B, L, D_{model})

Table 4.1: Steps of the ProbSparse Multi-Head Attention mechanism with tensor shapes

Where B is the batch size, L is the sequence length, H the number of attention heads, D_{model} the global embedding dimension, and D_h the per-head dimension (i.e. D_{model}/H). k is the number of selected queries based on sparsity.

  – **Normalization:**
  The outputs are added to a residual connection, and the sum is then normalized, as in standard *Transformers*.

  – **Distilling Encoder Layer:**
  Inspired by the Informer architecture, our implementation relies on two parallel Conv1D layers.

  The first convolution acts as an intelligent filter: with a kernel size of 3 and same padding, it scans the sequence, capturing local patterns while smoothing out micro-

variations. The following ReLU acts as a high-pass filter, removing noise while preserving the essential signal.

The second convolution; its twin; performs a second pass over the already transformed features. Like an editor refining a previously reviewed text, it fine-tunes the representations by detecting subtler relationships. This duplication is not redundant: by stacking two identical convolutions, we achieve a progressive deepening effect, much like binocular vision, where each layer offers a distinct perspective on the same data.

Though different from the original Informer distilling approach, this method offers an ideal compromise between implementation simplicity and effectiveness for moderately sized datasets. The surrounding normalization and residual connections act as safeguards, ensuring each new transformation preserves essential information.

- **Decoder**

  In its original version, the *Informer* employs a decoder based on the *Transformer* architecture a powerful yet resource-intensive choice. It begins with an initial signal (a simple "start token") and progressively incorporates the global context provided by the encoder. Its strength lies in a dual attention mechanism: on one hand, it models the relationships between the different forecast time steps (self-attention), and on the other, it continuously aligns with the encoder outputs (cross-attention).

  Our adaptation introduces a more pragmatic alternative by replacing this Transformer-based decoder with a single-layer *LSTM* decoder akin to a craftsman opting for more specialized tools. Here's how it works: the encoder delivers its distilled knowledge as a context vector (h_T, c_T). Instead of leveraging complex attention mechanisms, we simply replicate this capsule for each forecast time step. The *LSTM* then acts as a careful interpreter, processing this information sequentially while preserving its temporal richness. Finally, a final transformation layer (*TimeDistributed*) reshapes the output to match the specific requirements of our forecasting task.

  This hybrid design combines the best of both worlds: the global insight of the *Transformer* encoder and the temporal sensitivity and efficiency of the *LSTM* decoder. This makes the model particularly well-suited for time series with pronounced local patterns or when computational resources are limited. Our model is thus capable of capturing essential features while maintaining an agile and efficient decoding process.

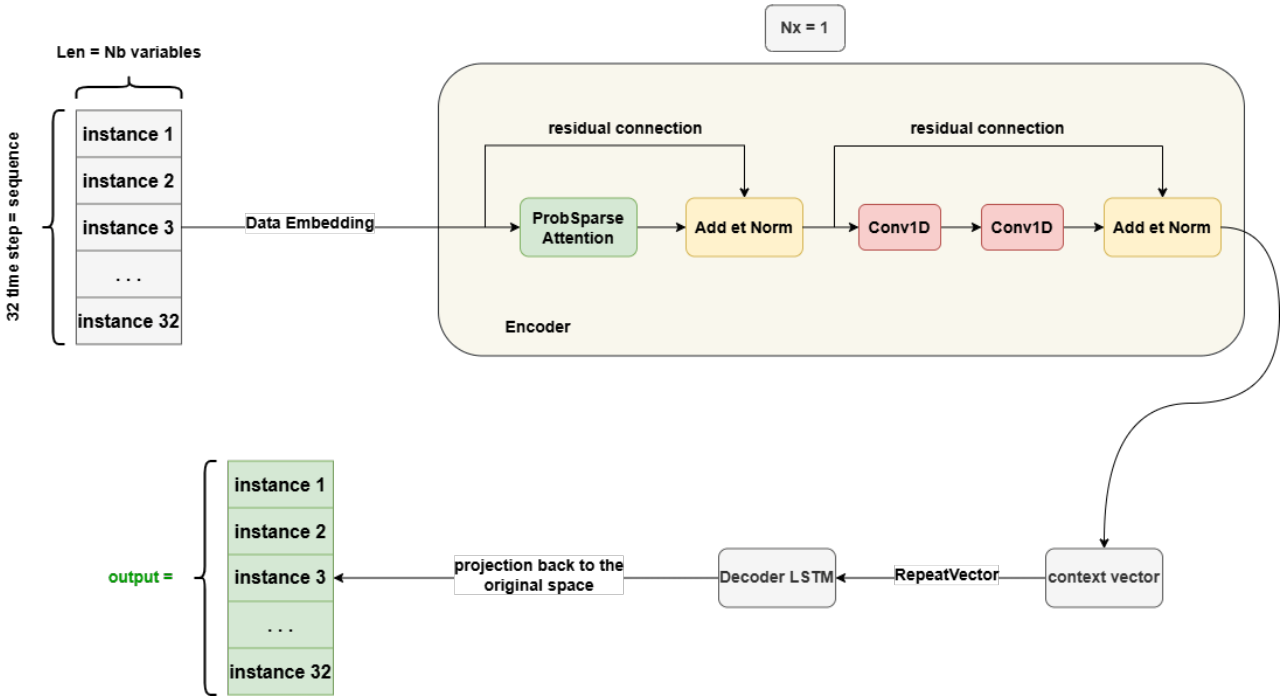


Figure 4.9: Architecture of our Informer-based model

Algorithm 2 Train_Informer_Model

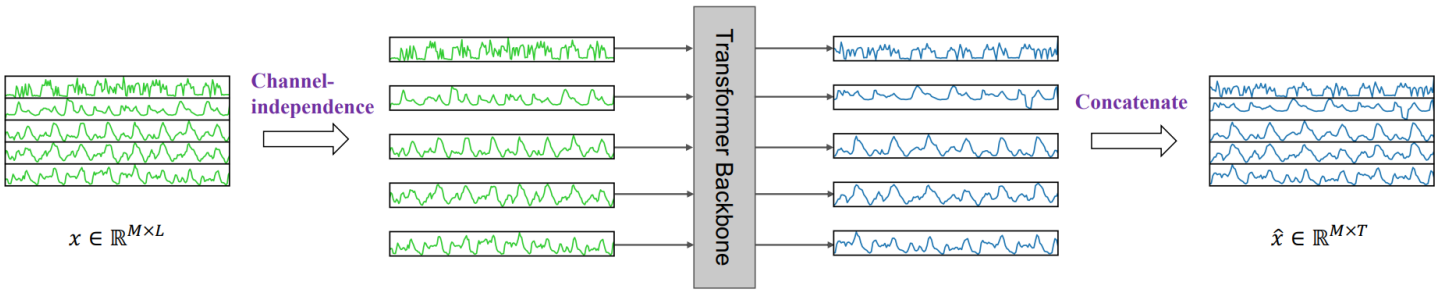
function INITIALIZE_INFORMER
Require: input sequence length L, output sequence length S, input dimension D, embedding dimension E, number of heads H
Ensure: Initialized model $\mathcal{M}$
    $inputs \leftarrow$ input sequence of shape (L, D);
    $x \leftarrow$ dense projection to E dimensions;
    $attn\_out \leftarrow$ probabilistic Multi-head Attention with H heads and top-k $= K$
    $x \leftarrow x + attn\_out$; //residual connection
    $x \leftarrow$ layer normalization;
    $x \leftarrow$ two 1D convolutions with E filters, kernel size 3, `ReLU` activation;
    $x \leftarrow$ duplication of the last time step across S steps using `RepeatVector`;
    $x \leftarrow$ decoding with LSTM layer of E units, return sequences;
    $outputs \leftarrow$ dense projection to D using `TimeDistributed`;
    $\mathcal{M} \leftarrow$ `Model`($inputs$, $outputs$);
    Compile $\mathcal{M}$ with `Adam` optimizer and `MSE` loss;
    return $\mathcal{M}$;
end function

Require: Training data $\mathcal{D}_{\text{train}}$, input sequence length $input\_steps$, output sequence length $forecast\_steps$, input dimension $input\_dim$, LSTM units $units$
Ensure: Trained model $\mathcal{M}$

Begin
$X \leftarrow$ create sequences from $\mathcal{D}_{\text{train}}$ with window size $input\_steps$;
// start_index_X = i
$Y \leftarrow$ create sequences from $\mathcal{D}_{\text{train}}$ with window size $forecast\_steps$;
// start_index_Y = start_index_X + forecast_steps
$\mathcal{M} \leftarrow$ INITIALIZE_INFORMER($input\_steps$, $forecast\_steps$, $\dim(X)$, $units$);
Initialize `EarlyStopping` with patience 20 and best weights restoration;
Train $\mathcal{M}$ on (X, Y) with 10% validation, E epochs and batch size B, no shuffle;
End

4.3.1.3 Patch Time Series Transformer Model (PatchTST)

The forecasting model explored in this phase is the *PatchTST* (Patch Time Series Transformer), which stands out for its innovative architecture based on *Transformers*. This model has been specifically designed for multivariate time series forecasting by introducing two key mechanisms: **patching** (segmenting data into temporal blocks) and **channel-independence** (independent processing of channels) [20].

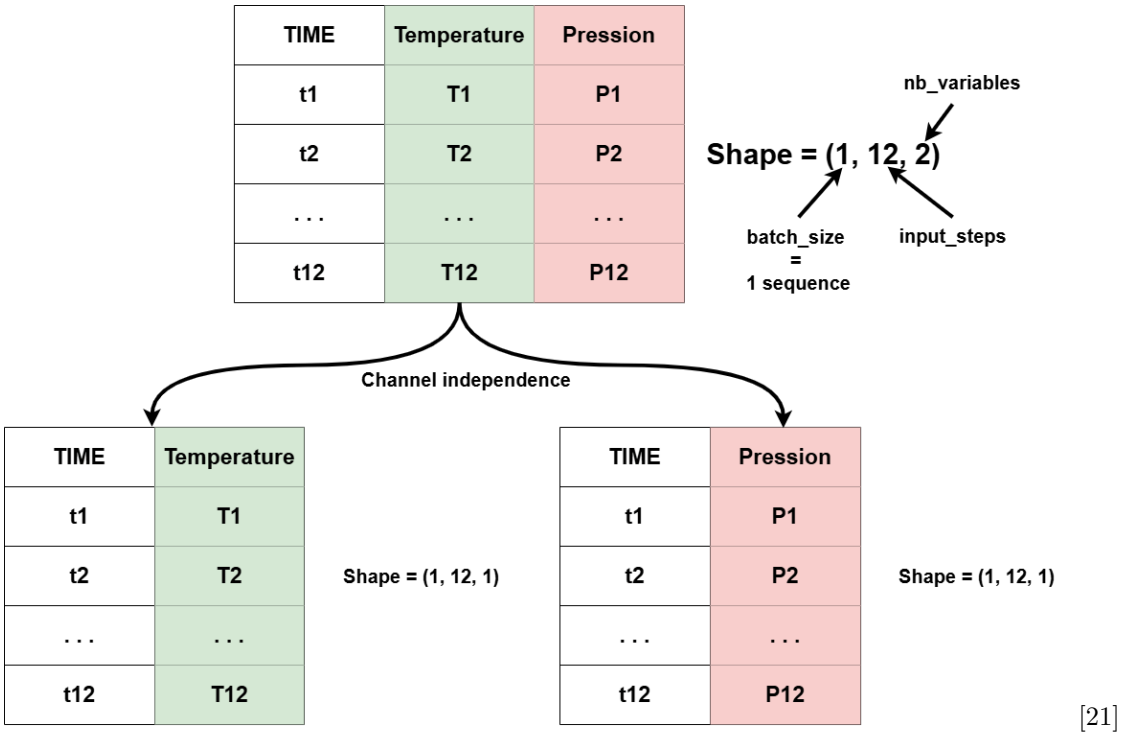


[21]

Figure 4.10: Global view of the PatchTST model

1. **Model Architecture**
   We now describe the structure of the model:

   - **Channel independence:**
     Channel independence is a key concept of *PatchTST*, allowing each channel (variable) to be processed independently, thus preserving its unique characteristics throughout the process [21].

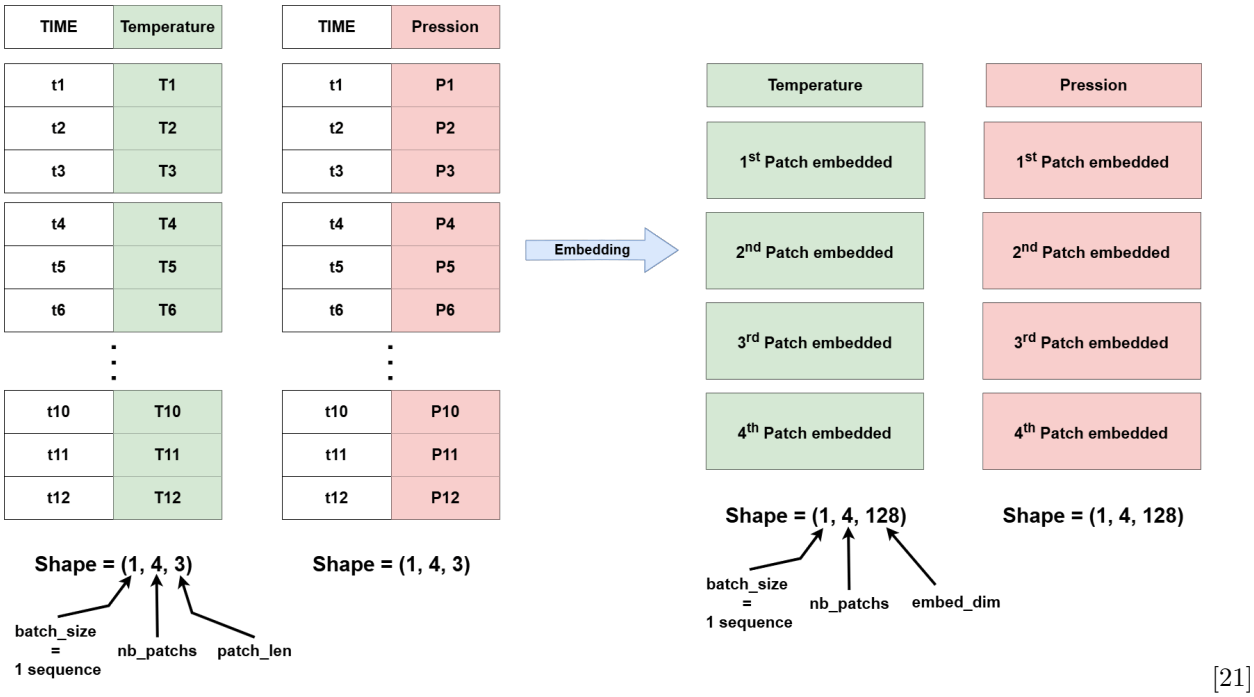


[21]

Figure 4.11: Illustration of the channel independence concept

   - **Patching:**
     The model applies another key innovation: slicing each sequence into time segments (patches) of *patch_len* steps.
     Thanks to *channel independence*, this transformation is performed separately for each

variable, avoiding any artificial mixing between potentially uncorrelated signals [20]. These patches are then projected, one by one, into a fixed-dimensional space defined by *embed_dim*, thereby revealing complex patterns not visible in raw data. The resulting data take the shape *(batch_size * num_features, num_patches, embed_dim)*, e.g., in Figure 4.12, the data go from shape (1*2, 4, 3) to (1*2, 4, 128).



[21]

Figure 4.12: Illustration of the patching mechanism followed by encoding

- **Encoder:**
  The representations obtained from the previous two steps are then processed by a *Transformer* block, applied independently to each variable. This block uses the same components as previously described, ensuring rich encoding while preserving each channel's specific structure.
  Throughout this phase, the tensor shape remains unchanged: *(batch_size * num_features, num_patches, embed_dim)*. At the end of the block, each patch of each variable is represented by a contextualized vector, integrating relevant temporal information without introducing channel mixing.

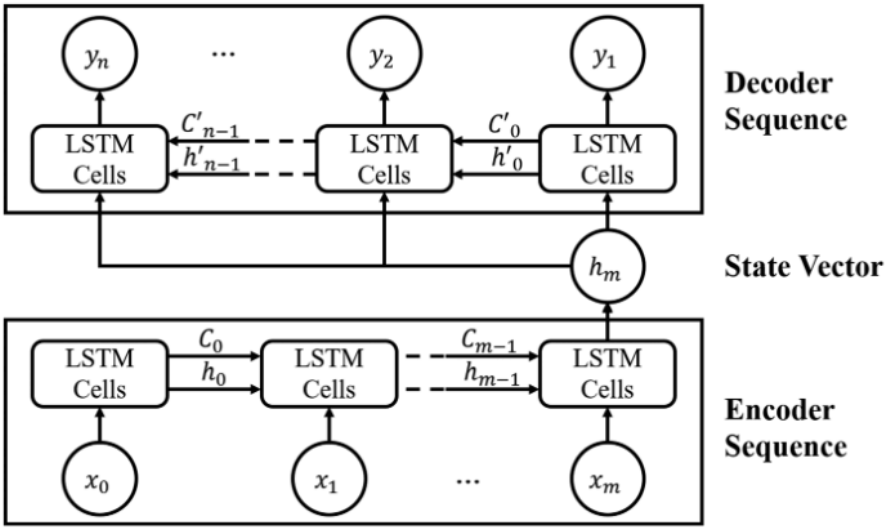
- **Outputs:**
  After encoding, the model selects only the last patch of each variable, assumed to carry the most recent and relevant information for prediction. This time reduction step yields a tensor of shape *(batch_size * num_features, embed_dim)*, which is then reshaped to *(batch_size, num_features, embed_dim)*.
  Each resulting vector is then projected via a dense layer into a temporal sequence of length equal to the number of forecasting steps, denoted as *forecast_steps*. The output shape becomes *(batch_size, num_features, forecast_steps)*, and is finally transposed into the standard format *(batch_size, forecast_steps, num_features)*.
  The model therefore produces, for each batch sample, a multivariate prediction over the next *forecast_steps*, maintaining explicit channel separation throughout the process.

Step	Tensor shape
Raw input	(batch_size, input_steps, num_features)
Reshape for channel independence	(batch_size × num_features, input_steps, 1)
Patching	(batch_size × num_features, num_patches, patch_len)
Projection (embedding)	(batch_size × num_features, num_patches, embed_dim)
Output from Transformer encoder	(batch_size × num_features, num_patches, embed_dim)
Select last patch	(batch_size × num_features, embed_dim)
Reshape to batch + channel	(batch_size, num_features, embed_dim)
Projection to forecast steps	(batch_size, num_features, forecast_steps)
Final transposition (model output)	(batch_size, forecast_steps, num_features)

Table 4.2: Summary of PatchTST model steps and successive tensor transformations

4.3.1.4 LSTM Sequence-to-Sequence Model (Seq2Seq)

The prediction model explored in this phase is based on a sequence-to-sequence *LSTM* architecture. It follows an Encoder-Decoder structure, which is well-suited for multivariate sequential forecasting tasks. It is composed of two main modules: an encoder and a decoder, both built with *LSTM* cells. The main principle is to encode an input sequence (of fixed length) into a latent vector and then decode it into an output sequence of the same size.



[22]

Figure 4.13: General architecture of LSTM seq2seq

1. **Model Architecture**
   The model is structured as follows:

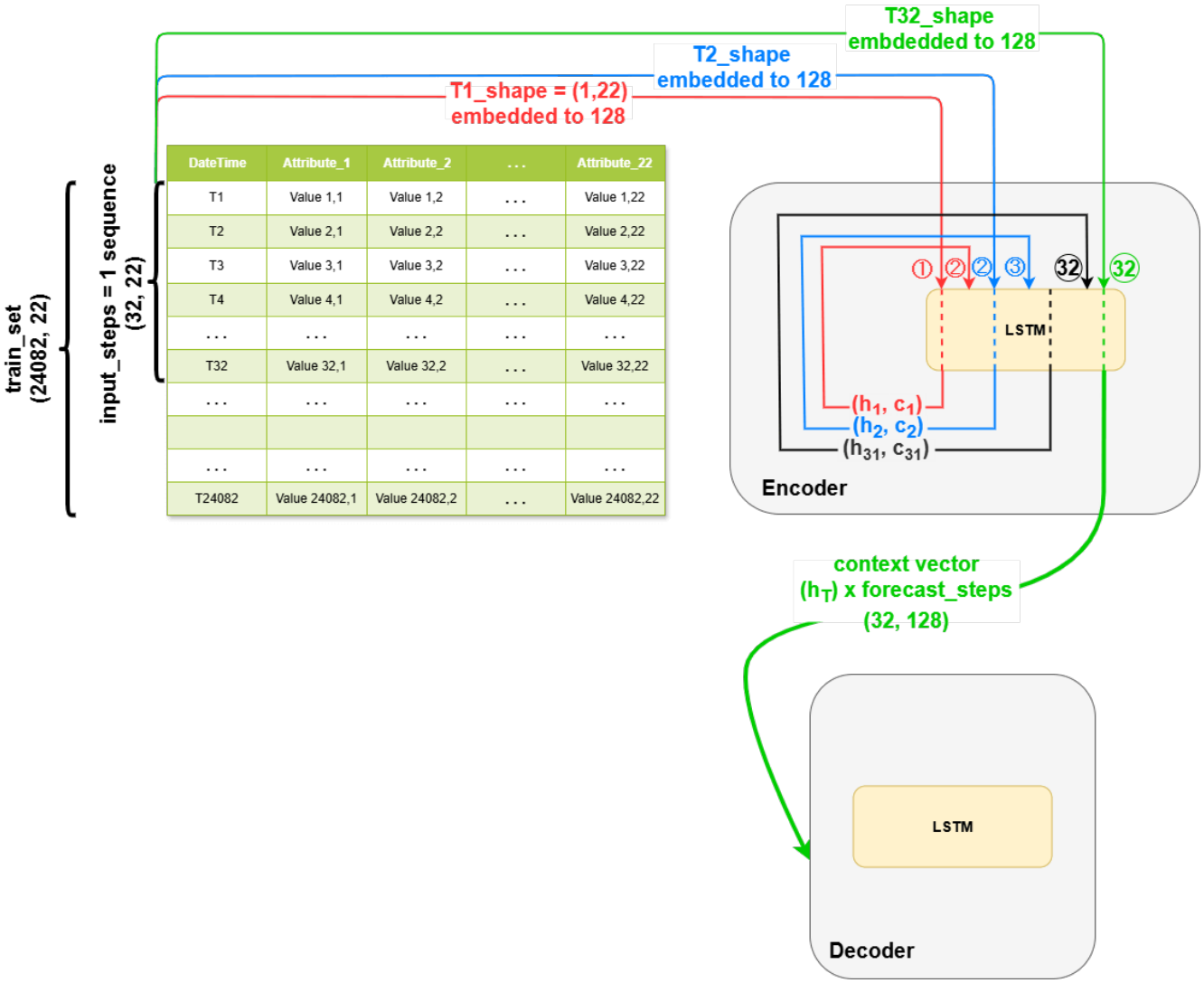
   - **Encoder:**
     The encoder consists of a single *LSTM* layer. It sequentially processes an input sequence of shape (32, 22), where each time step corresponds to a 22-dimensional feature

vector. In practice, this means it analyzes 32 consecutive steps, each representing a snapshot of the system's state.

At each step, the LSTM maps the input vector (1, 22) into a latent space of dimension *embed_dimension*, equivalent to the number of LSTM units. This operation can be seen as an intelligent translation, where raw observations are transformed into informative representations.

The mechanism relies on dynamic memory: the layer receives the current input as well as its previous internal states (h_{t-1}, c_{t-1}), and produces updated states (h_t, c_t). The hidden state h_t captures short-term patterns, while the cell state c_t encodes long-term dependencies. The initial states h_0 and c_0 are initialized to zero.

At the end of this process, only the final hidden state is retained: the context vector h_T (1, *embed_dimension*). This design choice ensures that the full sequence is condensed into a single vector. Note that with $return\_sequences = False$, intermediate states are discarded in favor of this final summary.



[22]

Figure 4.14: Encoder architecture of our LSTM seq2seq model
Note: Example shown with *input_steps*=32 and *embed_dimension*=128.

- **Context Vector:**
  The resulting h_T from the encoder is duplicated as many times as the number of time steps to predict (32 times). This results in a sequence of shape (32, *embed_dimension*), where each time step to forecast receives the same context vector h_T.
  This duplication allows the decoder to access a consistent context at each prediction step.

- **Decoder:**
  The decoder, also composed of a single *LSTM* layer, receives as input the sequence of duplicated context vectors of shape (32, *embed_dimension*). At each step, the decoder processes the same input vector h_T along with the evolving internal states (h_{t-1}, c_{t-1}). Although the inputs remain constant, the internal state evolution enables the decoder to generate a coherent output sequence.
  Unlike traditional *Teacher Forcing*, this model does not feed the actual past outputs during training. In contrast, a *Teacher Forcing* strategy would use h_T only to initialize

the decoder states (h_0, c_0), and would then input the true output of each prior step (see Figure 4.15).

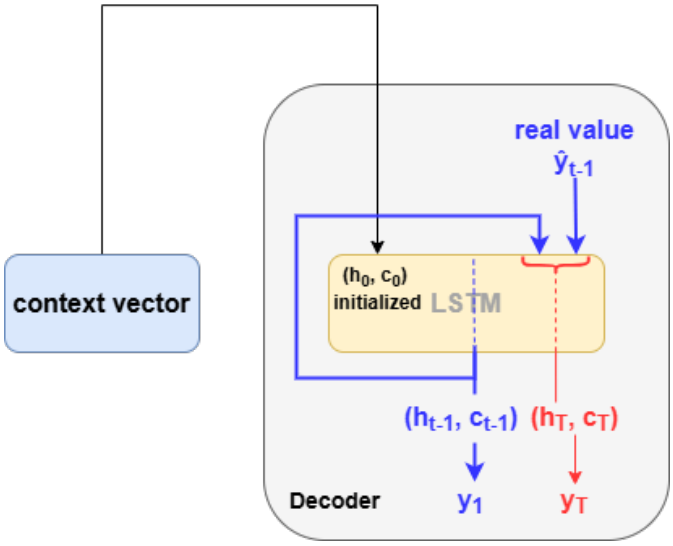


Figure 4.15: Decoder with Teacher Forcing mechanism

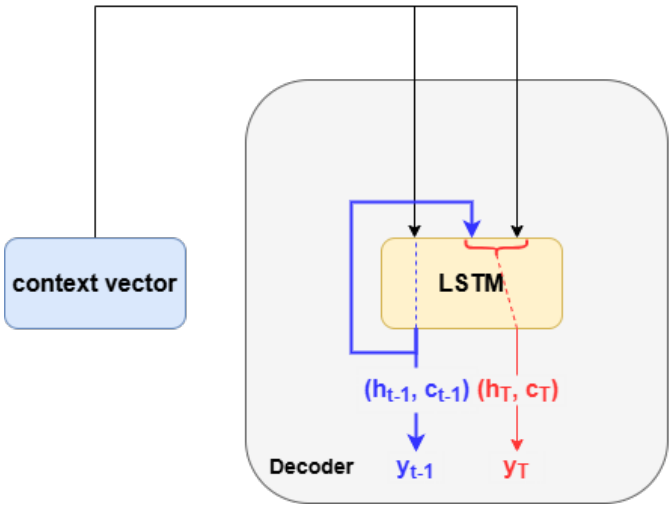


Figure 4.16: Decoder with no Teacher Forcing

- **Output:**
  The decoder outputs vectors of shape (1, *embed_dimension*). To transform them back into the same shape as the original input variables, a Dense layer is applied.
  Since the output is a sequence, the TimeDistributed(Dense(22)) layer is used to apply the same Dense transformation at each time step without needing to loop manually.

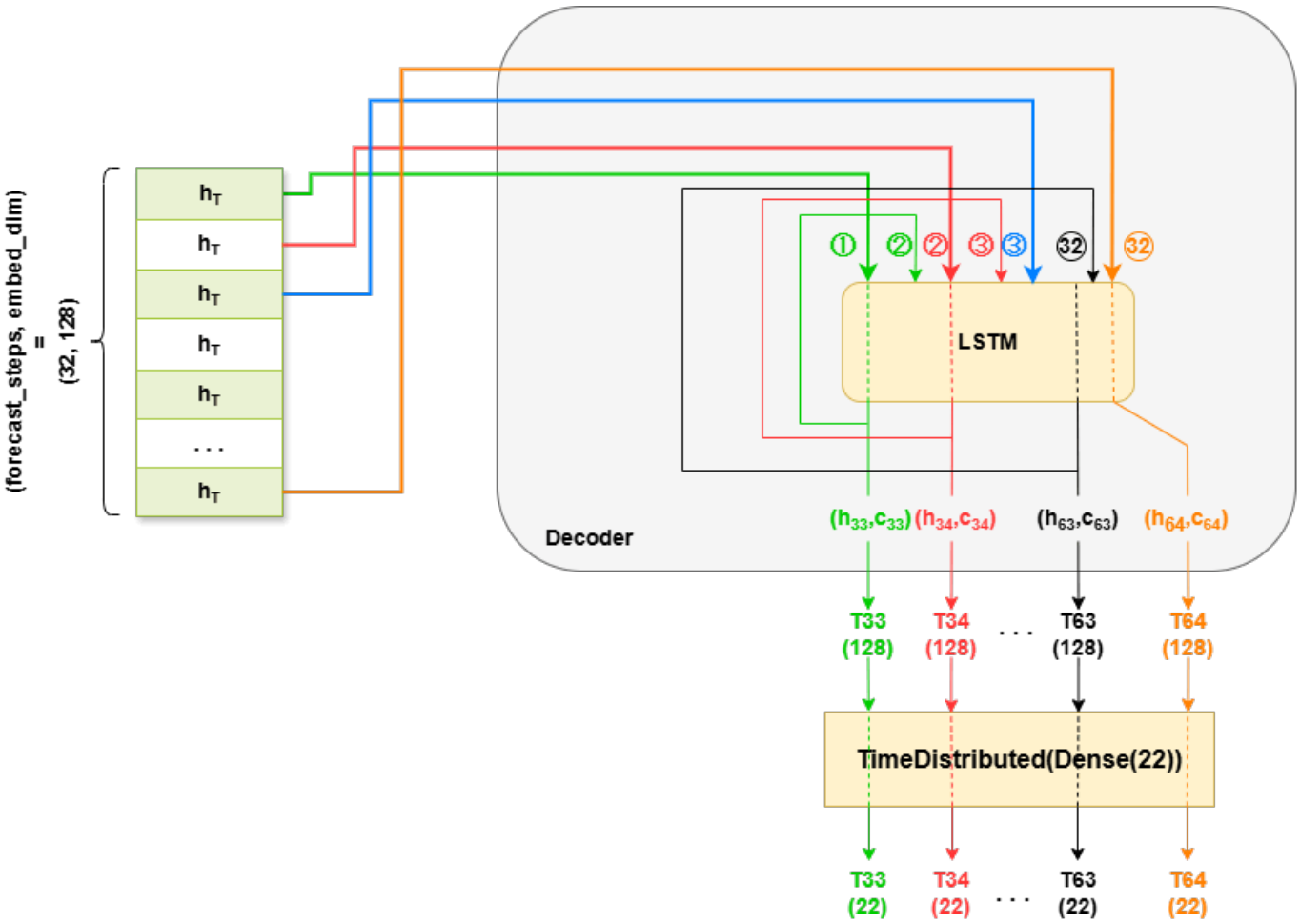


Figure 4.17: Decoder mechanism of our LSTM seq2seq model

Algorithm 3 Training_LSTM_Seq2Seq_Model

function INITIALIZE_LSTM_SEQ2SEQ
Require: Input sequence length L, output sequence length S, input dimension D, LSTM units U
Ensure: Initialized model $\mathcal{M}$
    $inputs \leftarrow$ input sequence of shape (L, D);
    $x \leftarrow$ LSTM encoding of $inputs$ with U units;
    $x \leftarrow$ duplicate x using `RepeatVector` over S time steps;
    $x \leftarrow$ LSTM decoding of x with U units;
    $outputs \leftarrow$ reconstructed output via `TimeDistributed(Dense(`D`))`;
    $\mathcal{M} \leftarrow$ Model($inputs$, $outputs$);
    Compile $\mathcal{M}$ using `Adam` optimizer and `MSE` loss;
    return $\mathcal{M}$;
end function

Require: Training data $\mathcal{D}_{\text{train}}$, input sequence length $input\_steps$, output sequence length $forecast\_steps$, input dimension $input\_dim$, LSTM units $units$
Ensure: Trained model $\mathcal{M}$

Begin
$X \leftarrow$ generate input sequences from $\mathcal{D}_{\text{train}}$ with window size $input\_steps$;
$Y \leftarrow$ generate output sequences from $\mathcal{D}_{\text{train}}$ with window size $forecast\_steps$;
$\mathcal{M} \leftarrow$ INITIALIZE_LSTM_SEQ2SEQ($input\_steps$, $forecast\_steps$, $\dim(X)$, $units$);
Initialize `EarlyStopping` with patience 20 and restore_best_weights=True;
Train $\mathcal{M}$ on (X, Y) with 10% validation, E epochs, batch size B, without shuffle;
End

4.3.1.5 Customized PatchTST Model

The model presented here is a customized version of *PatchTST* adapted to our dataset. In this variant, the *channel-independence* mechanism has been deliberately removed due to the strong correlation between the features, and a *LSTM*-based decoder has been integrated to improve forecasting capabilities.

1. **Model Architecture**
   We now describe the architecture of our adapted *PatchTST* model:

   - **Patch Division:**
     The patching process is executed as described for the standard *PatchTST* model.
   - **Transformer Encoder:**
     The Transformer block, referred to as the *backbone Transformer*, is then applied using its sophisticated attention mechanism. This block is identical to the one previously detailed in the *Transformer* section, except that the *Feed Forward Network* is excluded.
   - **Global Pooling:**
     After the attentive processing of the patches, we apply a *Global Average Pooling* layer that acts as an information condenser. This crucial step computes the average of the

features across all patches, yielding a unique signature that captures the dominant patterns in the sequence. Similar to an expert summary, it distills the essential information while reducing dimensionality, effectively preparing the data for the forecasting phase.

- **LSTM Decoder:**
  The model is adapted to our dataset by integrating a decoder based on a single-layer *LSTM* architecture. This decoder generates predictions in an autoregressive manner, starting from the compressed global representation, which is repeated *forecast_steps* times using a *RepeatVector* layer to produce the full temporal output.
  The detailed operation of the LSTM decoder and the output generation have been explained in the previous section.

Algorithm 4 Train_Customized_PatchTST_Model

function INITIALIZE_CUSTOM_PATCHTST(L, D, P, E, H, S)
Require: L: input sequence length, D: number of features, P: patch length, E: embedding dimension, H: number of attention heads, S: output sequence length
Ensure: Initialized model $\mathcal{M}$
  Ensure L is divisible by P
  $N_p \leftarrow L/P$ ▷ Number of patches
  $inputs \leftarrow$ input sequence of shape (L, D)
  $patches \leftarrow$ reshape $inputs$ to $(N_p, P \times D)$
  $embedded \leftarrow$ Dense(E) applied to each patch
  $attn\_output \leftarrow$ `MultiHeadAttention`($num\_heads = H$, $key\_dim = E$) on $embedded$
  $x \leftarrow$ Add($embedded$, $attn\_output$)
  $x \leftarrow$ LayerNormalization(x)
  $x \leftarrow$ `GlobalAveragePooling1D`(x)
  $x \leftarrow$ `RepeatVector`(S) of x
  $x \leftarrow$ `LSTM`(E, return_sequences=True) on x
  $output \leftarrow$ `TimeDistributed(Dense(`D`))` applied to x
  $\mathcal{M} \leftarrow$ Model($inputs$, $output$)
  Compile $\mathcal{M}$ with `Adam` optimizer and `MSE` loss
  return $\mathcal{M}$
end function

Require: Training data $\mathcal{D}_{\text{train}}$, hyperparameters $(L, S, D, P, E, H, B, epochs)$
Ensure: Trained model $\mathcal{M}$

Start
$X \leftarrow$ generate input sequences of length L from $\mathcal{D}_{\text{train}}$
$Y \leftarrow$ generate aligned target sequences of length S from $\mathcal{D}_{\text{train}}$
$\mathcal{M} \leftarrow$ INITIALIZE_CUSTOM_PATCHTST(L, D, P, E, H, S)
Initialize `EarlyStopping` with patience 20 and restore_best_weights=True
Train $\mathcal{M}$ on (X, Y) using validation_split $= 0.1$, epochs $= epochs$, batch_size $= B$, no shuffling
return $\mathcal{M}$
End

4.3.2 Anomaly Detection

Anomaly detection might initially be viewed as a simple binary classification task (anomaly or not). However, this interpretation is overly simplistic. In our approach, the *Autoencoder* model based on *LSTM* does not directly classify data. Instead, it learns to faithfully reconstruct the sequences it receives, based on what it has learned from the normal behavior of the system.

What the model effectively learns is a sequence-to-sequence transformation, known as transduction. Only after this step, by comparing the input sequence to its reconstruction, can significant deviations be identified these deviations are indicative of potential anomalies. Thus, the final decision (normal or abnormal) relies on reconstruction errors, and not on direct classification, which clearly distinguishes this method from conventional classifiers. The primary objective here is modeling and reproducing normal behavior, not predicting labels.

Regarding model selection, Transformer-based architectures were deliberately excluded for detection. While they are powerful and cutting-edge in many sequential applications, they are better suited for tasks such as long-term multivariate forecasting, where capturing global dependencies is critical. In contrast, in anomaly detection scenarios where the goal is to faithfully model normal behavior over relatively short windows LSTM-based autoencoders remain a robust, proven, and more suitable option (at least currently).

LSTM models are also lighter in computational cost compared to Transformers, which facilitates their training and integration into real-time industrial pipelines. They have also demonstrated more stable reconstruction behavior on our dataset and offer more interpretable temporal error dynamics. Therefore, the LSTM autoencoder naturally emerged as the most appropriate choice for our industrial context, ensuring both robustness and efficiency.

4.3.2.1 BiLSTM Autoencoder

In this autoencoder version of the *LSTM*, the core principle is to transform an input sequence into a condensed latent representation using a two-layer encoder: the first layer is a unidirectional LSTM, and the second is a *Bidirectional LSTM* (BiLSTM). This encoder block is followed by a decoder consisting of a single LSTM layer tasked with reconstructing the original sequence from the latent representation.

Unlike the prediction model (cf. §4.3.1.4), which maps a sequence to a future output (X→Y), here, the network reconstructs the same input sequence (X→X).

The idea is simple yet powerful: a model trained solely on normal behavior becomes excellent at reconstructing it, but fails when patterns deviate a common occurrence during faults or system malfunctions. This makes the method particularly effective in settings where anomalies are rare or poorly annotated, as is often the case in industrial systems.

The encoder receives an input sequence of shape $(32, 22)$ representing multivariate time series data from our forecasting model. It applies a first LSTM layer (units=128, return_sequences=True), followed by a Bidirectional LSTM layer (units=128, return_sequences=False). This BiLSTM processes the sequence forward and backward, returning a single output vector (the concatenation of the final forward and backward hidden states), capturing global dependencies.

This context vector is repeated 32 times using a `RepeatVector` layer to form a fixed-length input for the decoder. The decoder LSTM (units=128, return_sequences=True) processes this and attempts to reconstruct the original sequence. Finally, a `TimeDistributed(Dense(22))` layer is applied to output the reconstructed values for each time step.

This architecture is trained using the Mean Squared Error (MSE) between the original input

and its reconstruction.

We opted for the bidirectional encoder because, unlike forecasting where chronology is essential, reconstruction benefits from a global view of the sequence in both directions. This additional complexity also prevents the model from becoming too simplistic, allowing it to build richer intermediate representations before reaching the bottleneck.

Algorithm 5 Train_BiLSTM_Autoencoder_for_Anomaly_Detection

function INITIALIZE_BILSTM_AUTOENCODER
Require: Sequence length L, input dimension D, LSTM units U
Ensure: Initialized model $\mathcal{M}$
    $inputs \leftarrow$ input sequence of shape (L, D);
    $x \leftarrow$ LSTM(U, return_sequences=True) applied to $inputs$;
    $x \leftarrow$ Bidirectional(LSTM(U)) applied to x;
    $x \leftarrow$ `RepeatVector`(L);
    $x \leftarrow$ LSTM(U, return_sequences=True) applied to x;
    $outputs \leftarrow$ `TimeDistributed(Dense(`D`))` applied to x;
    $\mathcal{M} \leftarrow$ Model($inputs$, $outputs$);
    Compile $\mathcal{M}$ with optimizer `Adam` and loss `MSE`;
    return $\mathcal{M}$
end function
Require: Training data $\mathcal{D}_{\text{train}}$, window size L
Ensure: Trained model $\mathcal{M}$
    Start
$X \leftarrow$ Generate sequences from $\mathcal{D}_{\text{train}}$ of shape (L, D);
$\mathcal{M} \leftarrow$ INITIALIZE_BILSTM_AUTOENCODER(L, D, U);
Initialize `EarlyStopping` with patience 20 and restore best weights;
Train $\mathcal{M}$ on (X, X) with 10% validation, E epochs, batch size B;
    End

4.4 Conclusion

In this chapter, we presented the candidates models for forecasting and anomaly detection based on time series from the RFCC unit.

Six models were described for the multivariate prediction task, along with one LSTM-based autoencoder model for anomaly detection.

These choices are grounded in both their theoretical relevance and their empirical effectiveness in sequential modeling. In the following chapter, dedicated to implementation, we will detail the training and comparative evaluation of these models on real-world industrial data. This experimental analysis will guide us in selecting the most suitable prediction model for deployment in our operational context.

Chapter 5: Implementation

5.1 Introduction

In this chapter, we detail the tools used, execution environments, the architecture of the developed application, and the process of integrating our application into the Sonatrach system. We also present the performance comparison of the models from the previous chapter as well as a final evaluation of our pipeline.

5.2 Implementation of Our Approach

After defining a rigorous methodological strategy and selecting the appropriate learning models, we began the implementation phase of our solution. This phase required the integration of data processing tools, advanced modeling frameworks, and modern solutions for the development of interactive user interfaces. All of these components were orchestrated with the aim of producing a robust and scalable application capable of meeting the specific constraints of the energy and industrial sectors.

5.2.1 Tools Used

The implementation of our approach is based on a set of technological tools recognized for their efficiency in the fields of artificial intelligence and software development.

5.2.1.1 Python

Python is an interpreted, versatile, object-oriented, and dynamic programming language. It is widely used in data science, artificial intelligence, and software engineering due to its clear syntax, flexibility, and rich library ecosystem [23].

5.2.1.2 TensorFlow

TensorFlow is a Google open-source library for numerical computation based on data flow graphs. It is particularly well-suited for deep learning and supports complex architectures such as convolutional neural networks (CNNs), recurrent networks (RNNs/LSTMs), as well as distributed processing on GPU [24].

5.2.1.3 scikit-learn

scikit-learn is also an open-source library designed for machine learning in Python. It provides efficient tools for data preprocessing, statistical modeling (regression, classification, dimensionality reduction, etc.), and model evaluation using standard metrics [25].

5.2.1.4 Conda

Conda is an open-source environment and package manager used to install, run, and update libraries in isolated environments [26].

5.3 Hardware Environment

The data preprocessing, training, and evaluation of the prediction and anomaly detection models were performed using the GIGABYTE machine detailed below, which has performance comparable to that of the refinery.

Additionally, we used another HP machine for the same purposes, but the final execution was carried out on the first machine for performance reasons.

PC	**OS**	**CPU**	**GPU**	**GPU Memory**	**RAM**
HP	Windows	Intel Core i7-13700H, 2.40–2.60 GHz	Nvidia RTX A500	4 GB	32 GB
GIGABYTE	Windows	Intel Core i5-12500H, 3.20–3.50 GHz	Nvidia RTX 4050	6 GB	16 GB

Table 5.1: Hardware environment used

5.4 Implementation of Our Approach

5.4.1 Technologies Used

Below are the main technologies used for the development of our solution, each chosen for its reliability, performance, and suitability for the requirements of modern distributed systems.

5.4.1.1 FastAPI

FastAPI is an open-source web framework for creating RESTful APIs with Python, known for its speed. It is based on the ASGI standard and includes automatic documentation handling via OpenAPI [27].

5.4.1.2 Uvicorn

Uvicorn is a lightweight and high-performance web server designed to run asynchronous Python applications. Compatible with the ASGI standard, it uses optimized tools like uvloop (to speed up task management) and httptools (to quickly process HTTP requests), enabling it to deliver very high performance [28].

5.4.1.3 ReactJS

ReactJS is a JavaScript library developed by Meta (formerly Facebook) for creating dynamic and modular user interfaces. It is based on a system of reusable components and a performant virtual DOM [29].

5.4.1.4 Git

Git is a distributed version control system that allows tracking of changes made to a project's source code, facilitates team collaboration, and maintains a history of changes [30].

5.4.2 Software Architecture

The solution we developed is based on a well-structured software architecture, organized around two main components: a **backend** and a **frontend**.

The **backend**, developed using the *FastAPI* framework, is responsible for processing input data (a time sequence in CSV format), executing inference models (prediction and anomaly detection), and sending the results to the user interface (in JSON format).

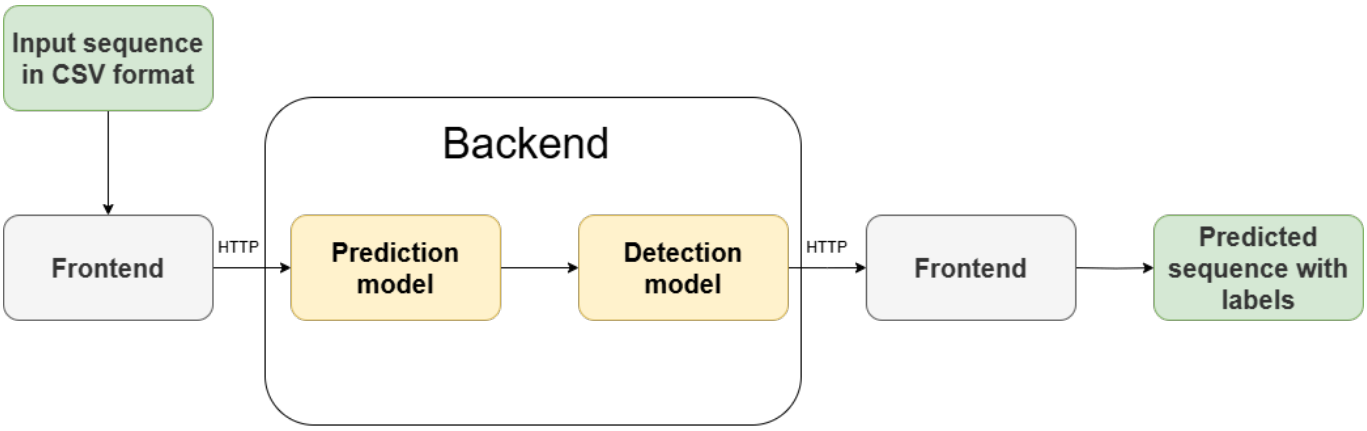


Figure 5.1: Application pipeline.

The **frontend**, designed with *ReactJS*, represents the application's user interface. It allows users to upload the input file, view it, inspect the generated predictions, the detected anomalies, and descriptive statistics. Special attention was paid to the interface's ergonomics to offer a smooth, intuitive, and interactive experience, particularly through the integration of dynamic graphical visualizations (line charts with zoom and anomaly highlighting).

Communication between these two components is carried out via HTTP calls, in a client-server architecture. This decoupling between backend and frontend offers many advantages: it allows better code maintainability, facilitates the scalability of the solution (addition of features or replacement of components), and permits distributed deployment in heterogeneous environments.

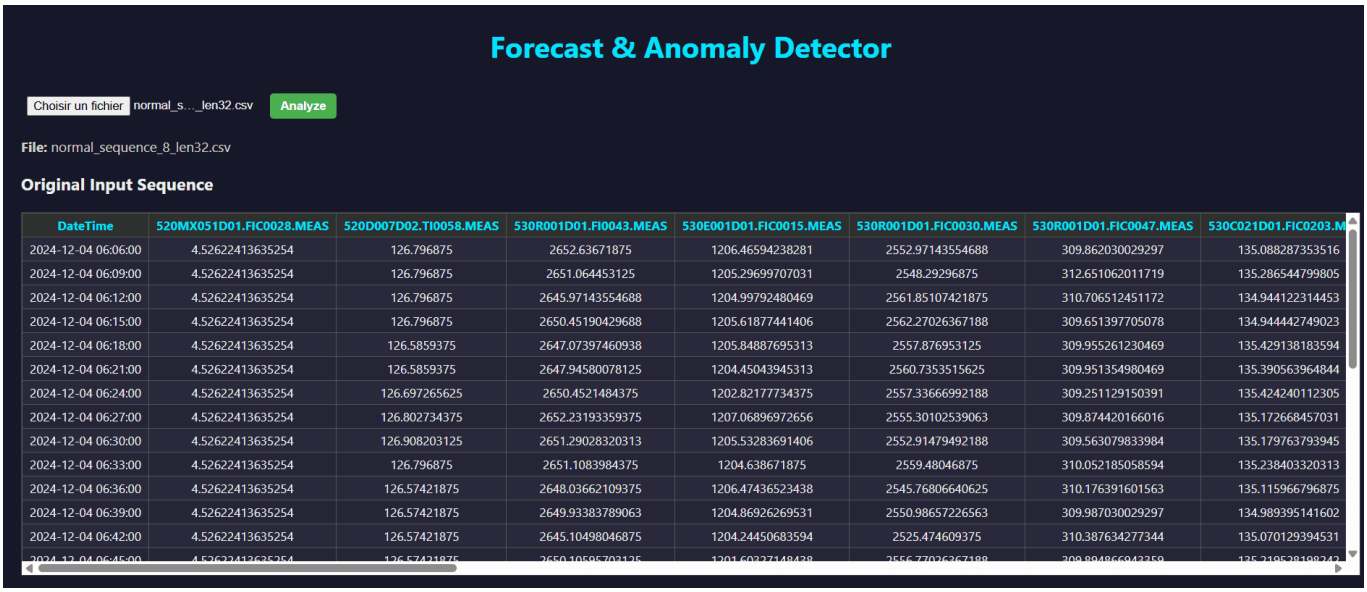
5.4.3 Application Features

The developed application offers an intuitive and modern interface to leverage the power of predictive and anomaly detection models applied to industrial time series. It consists of an interactive dashboard accessible via a web interface developed with ReactJS, connected to a FastAPI backend that handles data processing and model inference.

The main features of the application are as follows:

- **CSV File Upload**: the user can upload a file containing a historical sequence via a file selection field. The file name is then displayed for confirmation. This CSV is manually extracted by the engineer from the RFCC unit database via an SQL query.

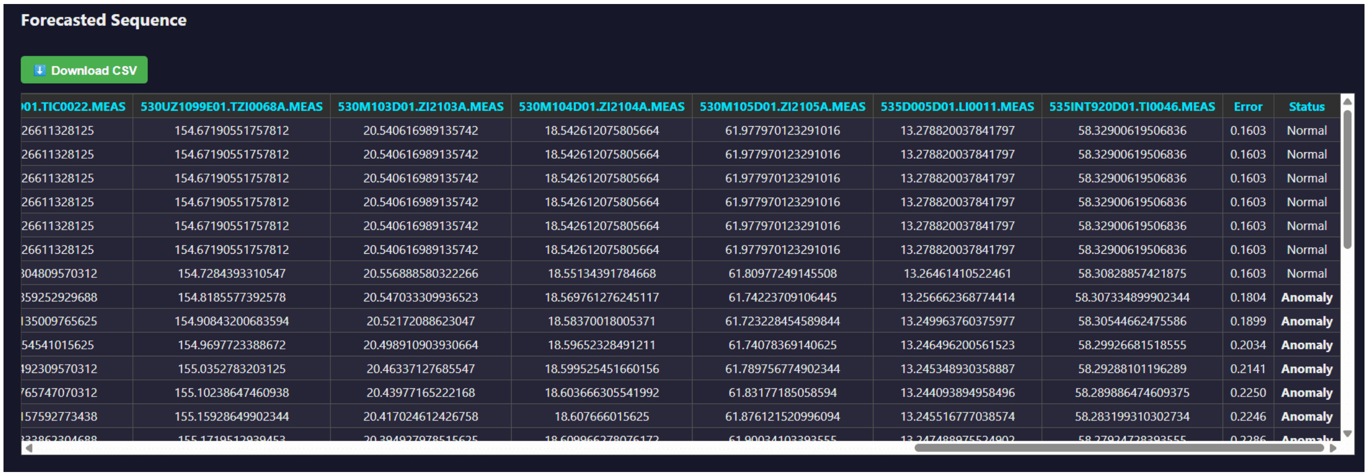
- **Input Data Preview**: after uploading, the input sequence is displayed in a styled table, allowing visual verification of columns, values, and timestamps. This ensures that the data is correctly recognized by the system.
- **Future Behavior Forecasting**: once the file is processed, the application displays the prediction results produced by the forecasting model. The predicted values are shown in a table including timestamps, measurements for each variable, and a binary status indicator (Normal or Anomaly).
- **Download Results**: a button allows downloading the forecasted sequence with anomaly flags in CSV format, facilitating further analysis or results archiving.
- **Custom Graphical Visualization**: a dedicated section allows the user to select a variable to be visualized as a curve. Detected anomalies are highlighted in red. A zoom and temporal scrolling tool enables precise signal inspection.
- **Global Statistics**: descriptive statistics (mean, minimum, maximum) are automatically calculated on the forecasted sequence for each variable. These are presented in a clear table with horizontal scrolling.
- **Modern and Responsive Ergonomics**: the application adopts a dark color scheme with neon accents to enhance readability, offer a professional look, and adapt to various screen types (desktop, laptop, etc.). This aesthetic choice is also motivated by health considerations: according to occupational medicine recommendations, engineers and technicians in industrial environments are often exposed to light-background interfaces for extended periods, which can lead to visual fatigue, migraines, dry eyes, and decreased concentration. Using a dark theme helps improve visual comfort by reducing emitted blue light and minimizing glare during continuous use.



DateTime	520MX051D01.FIC0028.MEAS	520D007D02.TI0058.MEAS	530R001D01.FI0043.MEAS	530E001D01.FIC0015.MEAS	530R001D01.FIC0030.MEAS	530R001D01.FIC0047.MEAS	530C021D01.FIC0203.M
2024-12-04 06:06:00	4.52622413635254	126.796875	2652.63671875	1206.46594238281	2552.97143554688	309.862030029297	135.088287353516
2024-12-04 06:09:00	4.52622413635254	126.796875	2651.064453125	1205.29699707031	2548.29296875	312.651062011719	135.286544799805
2024-12-04 06:12:00	4.52622413635254	126.796875	2645.97143554688	1204.99792480469	2561.85107421875	310.706512451172	134.944122314453
2024-12-04 06:15:00	4.52622413635254	126.796875	2650.45190429688	1205.61877441406	2562.27026367188	309.651397705078	134.944442749023
2024-12-04 06:18:00	4.52622413635254	126.5859375	2647.07397460938	1205.84887695313	2557.876953125	309.955261230469	135.429138183594
2024-12-04 06:21:00	4.52622413635254	126.5859375	2647.94580078125	1204.45043945313	2560.7353515625	309.951354980469	135.390563964844
2024-12-04 06:24:00	4.52622413635254	126.697265625	2650.4521484375	1202.82177734375	2557.33666992188	309.251129150391	135.424240112305
2024-12-04 06:27:00	4.52622413635254	126.802734375	2652.23193359375	1207.06896972656	2555.30102539063	309.874420166016	135.172668457031
2024-12-04 06:30:00	4.52622413635254	126.908203125	2651.29028320313	1205.53283691406	2552.91479492188	309.563079833984	135.179763793945
2024-12-04 06:33:00	4.52622413635254	126.796875	2651.1083984375	1204.638671875	2559.48046875	310.052185058594	135.238403320313
2024-12-04 06:36:00	4.52622413635254	126.57421875	2648.03662109375	1206.47436523438	2545.76806640625	310.176391601563	135.115966796875
2024-12-04 06:39:00	4.52622413635254	126.57421875	2649.93383789063	1204.86926269531	2550.98657226563	309.987030029297	134.989395141602
2024-12-04 06:42:00	4.52622413635254	126.57421875	2645.10498046875	1204.24450683594	2525.474609375	310.387634277344	135.070129394531

Figure 5.2: Historical sequence introduced in the application interface.

The figure 5.2 illustrates the preview of the past sequence as displayed to the user after uploading the CSV file. It allows immediate verification of the data structure before processing.



01.TIC0022.MEAS	530UZ1099E01.TZI0068A.MEAS	530M103D01.ZI2103A.MEAS	530M104D01.ZI2104A.MEAS	530M105D01.ZI2105A.MEAS	535D005D01.LI0011.MEAS	535INT920D01.TI0046.MEAS	Error	Status
26611328125	154.67190551757812	20.540616989135742	18.542612075805664	61.977970123291016	13.278820037841797	58.32900619506836	0.1603	Normal
26611328125	154.67190551757812	20.540616989135742	18.542612075805664	61.977970123291016	13.278820037841797	58.32900619506836	0.1603	Normal
26611328125	154.67190551757812	20.540616989135742	18.542612075805664	61.977970123291016	13.278820037841797	58.32900619506836	0.1603	Normal
26611328125	154.67190551757812	20.540616989135742	18.542612075805664	61.977970123291016	13.278820037841797	58.32900619506836	0.1603	Normal
26611328125	154.67190551757812	20.540616989135742	18.542612075805664	61.977970123291016	13.278820037841797	58.32900619506836	0.1603	Normal
26611328125	154.67190551757812	20.540616989135742	18.542612075805664	61.977970123291016	13.278820037841797	58.32900619506836	0.1603	Normal
304809570312	154.7284393310547	20.556888580322266	18.55134391784668	61.80977249145508	13.26461410522461	58.30828857421875	0.1603	Normal
359252929688	154.8185577392578	20.547033309936523	18.569761276245117	61.74223709106445	13.256662368774414	58.307334899902344	0.1804	**Anomaly**
135009765625	154.90843200683594	20.52172088623047	18.58370018005371	61.723228454589844	13.249963760375977	58.30544662475586	0.1899	**Anomaly**
54541015625	154.9697723388672	20.498910903930664	18.59652328491211	61.74078369140625	13.246496200561523	58.29926681518555	0.2034	**Anomaly**
492309570312	155.0352783203125	20.46337127685547	18.599525451660156	61.789756774902344	13.245348930358887	58.29288101196289	0.2141	**Anomaly**
765747070312	155.10238647460938	20.43977165222168	18.603666305541992	61.83177185058594	13.244093894958496	58.289886474609375	0.2250	**Anomaly**
157592773438	155.15928649902344	20.417024612426758	18.607666015625	61.876121520996094	13.245516777038574	58.283199310302734	0.2246	**Anomaly**

Figure 5.3: Resulting forecasted sequence in the application interface.

The figure 5.3 presents a table that groups the values predicted by the model, accompanied by status indicators (Normal or Anomaly), thus facilitating rapid interpretation of the results.

Feature	520MX051D01.FIC0028.MEAS	520D007D02.TI0058.MEAS	530R001D01.FI0043.MEAS	530E001D01.FIC0015.MEAS	530R001D01.FIC0030.MEAS	530R001D01.FIC0047.MEAS	530C021D01.FIC0203.MEAS
Mean	4.53	127.36	2647.07	1214.89	2587.33	310.12	132.21
Min	4.53	127.16	2646.18	1213.03	2586.48	310.00	132.01
Max	4.54	127.67	2648.13	1217.61	2588.84	310.17	132.61

Figure 5.4: Statistics of the forecasted sequence in the application interface.

The figure 5.4 shows the graph of predicted values for a selected feature along with the labels, as well as a statistical summary (mean, minimum, maximum) of each predicted variable, providing a concise overview of the behavior of the analyzed system.

These features were designed in close alignment with the real needs of industrial monitoring, particularly in critical contexts such as Sonatrach installations. The focus was on automation, ease of use, readability of results, and the ability to integrate into an existing industrial environment.

5.4.4 Deployment

The deployment of our solution within the RFCC unit of Sonatrach required a careful approach compliant with current security constraints. As interns, we were not authorized to directly inte-

grate our application into the unit's IT environment, which is classified as critical infrastructure for national security. Any intervention on this system is strictly regulated and requires specific authorizations, reserved for accredited engineers.

Initially, we developed a complete prototype integrating a Kafka stream to ensure real-time data transmission. This prototype was presented to the unit's technical team, accompanied by a formal request to proceed with integration tests. However, for compliance reasons with internal cybersecurity policy, this request was not approved.

In light of these constraints, an alternative was validated in collaboration with the responsible engineers: we delivered the two components of the application (the FastAPI backend and the ReactJS frontend) as standalone solutions ready to be executed in their environment. The planned integration relies on interconnection with the existing information system via Kafka, enabling automatic feeding of the application with real-time collected data sequences, without requiring manual extractions via SQL queries.

To facilitate adoption and scalability of the solution, we ensured the installation of the entire environment necessary for its proper operation (Python, Node.js, libraries, dependencies, etc.), and documented all execution and modification steps. Thus, the Sonatrach engineers not only have a functional application but also the means to maintain it, evolve it, or adapt it to new needs autonomously.

5.5 Comparison of Prediction Models

We now proceed to test our six models. First, for each architecture, we will explore multiple configurations in order to find the one that yields the best results (see Appendix). Then, equipped with these optimal versions, we will compare all the models side-by-side to determine which one excels the most in forecasting our time series.

The figures 5.5 to 5.10 present the Training Loss and Validation Loss curves:

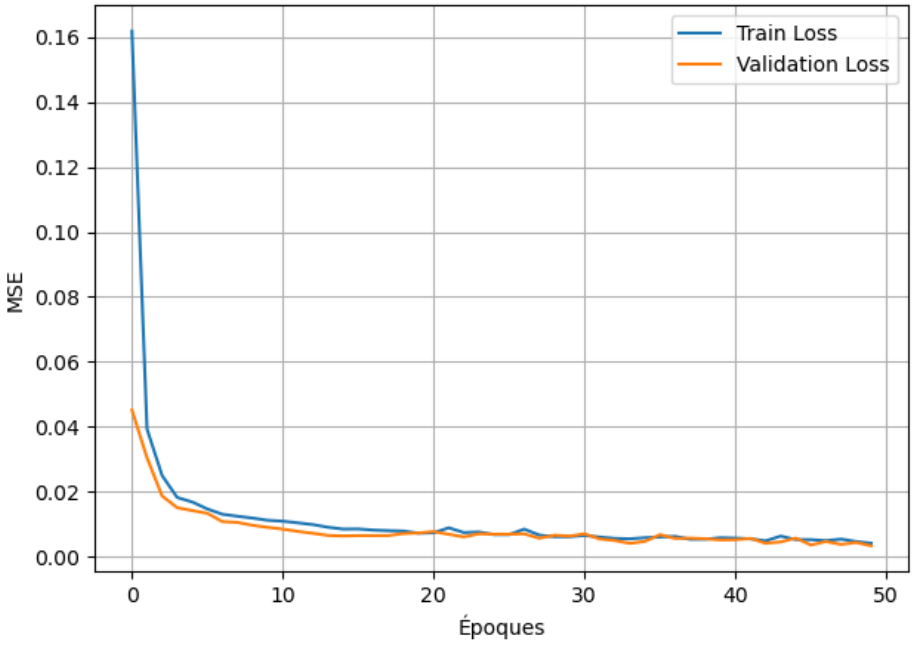


Figure 5.5: Loss curve – LSTM seq2seq

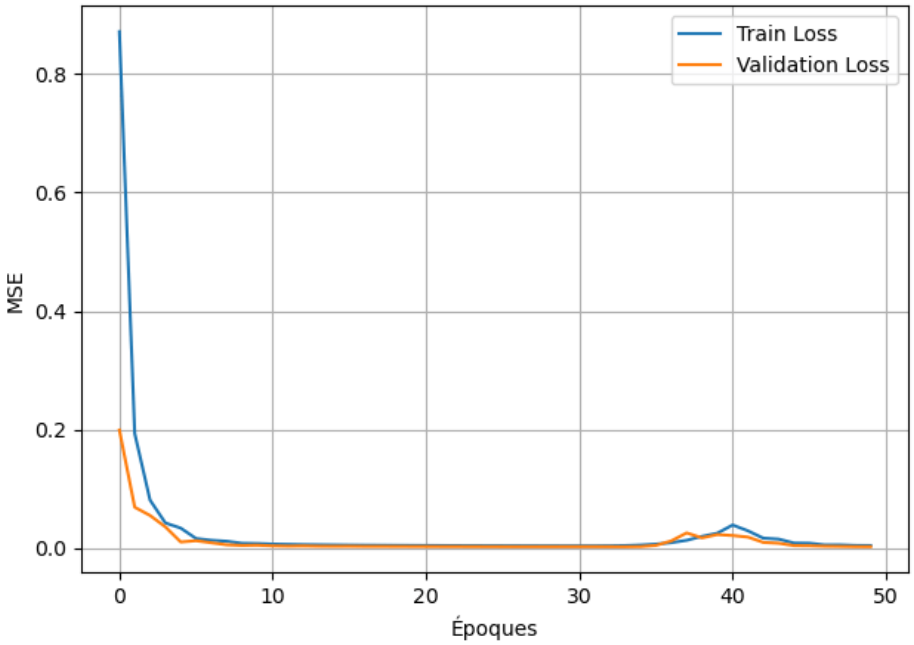


Figure 5.6: Loss curve – LSTM Multihead Attention

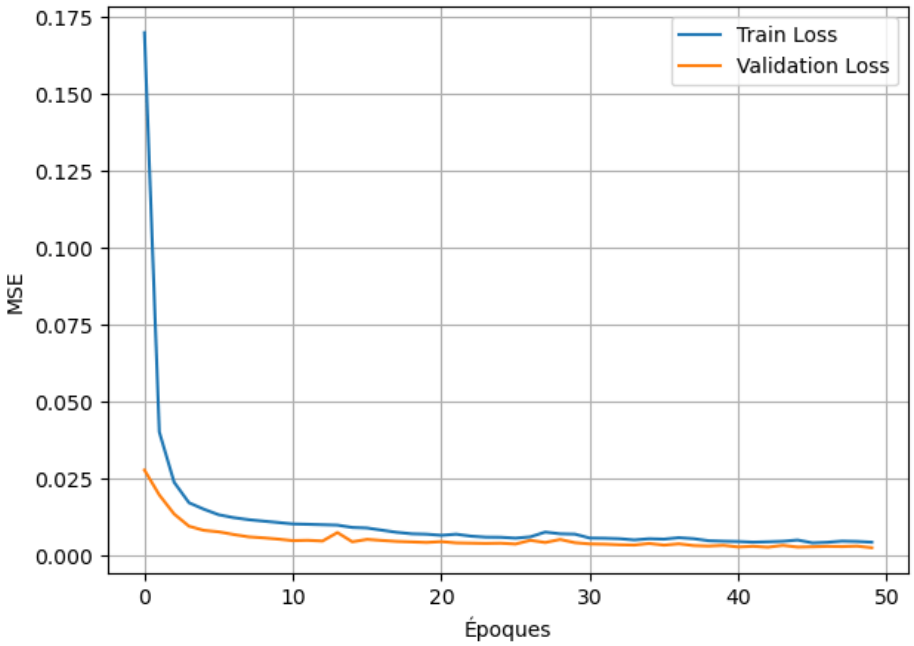


Figure 5.7: Loss curve – Informers

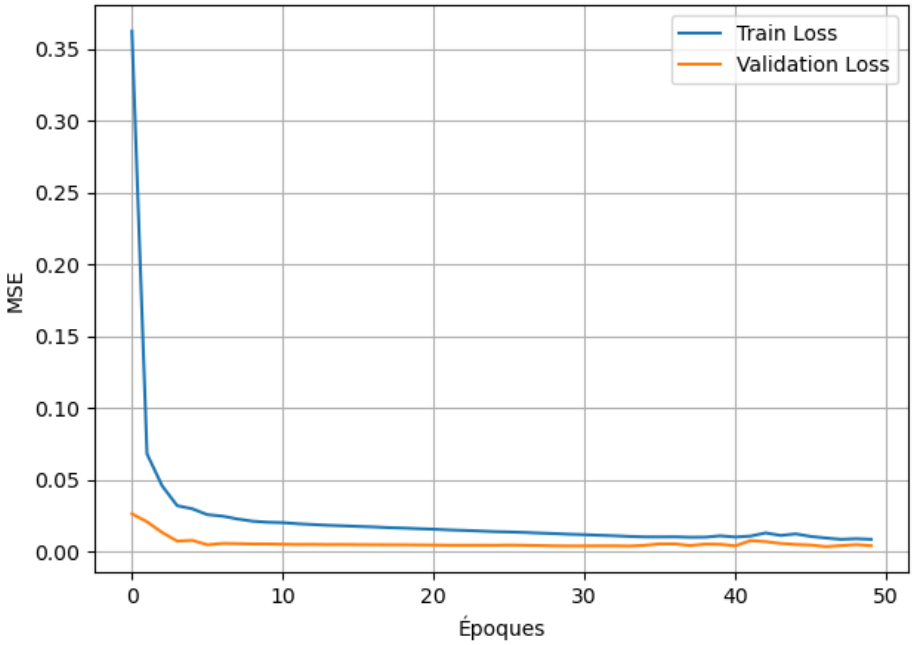


Figure 5.8: Loss curve – Transformers

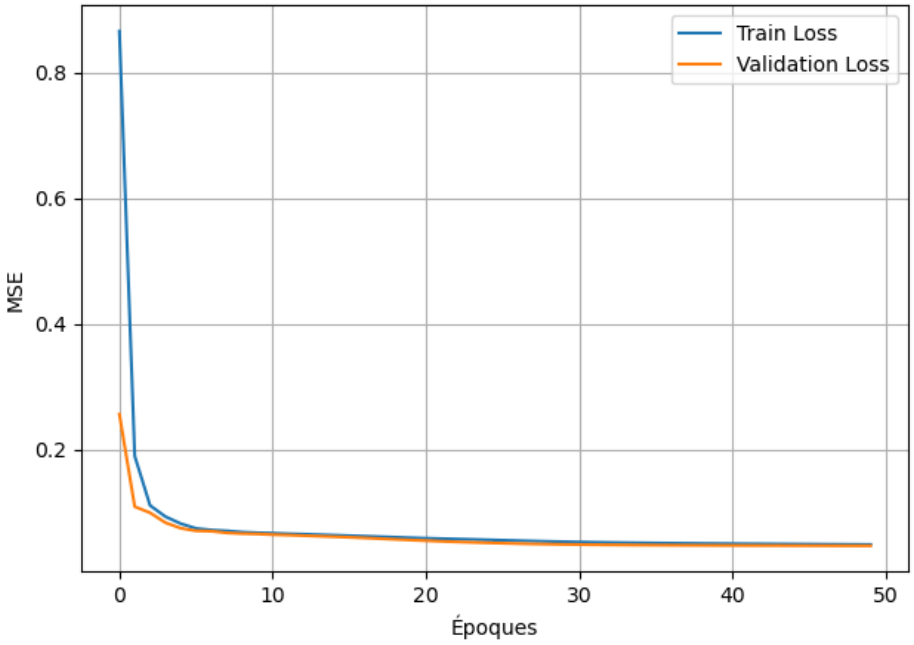


Figure 5.9: Loss curve – PatchTST (standard)

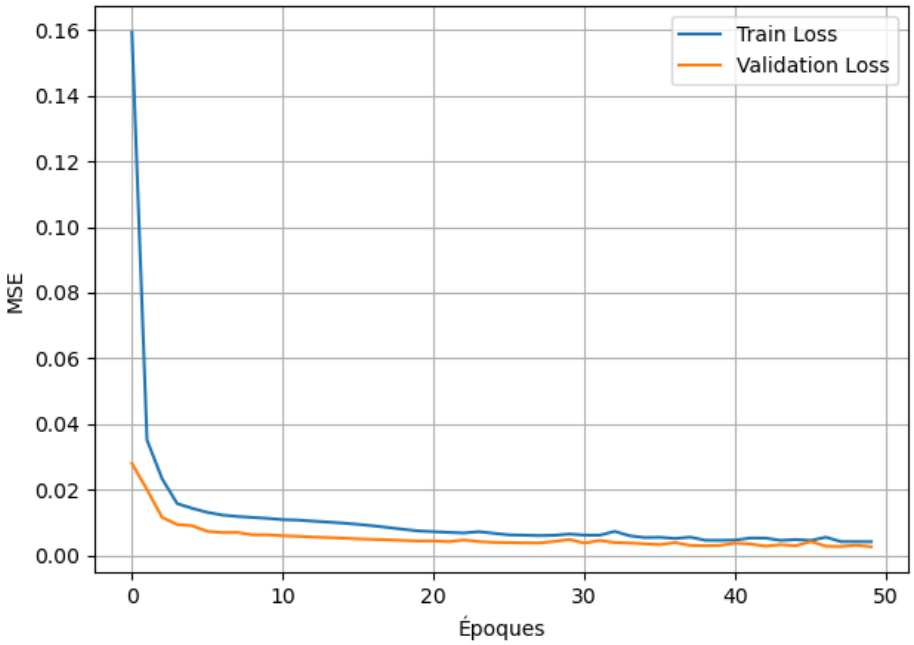


Figure 5.10: Loss curve – PatchTST (adapted)

All models show relatively fast convergence from the first epochs. This indicates their ability to adapt to the industrial data used. However, we observe some interesting phenomena:

- For several models (notably those based on transformers), the validation loss is sometimes lower than the training loss. This may seem counter-intuitive but is not necessarily problematic. It can be explained by implicit regularization or a validation dataset that is more homogeneous than the training set.

- The absence of overfitting is generally confirmed by the stability of the curves over the 50 epochs.
- The losses stabilize at different levels depending on the model, which corresponds to the MSE/MAE scores reported in the comparison table.

To assess the prediction quality, we used two standard metrics in the field of multivariate time series:

- **Mean Squared Error (MSE)**: this is the mean of the squared errors between the actual and predicted values. MSE is always positive, and the closer it is to zero, the better the model's predictions. This metric heavily penalizes large errors, making it sensitive to significant deviations.

$$\text{MSE} = \frac{1}{N} \sum_{n=1}^{N} (y_n - \hat{y}_n)^2$$

  where:

  - N: total number of observations,
  - y_n: actual value for observation n,
  - $\hat{y}_n$: predicted value for observation n.

- **Mean Absolute Error (MAE)**: this is the mean of the absolute values of the errors between the actual and predicted values. Less sensitive to extreme values than MSE, MAE provides a more linear estimate of the average error and is often more interpretable in industrial contexts.

$$\text{MAE} = \frac{1}{N} \sum_{n=1}^{N} |y_n - \hat{y}_n|$$

  where:

  - N: total number of observations,
  - y_n: actual value for observation n,
  - $\hat{y}_n$: predicted value for observation n,
  - $|x|$: absolute value of x.

These two metrics are widely used in recent state-of-the-art works on time series forecasting, particularly in the papers introducing the *PatchTST* [21] and *Informer* [31] models, which also allows us to compare our results within a consistent academic framework.

Model	Validation MSE	Validation MAE	Time (s)	Spatial Complexity (Big-O)
LSTM seq2seq	0.003224	0.036543	9.9	$O(u^2 + u \cdot id)$
LSTM+Attention	0.004267	0.046870	11.1	$O(u^2 + u \cdot id + nh \cdot u^2)$
Transformer	0.005926	0.056789	8.7	$O(ed^2 + ed \cdot ffd + sl \cdot ed)$
Informer	0.003800	0.041697	17.3	$O(ed^2 + ed \cdot id + k \cdot sl)$
PatchTST	0.051289	0.184532	13.2	$O(np \cdot pl \cdot ed + nl \cdot ed^2)$
PatchTST custom	0.003849	0.042100	11.1	$O(np \cdot pl \cdot ed + ed^2)$

Table 5.2: Hyperparameter tuning results for prediction models

u: LSTM units, id: input_dim, nh: num_heads, ed: embed_dim, ffd: ff_dim, sl: sequence_length, np: num_patches, pl: patch_len, nl: num_layers, k: top_k (ProbSparse)

Note on complexity and execution time: In our context of time series inference, neither spatial complexity nor training execution time are limiting factors. In production, the input will consist of a single sequence of 32 time steps, making inference time negligible, even for complex models. This is why we have deliberately chosen not to mention time complexity and only included spatial complexity for informational purposes. Even this is not a critical factor in our case, as the memory footprint of the models remains acceptable on modern GPUs.

Furthermore, we kept a sequence length of 32 in order to maximize the number of training iterations. Increasing this length would drastically reduce the number of possible windows in the series, and thus the total amount of training data.

Overall, LSTM architectures proved to be more effective for our use case. The *LSTM seq2seq* model achieved the best MSE performance, while the *LSTM-MultiheadAttention* offered an interesting trade-off between complexity, stability, and accuracy.

However, our final choice was not based solely on validation scores (MSE and MAE). We also assessed the models' ability to faithfully reproduce the statistical distribution of the real data, particularly in terms of variance, amplitude of the predicted signals, and their visual behavior relative to the original time series. The LSTM-MultiheadAttention model better preserved this overall consistency, avoiding overly smoothed or stationary predictions. This criterion is crucial in an industrial context, where the quality of the signal shape is as critical as its numerical accuracy.

The Transformer and Informer, although effective in classical benchmarks, showed slightly inferior performance in our industrial context. This is likely due to the relatively modest size of our training dataset (80% of the window, about 24,000 rows) and the limited sequence length (32 steps in input and output).

Regarding *PatchTST*, the base version yielded less satisfactory results. This may be because it was initially designed for long-term forecasting on very large data volumes. On the other hand, after customization (removal of the Channel Independence concept and the addition of an LSTM-based decoder), the model achieved excellent scores comparable to LSTMs, demonstrating that it can be adapted to constrained contexts, provided it is intelligently reconfigured.

Thus, while Transformer-based models are powerful, recurrent architectures (particularly LSTM

variants enhanced with an attention mechanism) proved to be the most suitable for our case study, both in terms of metrics and fidelity to the physical signals of the process.

It is also important to emphasize that good performance in MSE or MAE does not necessarily guarantee prediction quality in practice. These metrics can sometimes be misleading, especially in the case of models that tend to smooth sequences or produce averages, which mathematically minimizes error but does not accurately reflect the system's real dynamics. This is why visual analysis and statistical consistency of the predictions were essential criteria in our selection process.

The figures from 5.11 to 5.16 show, for each evaluated model, an example of prediction on a single attribute, comparing the real sequence (in blue) and the predicted sequence (in orange). Although all these models display good MSE and MAE scores, these curves reveal significant differences in terms of distribution, value shifts, and temporal behavior.

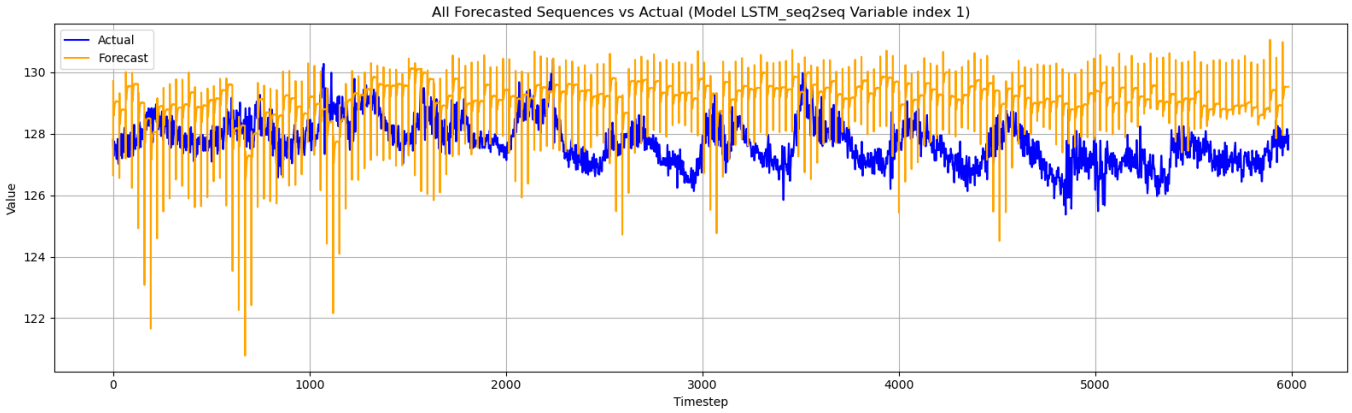


Figure 5.11: Actual vs predicted forecast – LSTM seq2seq model

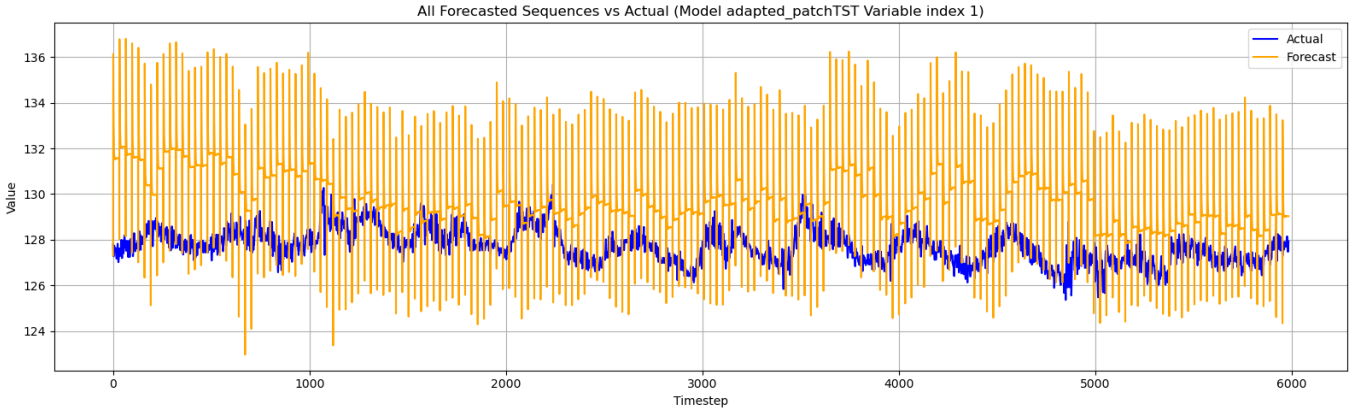


Figure 5.12: Actual vs predicted forecast – Adapted PatchTST model

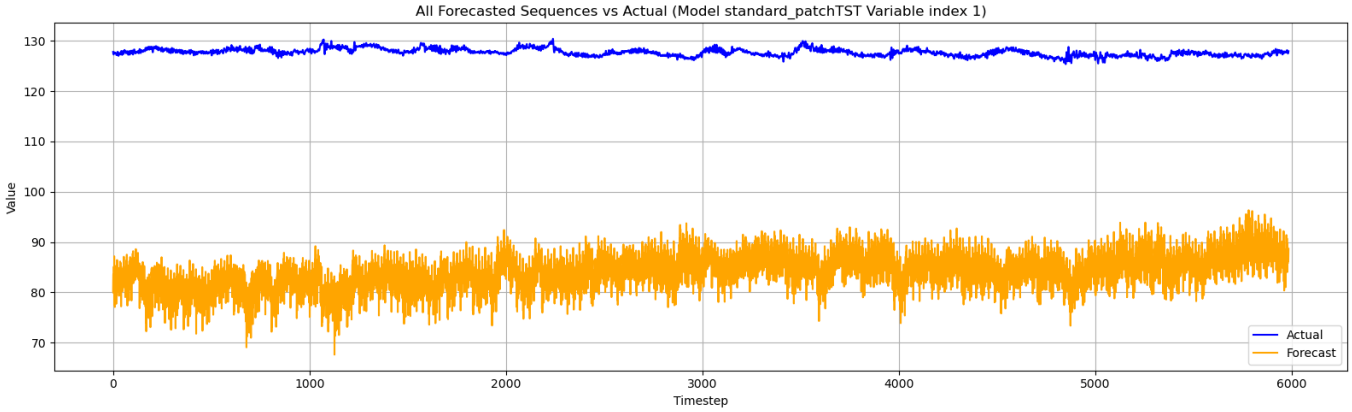


Figure 5.13: Actual vs predicted forecast – Standard PatchTST model

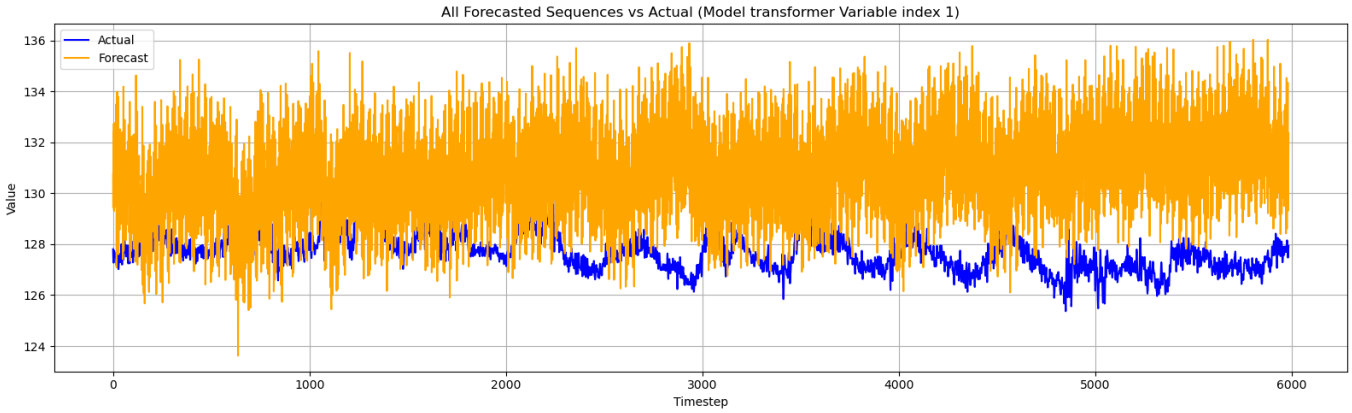


Figure 5.14: Actual vs predicted forecast – Transformer model

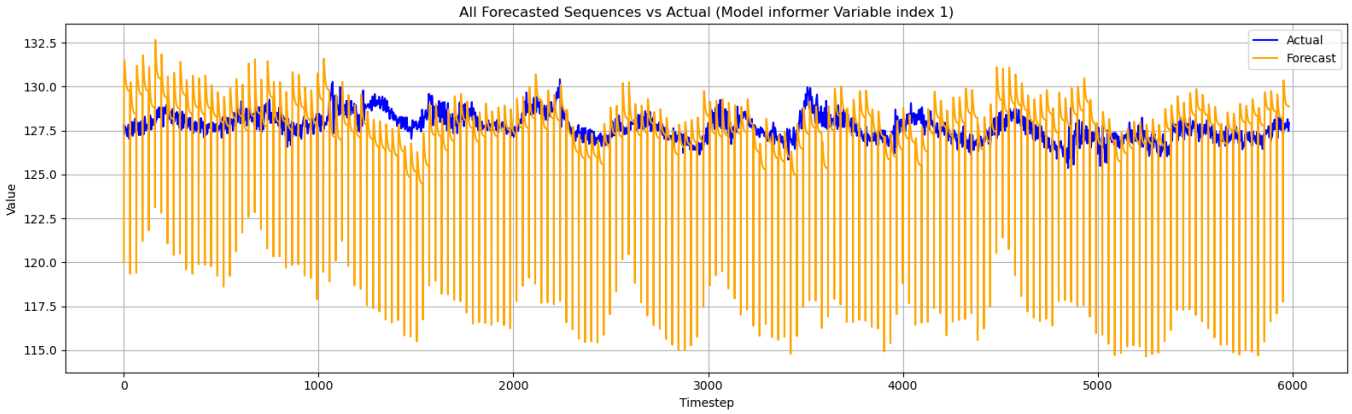


Figure 5.15: Actual vs predicted forecast – Informer model

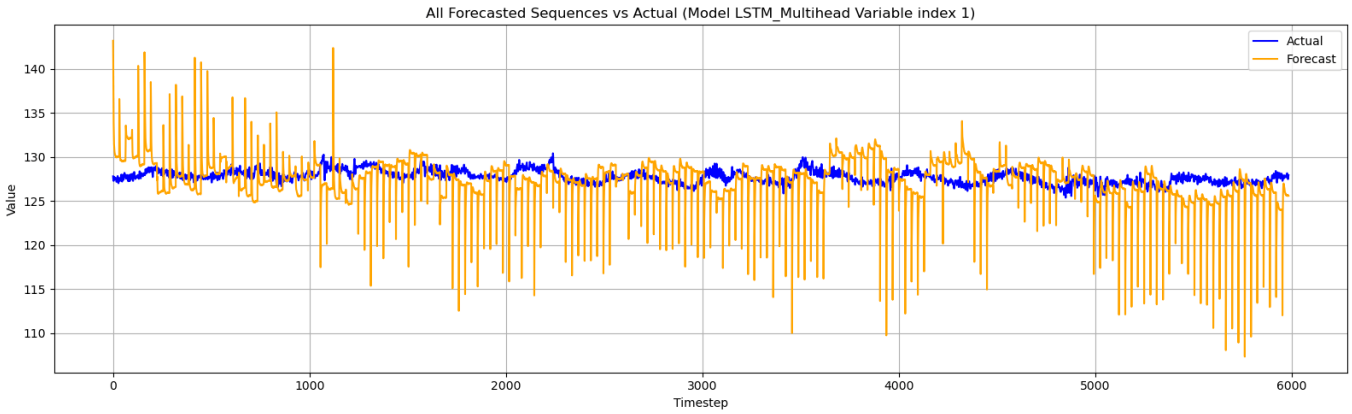


Figure 5.16: Actual vs predicted forecast – LSTM Multihead Attention model

A close examination of the figures from 5.11 to 5.16 highlights the following points:

- This type of instability at the beginning of sequences is common in forecasting models, particularly those using an encoder architecture. The model lacks context at startup, leading to sharp fluctuations in the first predicted values. To correct this in production, a smoothing is applied: the sixth value is assigned to the first five, removing initial spikes without altering the overall dynamics.
- Some models, like **PatchTST** or **Transformer**, produce predictions that deviate significantly in level or amplitude, despite acceptable MSE scores. The distribution of predicted values does not faithfully reflect the real signal dynamics.
- Other models, like **Informer** or **LSTM seq2seq**, manage to capture the overall shape, but suffer from consistent shifts (bias in value), which penalizes business accuracy.
- The **Adapted PatchTST** model partially improves the distribution, but still shows high dispersion, making it unstable in industrial environments.
- Finally, the **LSTM Multihead Attention** model stands out. It not only respects the overall shape of the signal but also stays within a coherent value range. Its ability to follow the real distribution while limiting outliers justifies its integration into our final pipeline.

This type of visualization is crucial to complement traditional metrics like MSE or MAE. Indeed, a good aggregate score can mask systematic biases, loss of variance, or errors on specific attributes. Hence the necessity of visual validation before industrial deployment.

5.6 Model Validation for Anomaly Detection

To ensure the stability and effectiveness of the *Autoencoder-BiLSTM* model designed for anomaly detection, an analysis of its training behavior was conducted. The table below shows that the model converges properly, with a validation loss (0.001532) very close to the training loss (0.001814), indicating good generalization capability and the absence of overfitting.

Model	Validation loss	Training loss	Execution time	Spatial complexity
Autoencoder-LSTM	0.0015	0.0018	17.4 s	$\mathcal{O}(u^2 + ud + u + d)$

Table 5.3: Performance of the Autoencoder-LSTM model after hyperparameter tuning

The figure below shows the evolution of the loss function (MSE) during training of the autoencoder model based on an LSTM architecture. A rapid decrease in training loss is observed during the first epochs, followed by gradual convergence.

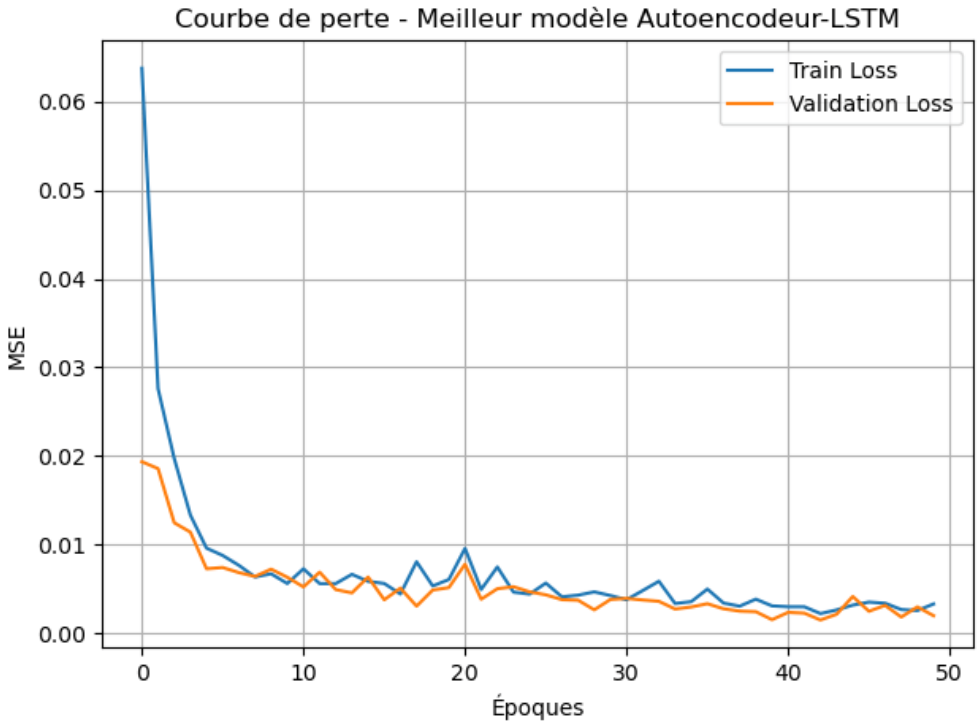


Figure 5.17: Learning curve of the LSTM Autoencoder model for anomaly detection.

The figure 5.17 represents the validation loss curve which closely follows that of the training loss, indicating that the model does not suffer from overfitting or underfitting. The low loss value at the end of training, along with the stability of both curves, confirms the model's ability to generalize effectively to unseen data.

This stability is essential for an anomaly detection system, where even slight overfitting could lead to false alerts, or conversely, undetected anomalies.

The actual performance of the model was tested on a separate dataset manually labeled by process engineers. To evaluate its quality, three metrics from binary classification were used:

- **Precision**: the ratio between the number of instances correctly identified as anomalies and the total number of instances predicted as anomalies. This metric reflects the model's ability to avoid false positives. The higher the precision, the more reliable the model is when flagging an anomaly.

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

- **Recall**: the ratio between the number of anomalies correctly detected and the total number of actual anomalies in the data. Recall measures the model's ability to detect true anomalies (avoiding false negatives). It is particularly important in industrial contexts, where it is preferable to trigger a few false alerts rather than miss a real anomaly.

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

- **F1-score**: the harmonic mean between precision and recall. It provides a balanced compromise between these two metrics, accounting for both false positives and false negatives.

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The results obtained on the test set are as follows:

- **Precision**: 0.7273
- **Recall**: 0.8889
- **F1-score**: 0.8000

These metrics are standard in the literature for systems detecting rare or critical events [32]. In the context of an industrial monitoring system, priority is given to **recall**, as it is better to generate a few false alerts than to miss real anomalies that could damage the facility or compromise safety. This high recall rate (88.89%) confirms the model's relevance in this context.

5.7 Evaluation Pipeline

In this section, we present the final selection of models to be integrated into our pipeline. This selection is based on objective performance criteria tailored to each task (prediction and detection), as well as specific business rules in the case of detection.

4.3.3.1 Prediction Model

The model selected for prediction is the LSTM-MultiheadAttention. This choice is based on the metrics and methods previously discussed.

4.3.3.2 Detection Model

The model used for anomaly detection is an Autoencoder based on an LSTM architecture. It was selected based on its performance in *Training loss* and *Validation loss*.

Additionally, this approach was enriched with a domain-specific rule: if the value of the tag `TI0037` exceeds 729 degrees Celsius, the corresponding sequence is automatically considered anomalous, regardless of the model output. This threshold was set at 729, slightly below the estimated critical value of 730, as a precaution. The goal is to anticipate the emergence of hazardous

conditions as much as possible, even at the cost of occasional false alarms. This preventive margin ensures increased system responsiveness, which is essential in a high-risk industrial environment.

This rule, developed in collaboration with process engineers, helps capture critical situations not learned by the detection model. It is a safety mechanism based on expert-defined thresholds, aimed at enhancing the overall robustness and reliability of the system.

Moreover, another step was added to the pipeline prior to detection to mitigate a bias introduced by prediction. Forecast model outputs tend to be slightly smoothed, which artificially reduces the natural variations present in real data. To correct this, we applied a scaling of deviations around the mean using a controlled factor.

Mathematically, for each variable X, the following transformation was applied:

$$X_{\text{amplified}} = \mu_X + \min\left(\frac{\sigma_{\text{real}}}{\sigma_{\text{forecast}}}, \alpha\right) \cdot (X - \mu_X)$$

where:

- μ_X is the mean of the variable in the forecast data,
- σ_{real} is the standard deviation of the variable in the real data,
- σ_{forecast} is the standard deviation in the forecast data,
- α is a maximum amplification factor set empirically (in our case to 6) to avoid overcorrection.

This correction realigns the distribution of forecasted data with that of the real dataset, facilitating the detection model's task and preventing it from misinterpreting normal sequences as being devoid of variation (and thus falsely normal). It also improves detection of discrete peaks that may arise in localized anomalous situations.

In summary, this detection model is based on a harmonious combination of deep learning (LSTM autoencoder), domain expertise (fixed threshold on `TI.0037`), and data engineering (statistical amplification), giving it precision, flexibility, and robustness within the complex context of an industrial refinery.

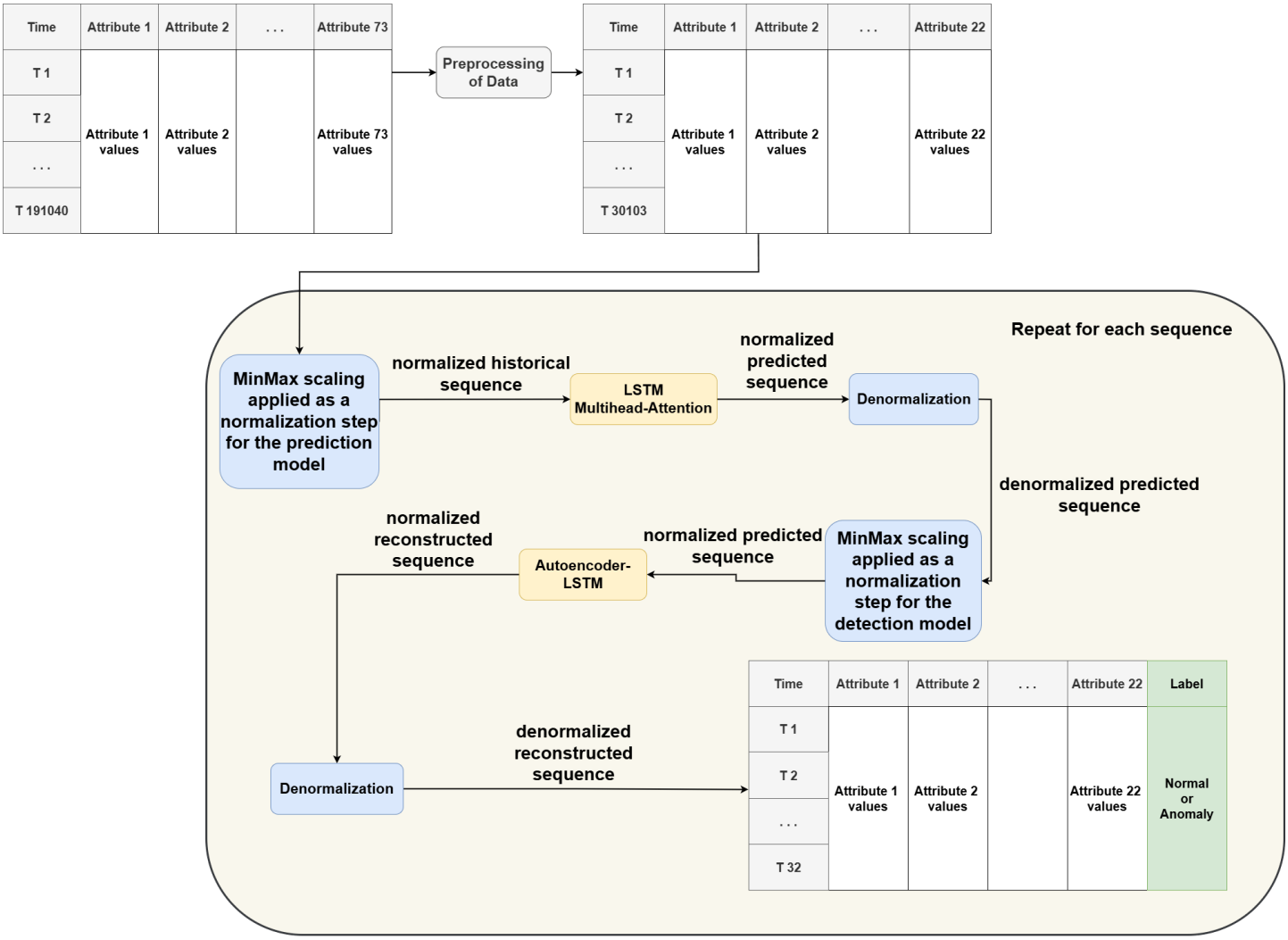


Figure 5.18: Final architecture of our solution

5.7.1 Model and Data Loading

The pre-trained models are loaded along with the corresponding scalers necessary for data normalization. The input file contains the time-stamped observations along with anomaly labels provided by Sonatrach process engineers. These labels were generated through a particularly complex labeling process due to the industrial nature of the data.

Indeed, several factors made this task challenging: the dataset structure is dense and not intuitive, the thresholds for normal operation are neither constant nor clearly defined, and the engineers had limited time for this task. Additionally, the failure or malfunction history is not digitized but exists only in the form of handwritten or printed weekly reports, which complicated their exploitation.

In this context, the engineers were able to annotate approximately 1408 rows, primarily based on these operation reports where issues were flagged. Although partial, this expert annotation represents a valuable basis for evaluating the detection model.

5.7.2 Prediction with the LSTM-MultiHeadAttention Model

The model receives sequences of length 32 (parameter `INPUT_STEPS`) and predicts the next 32 values (`FORECAST_STEPS`).

6.2.2.1 Quantitative Prediction Evaluation

The outputs are compared to the actual target values. The following results were obtained before denormalization:

- **MSE (Mean Squared Error)**: 0.00542
- **MAE (Mean Absolute Error)**: 0.05451

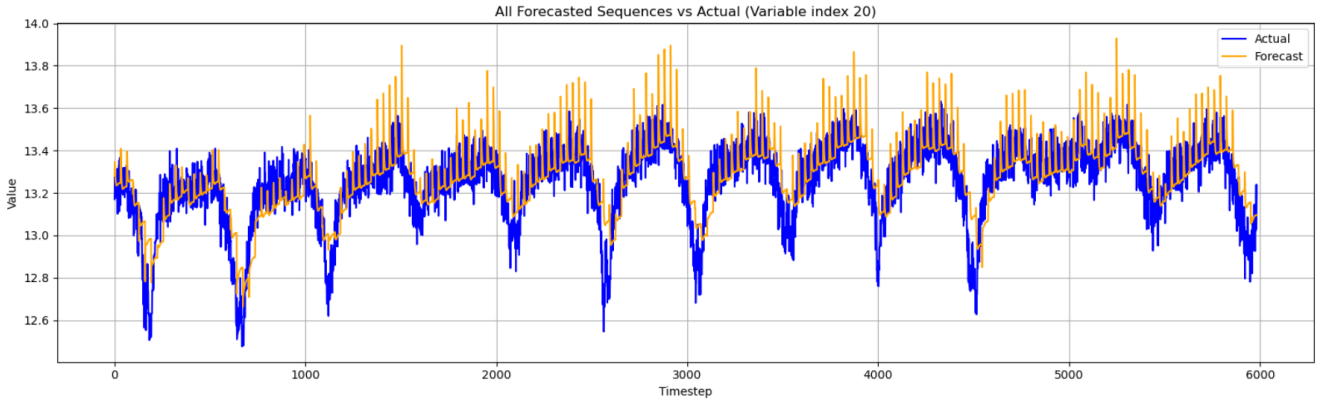


Figure 5.19: Forecast vs actual values on attribute 20

Although the MSE and MAE metrics are very satisfactory, the figure 5.19 reveals a slight transient shift between the predictions and actual values, mainly at the beginning of the sequences. This behavior, typical of sequential encoder-based models, is due to the lack of initial context. However, our model succeeds in faithfully capturing the general shape and distribution of the data, which indicates its ability to learn the underlying dynamics of the process. To further enhance output stability in production, smoothing is applied to the first five values of each sequence by replacing them with the sixth predicted value. This strategy eliminates initial fluctuations without compromising the overall signal coherence.

5.7.3 Anomaly Detection with the BiLSTM Autoencoder

The forecast model's predictions are denormalized and then fed into the autoencoder model. It reconstructs the input sequences and calculates a point-wise reconstruction error, reflecting the deviation between input data and its reconstruction.

6.2.3.1 Anomaly Threshold Definition

A fixed threshold $T = 0.178$ was chosen based on analysis of the ROC curve and manual comparison of several thresholds to achieve good precision, using the ROC-suggested threshold (0.156006) as a starting point. This threshold helps differentiate normal time steps from those suspected of being anomalous.

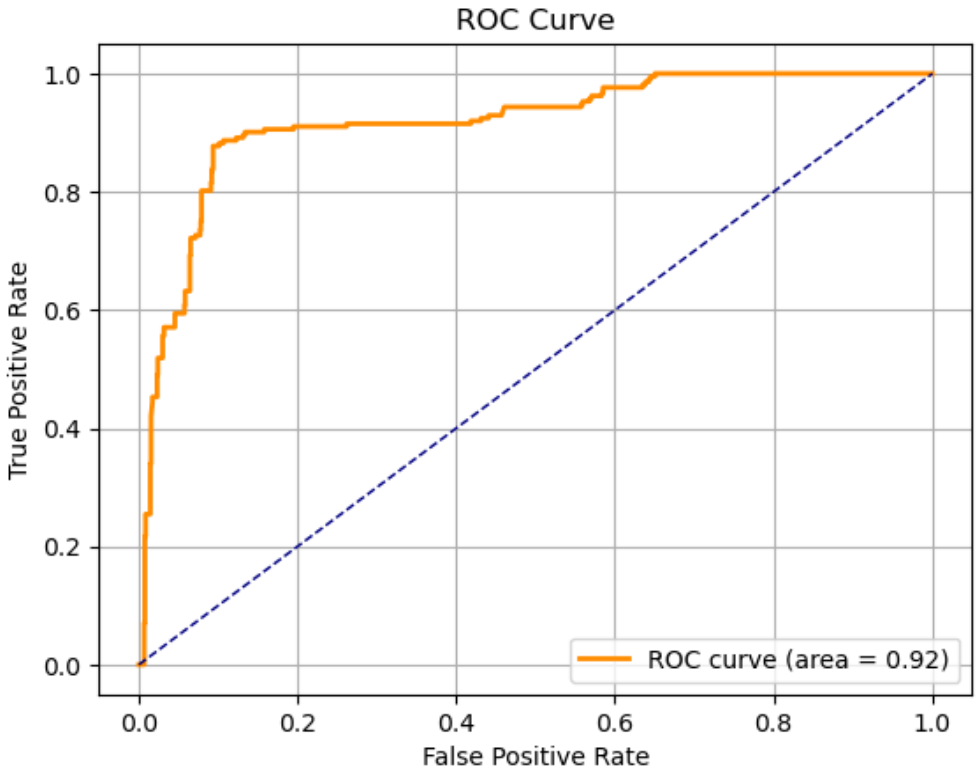


Figure 5.20: ROC curve of the detection model (AUC = 0.92)

6.2.3.2 Reconstruction Error Visualization

Choice of Error Metric To compute reconstruction error in our autoencoder model, we used the following formula:

$$\text{Error}(t) = \max_j |x_j(t) - y_j(t)|$$

where $x_j(t)$ is the actual value of attribute j at time t, and $y_j(t)$ is its reconstruction by the model.

Unlike MSE (Mean Squared Error), this metric uses the maximum absolute deviation among attributes at each time step. It is particularly suitable in our industrial context, as a single aberrant value from one sensor can indicate abnormal process deviation. This measure highlights localized and critical anomalies better than MSE, which tends to dilute them in a global average.

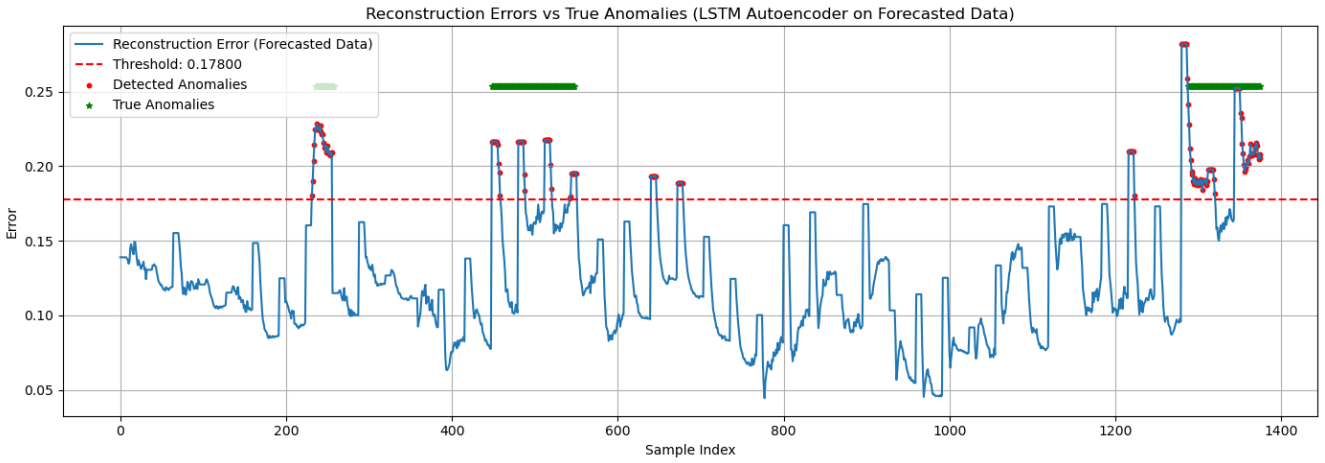


Figure 5.21: Reconstruction error vs actual anomalies

This figure 5.21 confirms that error peaks exceeding the threshold align well with the identified real anomalies. The model does not suffer from underdetection and shows remarkable ability to anticipate anomalous windows, detecting some sequences slightly before the intervals annotated by engineers. Even more impressively, the model flags several anomalies not identified in the original labels. This performance is explained by the sometimes incomplete nature of the manual labeling process.

6.2.3.3 Detection Evaluation

- **Precision**: 0.7273
- **Recall**: 0.8889
- **F1-score**: 0.8000

These results show a good trade-off between false alerts and missed detections. High recall is particularly important in an industrial context: it is better to alert too often than to miss a critical anomaly.

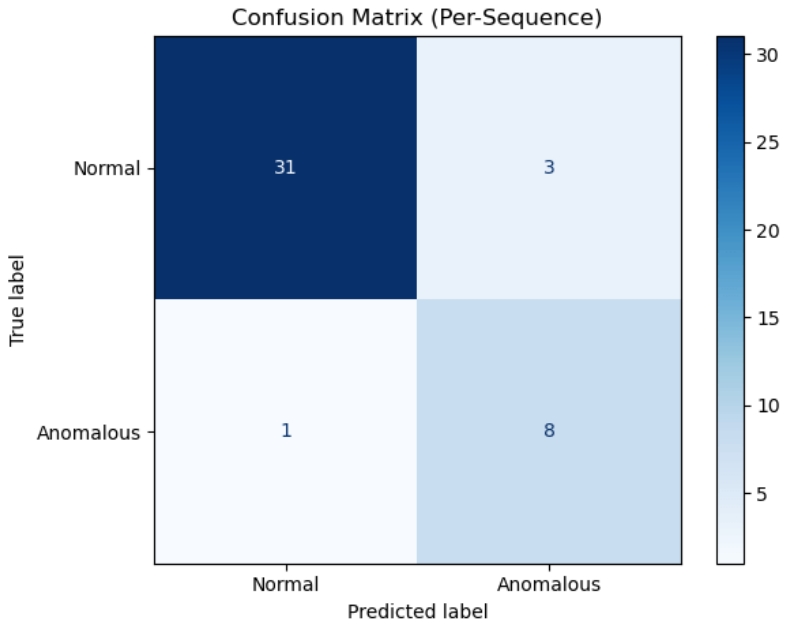


Figure 5.22: Confusion matrix (sequence-level flagging)

The confusion matrix confirms these observations: only 3 false positives and 1 false negative were recorded.

5.7.4 Neutral Analysis by Smoothed Mean Absolute Deviation

Given that our detection model is unsupervised and the labels provided by process engineers are manually created, we implemented a neutral and objective method to assess the consistency of detected anomalies: smoothed Mean Absolute Deviation.

This method calculates the mean absolute deviation (MAD) at each time t between the attribute values and their mean at that same time. The formula used is:

$$\text{MAD}(t) = \frac{1}{n}\sum_{i=1}^{n} |x_i(t) - \mu(t)| \quad \text{where} \quad \mu(t) = \frac{1}{n}\sum_{i=1}^{n} x_i(t)$$

where $x_i(t)$ is the value of attribute i at time t, and $\mu(t)$ is the average of all attributes at that time. This series is then smoothed with a moving average of size 32 (corresponding to the sequence size used in our model).

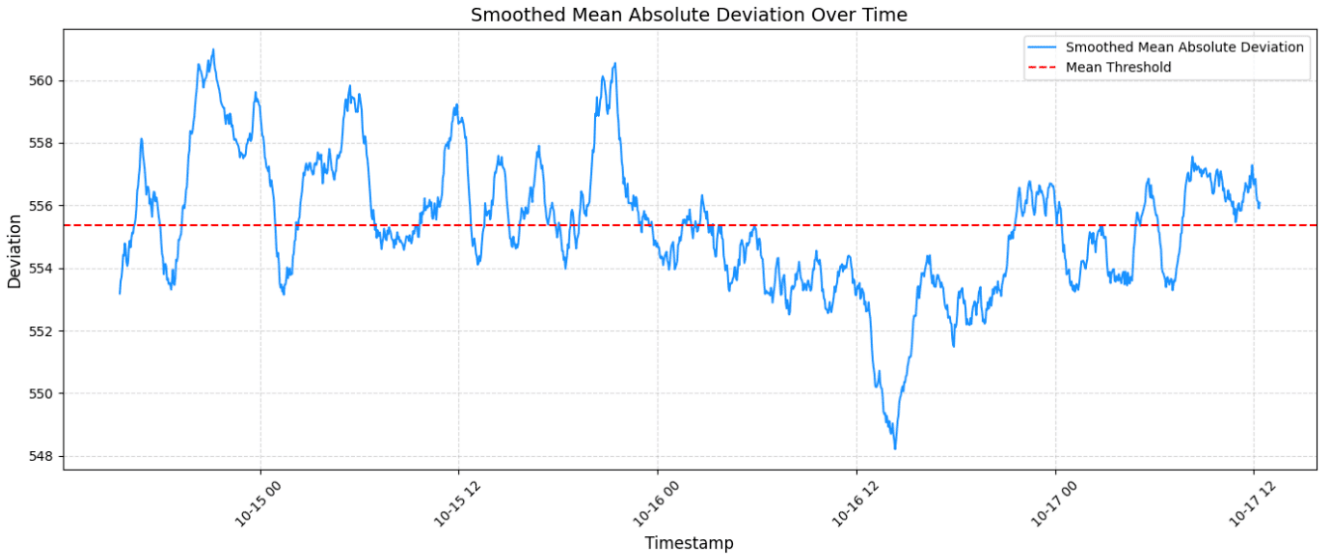


Figure 5.23: Evolution of smoothed mean absolute deviation (window = 32)

The curve of the figure 5.23 shows significant deviation peaks that more or less coincide with the anomalies detected by our model. While this supports the credibility of our approach, it's important to note that the data deviation curve is not a fully reliable evaluation method, as it is based solely on statistical criteria and does not account for domain context or the actual process complexity.

5.8 Limitations of Causal Approaches and Root Cause Analysis

In an attempt to go beyond simple detection and prediction, we invested significant time at the beginning of the project exploring causal analysis and *Root Cause Analysis* (RCA) methods, even though this was not among the original objectives. This initiative was personally motivated by a desire to better understand the internal workings of the process and identify attributes responsible for anomalies. However, these experiments proved unfruitful in our specific context and highlighted several structural limitations.

For root cause analysis, we tested different approaches focusing on identifying attributes that contributed most to the detected anomalies, based on sudden fluctuations, abnormal local variations, and reconstruction error analysis from the autoencoder model. The idea was to identify variables whose reconstruction consistently failed in the presence of anomalies, potentially indicating their role in degrading overall behavior.

While these methods could highlight attributes heavily involved in anomalous sequences, results were systematically biased toward variables representing valves or control actions directly manipulated by operators. Although these attributes show dramatic signals (spikes, oscillations, breakpoints), they are actually consequences of anomalies; often human or automatic system responses; not their root causes. This greatly limits the relevance of proposed diagnostics and underscores the difficulty of isolating a "first cause" without expert domain interpretation.

On the causal relationship extraction side, we explored a wide range of techniques from statistical and probabilistic methods (e.g., Granger causality, PCMCI) to advanced graph neural networks and deep learning-based causal models. However, all these methods failed to produce informative graphs: due to strong correlations among most attributes, the generated graphs had

numerous indistinct links with similar weights, making result interpretation difficult or practically useless.

In light of these findings, we concluded that causal analysis and automated root cause identification cannot be reliably addressed without a deep understanding of the chemical and operational workings of the RFCC process. It is clear that this task warrants a dedicated project, led by a multidisciplinary team including experts in petrochemical processes, industrial instrumentation, and AI specialists in explanatory modeling.

Despite the significant time invested in this direction, we consider this attempt an educational experience that helped us better understand the limitations of AI methods in complex industrial systems. It also reinforced the relevance of refocusing efforts on detection and prediction, where the results are concrete, measurable, and actionable by field engineers.

5.9 Conclusion

This chapter presented the implementation of our approach in the form of a complete application, combining a performant backend and an interactive frontend, designed to meet the constraints of the industrial sector. The entire system was built using modern technologies, ensuring robustness, modularity, and ease of use. Beyond implementation, we conducted a thorough comparative evaluation of several forecasting architectures, highlighting the superiority of the LSTM-MultiheadAttention model for prediction and the LSTM Autoencoder for anomaly detection. These models were integrated into a final pipeline, successfully tested on real annotated data. The results obtained, in terms of both metrics and visual coherence, confirm the relevance and reliability of our solution for intelligent monitoring of industrial processes.

General Conclusion

This thesis presented a hybrid and intelligent solution for the forecasting and anomaly detection in industrial time series originating from a Residue Fluid Catalytic Cracking (RFCC) unit, with a real-world case study based on the Algiers refinery. The main objective was to build a robust and interpretable pipeline capable of identifying critical abnormal behaviors, while adapting to the complexity of real-world data and industrial constraints.

The proposed pipeline is based on an effective combination of two models: a *LSTM with Multi-Head Attention* for forecasting future values, and an *BiLSTM Autoencoder* for detecting anomalies based on reconstruction error. This system was enhanced by a domain-specific heuristic rule (threshold applied to sensor `TI.0037`), validated by Sonatrach process engineers.

The results obtained were satisfactory both in terms of classical metrics (MSE, MAE, precision, recall, F1-score) and from a visual and functional perspective. Our model not only successfully identified the anomalous windows previously reported by engineers, but also anticipated certain anomalies prior to their peak, and even revealed suspicious events that had not been initially detected. A neutral validation method based on Mean Absolute Deviation further confirmed the system's relevance.

This work also highlighted several challenges, including label quality issues, high inter-feature correlation, and the difficulty of explainability and root cause analysis in complex systems. Nevertheless, these challenges were mitigated through sound methodological choices, model adaptations tailored to the nature of the data, and the integration of domain expertise.

Future Work

This project opens several perspectives for further development and industrial valorization:

- **Industrial Deployment**: A logical next step would be the deployment of our solution in a real production environment, using a microservices-based architecture with real-time streaming (e.g., Kafka), as envisioned by Sonatrach's technical team.
- **Active Learning**: To address the issue of limited labeled data, an active learning strategy could be implemented, where the model suggests suspicious windows to domain experts for validation, enabling iterative improvement of the system.
- **Root Cause Analysis**: Although this task proved complex in our setting, close collaboration with chemical process experts could enable the detected anomalies to be traced back to their origin, through the combination of artificial intelligence and domain-driven diagnostic approaches.
- **Self-Supervised Approaches**: Recent advances such as *self-supervised Transformers* or dynamic causal graph modeling for multivariate time series could further enhance early anomaly detection capabilities.

- **Generalization**: The designed pipeline can be adapted to other industrial installations or processes with minimal customization, making the solution scalable and reusable across various units or even other sectors.

In conclusion, this thesis demonstrates how a data-centered approach, integrating recent advances in deep learning while respecting domain constraints, can lead to concrete and impactful solutions for Industry 4.0.

References

bibliography

[3] D.E. Adanenche, A. Aliyu, A.Y. Atta, and B.J. El-Yakubu. "Residue fluid catalytic cracking: A review on the mitigation strategies of metal poisoning of RFCC catalyst using metal passivators/traps". In: *Fuel* 343 (2023). ISSN: 0016-2361. DOI: 10.1016/j.fuel.2023.127894. URL: https://www.sciencedirect.com/science/article/pii/S0016236123005070.

[4] CPECC et SONATRACH Activité LRP, Pôle Raffinage. "Manuel Opératoire – Unité de Craquage Catalytique du Fluide Résiduel". In: (2020). Document technique interne, révision 4, 324 pages.

[6] Varun Chandola, Arindam Banerjee, and Vipin Kumar. "Anomaly Detection: A Survey". In: *ACM Comput. Surv.* 41 (July 2009). DOI: 10.1145/1541880.1541882.

[7] M. Hejazi and Y. P. Singh. "One-Class Support Vector Machines Approach to Anomaly Detection". In: *Applied Artificial Intelligence* 27.5 (2013). DOI: 10.1080/08839514.2013.785791.

[8] Jiawei Han and Micheline Kamber. "Data Mining: Concepts and Techniques". In: The Morgan Kaufmann Series in Data Management Systems (Mar. 2006).

[9] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. "Isolation forest". In: (2008).

[10] Tolga Ergen and Suleyman Serdar Kozat. "Unsupervised Anomaly Detection With LSTM Neural Networks". In: *IEEE Transactions on Neural Networks and Learning Systems* 31.8 (Aug. 2020).

[11] Krzysztof Zarzycki and Maciej Ławryńczuk. "LSTM and GRU Neural Networks as Models of Dynamical Processes Used in Predictive Control: A Comparison of Models Developed for Two Chemical Reactors". In: *Processes* 8.5 (2020). DOI: 10.3390/pr8050582. URL: https://www.mdpi.com/1424-8220/21/16/5625.

[12] Tzu-Hsuan Lin and Jehn-Ruey Jiang. "Anomaly Detection with Autoencoder and Random Forest". In: *2020 International Computer Symposium (ICS)*. 2020. DOI: 10.1109/ICS51289.2020.00028.

[13] Kukjin Choi, Jihun Yi, Changhwa Park, and Sungroh Yoon. "Deep Learning for Anomaly Detection in Time-Series Data: Review, Analysis, and Guidelines". In: *IEEE Access* 9 (2021). DOI: 10.1109/ACCESS.2021.3107975.

[14] AIML.com. "Sequence Models Compared: RNNs, LSTMs, GRUs, and Transformers". In: (2025).

[15] Saidul Islam, Hanae Elmekki, Ahmed Elsebai, Jamal Bentahar, Nagat Drawel, Gaith Rjoub, and Witold Pedrycz. "A comprehensive survey on applications of transformers for deep learning tasks". In: *Expert Systems with Applications* 241 (2024). ISSN: 0957-4174. DOI: https://doi.org/10.1016/j.eswa.2023.122666.

[16] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. "Attention Is All You Need". In: 30 (2017).

[17] Chenwei Tang, Jialiang Huang, Mao Xu, Xu Liu, Fan Yang, Wentao Feng, Zhenan He, and Jiancheng Lv. "Attention-based early warning framework for abnormal operating conditions in fluid catalytic cracking units". In: *Applied Soft Computing* 153 (2024), p. 111275. ISSN: 1568-4946. DOI: https://doi.org/10.1016/j.asoc.2024.111275.

[18] Ge He, Lei Luo, Li Zhou, Yiyang Dai, Xu Ji, Chao Guo, and Zhaopeng Lu. "Deep learning prediction of yields of fluid catalytic cracking via differential evolutionary dual-stage attention-based LSTM". In: *Fuel* 370 (2024), p. 131826. ISSN: 0016-2361. DOI: https://doi.org/10.1016/j.fuel.2024.131826.

[19] Wende Tian, Shaochen Wang, Suli Sun, Chuankun Li, and Yang Lin. "Intelligent prediction and early warning of abnormal conditions for fluid catalytic cracking process". In: *Chemical Engineering Research and Design* 181 (2022), pp. 304–320. ISSN: 0263-8762. DOI: https://doi.org/10.1016/j.cherd.2022.03.031.

[20] A. Trivedi. "Explainable Forecasting Models For Load Forecasting". In: (July 2024).

[21] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. "A Time Series is Worth 64 Words: Long-term Forecasting with Transformers". In: (2023). Published as a conference paper at ICLR 2023.

[22] Zhongrun Xiang, Jun Yan, and Ibrahim Demir. "A Rainfall-Runoff Model with LSTM-based Sequence-to-Sequence Learning". In: *Water Resources Research* 56 (Jan. 2020). DOI: 10.1029/2019WR025326.

[31] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang. "Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting". In: *Proceedings of the AAAI Conference on Artificial Intelligence* 35.12 (2021). DOI: 10.1609/aaai.v35i12.17325.

[32] David M W Powers. "Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness and Correlation". In: *Journal of Machine Learning Technologies* 2.1 (2011).

[39] Jérémy Robert. "Machine Learning : Définition, fonctionnement, utilisations". In: (Nov. 2020).

[41] Intelligence Artificielle School. "Qu'est-ce que l'overfitting et comment s'en protéger ?" In: (Juin 2024).

[42] Intelligence Artificielle School. "Underfitting : le sous-apprentissage en Machine Learning". In: (Février 2024).

[43] Alex Graves. "Sequence transduction with recurrent neural networks". In: *arXiv preprint arXiv:1211.3711* (2012). URL: https://arxiv.org/abs/1211.3711.

[45] Pure Illusion. *Définition de Framework : un socle pour construire une application.* Consulté en juillet 2025. Pure Illusion - Lexique Digital. Feb. 2024. URL: https://www.pure-illusion.com/lexique/definition-de-framework.

Webography

[1] MINISTÈRE DE L'ÉNERGIE , DES MINES ET DES ENERGIES RENOUVELABLES. *Produits pétroliers*. https://www.energy.gov.dz/?rubrique=produits-petroliers.

[5] Hussein Jawad, Mohamed Karim Abid. *Détection d'anomalies dans les séries temporelles*. https://www.quantmetry.com/blog/detection-danomalies-dans-les-series-temporelles/.

[23] Python Software Foundation. *Python Programming Language*. https://www.python.org/doc/essays/blurb/.

[24] Google Brain Team. *TensorFlow: An end-to-end open-source machine learning platform*. https://www.tensorflow.org.

[25] scikit-learn developers. *scikit-learn: Machine Learning in Python*. https://scikit-learn.org.

[26] Anaconda, Inc. *Conda: Package, dependency and environment management*. https://docs.conda.io/projects/conda/en/latest/user-guide/tasks/manage-environments.html.

[27] Sebastián Ramírez. *FastAPI - Modern, Fast (high-performance) web framework for building APIs with Python*. https://fastapi.tiangolo.com.

[28] Encode OSS. *Uvicorn: The lightning-fast ASGI server implementation, using uvloop and httptools*. https://www.uvicorn.org.

[29] Meta Platforms, Inc. *React – A JavaScript library for building user interfaces*. https://reactjs.org.

[30] Git SCM. *Git - Distributed Version Control System*. https://git-scm.com.

[33] Connaissance des Énergies. *Le raffinage pétrolier*. https://www.connaissancedesenergies.org/fiche-pedagogique/raffinage-petrolier. Juin 2024.

[34] Connaissance des Énergies. *Qu'est-ce qu'un hydrocarbure et comment migre-t-il vers la surface ?* https://www.connaissancedesenergies.org/questions-et-reponses-energies/pourquoi-et-comment-un-hydrocarbure-migre-t-il-vers-la-surface.

[35] Office québécois de la langue française. *Hydrodésulfuration*. https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8883584/hydrodesulfuration.

[36] StudySmarter. *Catalyseur*. https://www.studysmarter.fr/resumes/physique-chimie/chimie/catalyseur/.

[37] Encyclopædia Universalis. *Combustion*. https://junior.universalis.fr/encyclopedie/combustion.

[38] APEC. *Fiche métier : Ingénieur procédés*. https://www.apec.fr/tous-nos-metiers/services-techniques/ingenieur-procedes.html.

[40] IBM Corporation. *Que sont les réseaux neuronaux ?* https://www.ibm.com/fr-fr/think/topics/neural-networks.

[44] Jun Hosoi. *Qu'est-ce que l'horodatage ?* https://www.globalsign.com/fr/blog/horodatage-definition-et-fonctionnement.

[46] Linkweb. *Front-end.* https://linkweb.fr/creation-site-internet-toulouse/front-end/.

[47] Linkweb. *Back-end.* https://linkweb.fr/creation-site-internet-toulouse/back-end/.

[48] Red Hat. *Apache Kafka, qu'est-ce que c'est ?* URL: https://www.redhat.com/fr/topics/integration/what-is-apache-kafka.

Appendices

Chapter A: Technical Glossary

A.1 Refining

Petroleum refining is a critical industrial process that transforms crude oil (which is unusable in its raw form) into usable end products such as gasoline, fuel oil, or naphtha. In its natural state, crude oil is a complex and disordered mixture of various hydrocarbon molecules, mainly composed of carbon and hydrogen. To extract usable products, these components must be separated based on their properties. This is precisely the purpose of refining [33].

A.2 Hydrocarbons

Hydrocarbons are molecules made up exclusively of carbon and hydrogen atoms. They are primarily found in crude oil and natural gas and constitute the main components of fossil fuels used as energy sources [34].

A.3 Atmospheric distillation

This operation aims to separate the various components of a liquid based on their boiling points. In the case of crude oil, it is heated to approximately 400°C and then injected into a large distillation column about 60 meters tall.

Under the effect of heat, the hydrocarbons gradually vaporize: first the lightest, then the intermediate ones, and finally some of the heavier ones. The temperature decreases progressively as we go up the column, allowing each type of hydrocarbon to condense at a specific level. The lightest condense at the top of the column, while the heaviest remain at the bottom [33].

A.4 Vacuum distillation

This step applies the same principle as atmospheric distillation but targets the heaviest residuals of crude oil. A second, smaller column is used, in which a vacuum is created; hence the name "vacuum distillation". By lowering the pressure, boiling points are also reduced, allowing heavy products to be separated without excessive heating, thus avoiding their degradation.

In this column, diesel is recovered at the top, while heavy fuel oil settles at the bottom. The remaining residues (even heavier) are then stored for use in the production of lubricants [33].

A.5 Hydrodesulfurization

This is a process used to remove sulfur from crude oil. Hydrocarbons are heated in the presence of hydrogen, causing a chemical reaction that extracts sulfur in the form of gas. This treatment is

essential for reducing pollution caused by fuels and for complying with environmental standards [35].

A.6 Catalyst

A catalyst is a substance that facilitates a chemical reaction by accelerating its rate without being consumed or chemically altered. It works by lowering the activation energy required for the reaction, thereby enabling molecules to react more easily and reducing waste [36].

A.7 Combustion

This is a chemical reaction between a fuel (such as wood, gas, or gasoline) and an oxidizing agent, most commonly oxygen from the air. For instance, when wood burns, oxygen combines with its carbon content. This reaction releases heat and light, and generally produces gases such as carbon dioxide (CO_2) and water vapor [37].

A.8 Process engineer

The process engineer plays a vital role in ensuring the proper functioning of production units. Their mission is to design, analyze, and improve industrial processes so they are efficient, safe, and aligned with business needs.

They operate across various sectors, including chemistry, pharmaceuticals, agri-food, energy, and petrochemicals.

Their daily tasks include drafting technical documents, writing specifications, supervising equipment installation, and ensuring operational reliability and safety. They continuously monitor process performance and recommend improvements while adhering to quality, safety, and environmental standards [38].

A.9 Machine learning

Machine learning is a branch of artificial intelligence that allows algorithms to learn from data rather than being explicitly programmed for every task. The idea is to feed the algorithm with large amounts of data (numbers, texts, images, measurements...), enabling it to identify recurring patterns.

With sufficient training, the model becomes capable of making predictions or decisions on new data based on what it has previously learned. The more relevant examples it encounters, the more accurate its outputs. For example, such a system can predict temperature, detect anomalies, or recognize speech [39].

A.10 Neural network

Neural networks are machine learning models inspired by how the human brain functions. They learn to recognize patterns in data by simulating how our neurons communicate to process information, make decisions, or generate predictions.

A neural network consists of multiple layers: an input layer that receives the data, hidden layers where learning occurs, and an output layer that delivers the result.

Each artificial "neuron" is connected to others and has a weight and a threshold. If the input exceeds this threshold, the neuron activates and passes the signal to the next ones; otherwise, it remains inactive. Through exposure to numerous examples, the network adjusts its weights to improve its predictive or classification capabilities.

Today, neural networks can process massive volumes of data extremely rapidly, making them essential for tasks like voice recognition, image analysis, or search engines. For instance, Google uses such technologies to refine its search results.

Neural networks, also known as ANNs (Artificial Neural Networks), are a cornerstone of deep learning, a specialized field within machine learning [40].

A.11 Overfitting

Overfitting is a common issue in machine learning and modeling. It occurs when a model learns the training data too well, capturing not only the underlying patterns but also the noise and outliers. As a result, it performs very well on known data but poorly on unseen data, lacking generalization capability.

Instead of modeling the true underlying structure, the model becomes overly tailored to the training set, making it fragile when faced with new situations. This is typically observed as a large discrepancy between training and test performance [41].

A.12 Underfitting

In machine learning, underfitting occurs when a model fails to adequately learn from the data. This generally happens when the model is too simplistic or lacks the capacity to capture complex relationships present in the training data. Consequently, its predictions are inaccurate both on the training and test sets.

Underfitting is often associated with high bias, meaning the model makes overly strong or simplistic assumptions about the data [42].

A.13 Transduction

In machine learning, transduction refers to the process by which a model learns to map an input sequence to an output sequence. This is often referred to as sequence-to-sequence (seq2seq) transformation and is fundamental to many applications such as machine translation, speech recognition, text summarization, or chatbot response generation.

Unlike purely inductive approaches that aim to infer general rules from data for broad application, transduction focuses on predicting outputs for specific known inputs by leveraging observed pairings from the training data. The goal is not necessarily to extract an abstract rule but rather to perform accurate transformations for each known input-output pair.

In modern architectures such as Transformers, this mechanism is handled through encoder-decoder modules: the encoder processes the input sequence to extract meaningful representations, which the decoder then uses to produce the output sequence [43].

A.14 Timestamping

Timestamping refers to the process of assigning a precise date and time to an event or piece of data. It functions like a temporal stamp applied to a file, message, or action, marking the exact moment it occurred. This mechanism is widely used in the digital world; for organizing files, securing transactions, or ensuring document integrity [44].

A.15 Framework

A web framework is essentially a toolbox provided to developers. The term derives from English: "frame" means "structure" and "work" means "task". Concretely, it is a set of prebuilt components that facilitate the creation of websites or applications.

These tools manage common tasks such as data handling (CRUD), project structuring, security, and enforcement of best practices, allowing developers to save time and focus on business logic rather than reinventing the wheel.

In short, a framework provides a structured environment for faster, cleaner, and often more secure development [45].

A.16 Frontend

In web development, the frontend refers to everything the user sees and interacts with directly on a website or web application. This includes menus, buttons, images, texts, and animations. It is essentially the visible part of the iceberg [46].

A.17 Backend

Opposite to the frontend, the backend handles everything that happens behind the scenes: data processing, business logic, interactions with databases, etc. [47].

A.18 Kafka

Apache Kafka is a distributed platform designed to handle real-time data streams. It enables the publishing, storage, processing, and consumption of event streams (or logs) in a fast and reliable manner.

What makes it unique is its capacity to transmit massive volumes of data between multiple applications or systems simultaneously. Kafka acts as a central hub connecting numerous data sources (sensors, services, applications...) to consumer systems, ensuring high availability and low latency.

In summary, Kafka does more than just move data from point A to point B: it provides the flexibility to circulate data across multiple endpoints efficiently and continuously [48].

Chapter B: Data tables

B.1 Hyperparameter Tuning of Prediction Models

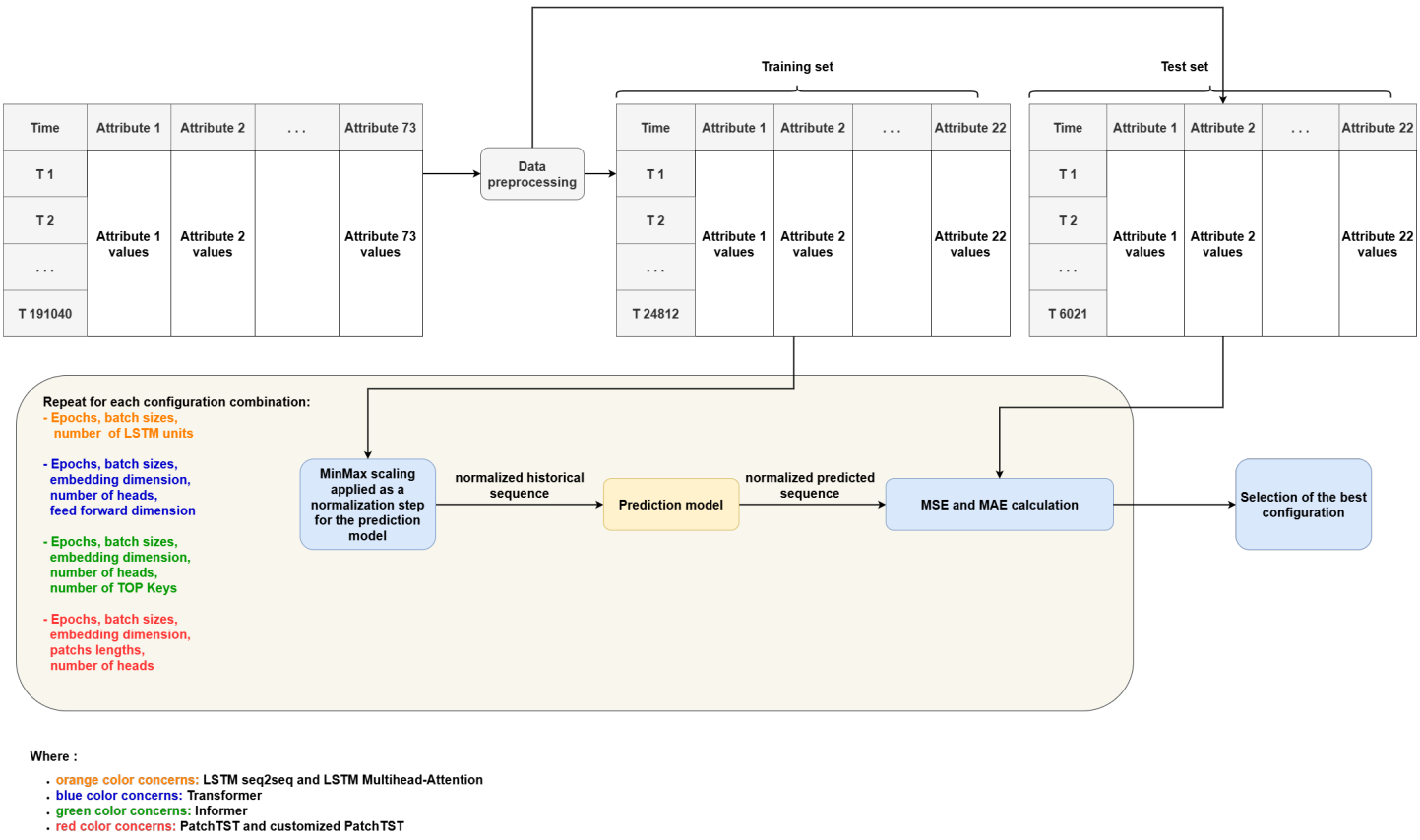


Figure B.1: Fine-tuning pipeline for the prediction models

This diagram illustrates the pipeline for forecasting. It incorporates various model types (LSTM, Transformer, PatchTST) and explores several configurations. Evaluation is carried out using metrics such as MSE and MAE on test data, in order to identify the best architecture for the prediction task.

B.1.1 LSTM seq2seq model

Epochs	Batch Size	LSTM Units	Validation MSE	Validation MAE
50	128	32	0.005997	0.054246
50	128	64	0.004316	0.044812
50	128	128	0.003224	0.036543
50	256	32	0.007994	0.060472
50	256	64	0.004210	0.043538
50	256	128	0.003644	0.038909
50	512	32	0.011214	0.069197
50	512	64	0.007929	0.061060
50	512	128	0.003951	0.043494
50	1024	32	0.014451	0.078944
50	1024	64	0.012268	0.073205
50	1024	128	0.009507	0.064183

Table B.1: Results of Hyperparameter Tuning for the LSTM Seq2Seq Model

B.1.2 LSTM-Multihead Attention model

Epochs	Batch Size	LSTM Units	Val MSE	Val MAE
50	128	32	0.005229	0.052088
50	128	64	0.004634	0.048936
50	128	128	0.004267	0.046870
50	256	32	0.007160	0.062592
50	256	64	0.006300	0.057995
50	256	128	0.004386	0.047417
50	512	32	0.008040	0.068366
50	512	64	0.008046	0.066489
50	512	128	0.005549	0.054649
50	1024	32	0.009999	0.075977
50	1024	64	0.009509	0.075292
50	1024	128	0.007346	0.064754

Table B.2: Hyperparameter tuning results for the LSTM-Multihead Attention model

B.1.3 Transformer model

Epochs	Batch Size	Embed Dims	Heads	FF Dims	Val MSE	Val MAE
50	128	64	4	64	0.005926	0.056789
50	128	64	4	128	0.006988	0.063287
50	128	64	8	64	0.006869	0.060477
50	128	64	8	128	0.006230	0.055680
50	128	64	16	64	0.008064	0.062720

Continued on next page

Epochs	Batch Size	Embed Dims	Heads	FF Dims	Val MSE	Val MAE
50	128	64	16	128	0.008132	0.063874
50	128	128	4	64	0.008560	0.066405
50	128	128	4	128	0.006056	0.057270
50	128	128	8	64	0.007906	0.068401
50	128	128	8	128	0.007060	0.061485
50	128	128	16	64	0.006439	0.056572
50	128	128	16	128	0.008873	0.071576
50	256	64	4	64	0.006935	0.060065
50	256	64	4	128	0.007436	0.060944
50	256	64	8	64	0.008264	0.064801
50	256	64	8	128	0.007423	0.063240
50	256	64	16	64	0.007065	0.060548
50	256	64	16	128	0.007112	0.060548
50	256	128	4	64	0.006300	0.057663
50	256	128	4	128	0.006947	0.058937
50	256	128	8	64	0.007933	0.062416
50	256	128	8	128	0.007735	0.063369
50	256	128	16	64	0.007505	0.062710
50	256	128	16	128	0.008240	0.063906
50	512	64	4	64	0.007632	0.063304
50	512	64	4	128	0.007905	0.063994
50	512	64	8	64	0.007676	0.062083
50	512	64	8	128	0.008072	0.065382
50	512	64	16	64	0.007370	0.060062
50	512	64	16	128	0.007335	0.060547
50	512	128	4	64	0.007264	0.059671
50	512	128	4	128	0.007694	0.063475
50	512	128	8	64	0.008014	0.063659
50	512	128	8	128	0.007642	0.061440
50	512	128	16	64	0.007672	0.061585
50	512	128	16	128	0.007950	0.062513
50	1024	64	4	64	0.009156	0.070008
50	1024	64	4	128	0.009934	0.072953
50	1024	64	8	64	0.009434	0.068627
50	1024	64	8	128	0.009784	0.069658
50	1024	64	16	64	0.007710	0.064009
50	1024	64	16	128	0.008552	0.066936
50	1024	128	4	64	0.008875	0.066876
50	1024	128	4	128	0.008524	0.065175
50	1024	128	8	64	0.008043	0.062020
50	1024	128	8	128	0.008534	0.065615
50	1024	128	16	64	0.007704	0.063836
50	1024	128	16	128	0.009210	0.068030

Table B.3: Hyperparameter tuning results for the Transformer model

B.1.4 Customized PatchTST Model

Epochs	Batch Size	Embed Dims	NB Heads	Patch Len	Val MSE	Val MAE
50	128	64	4	2	0.004572	0.046815
50	128	64	8	2	0.005710	0.057380
50	128	64	16	2	0.005645	0.053070
50	128	128	4	2	0.004062	0.045966
50	128	128	8	2	0.004417	0.047632
50	128	128	16	2	0.004784	0.049193
50	128	64	4	4	0.004591	0.049393
50	128	64	8	4	0.005717	0.055670
50	128	64	16	4	0.004181	0.045743
50	128	128	4	4	0.004048	0.043744
50	128	128	8	4	0.003991	0.045914
50	128	128	16	4	0.003849	0.042934
50	128	64	4	8	0.004667	0.047639
50	128	64	8	8	0.005014	0.050723
50	128	64	16	8	0.005640	0.055338
50	128	128	4	8	0.004288	0.046365
50	128	128	8	8	0.005089	0.051889
50	128	128	16	8	0.004107	0.044572
50	128	64	4	16	0.005740	0.053985
50	128	64	8	16	0.005063	0.052321
50	128	64	16	16	0.005930	0.053320
50	128	128	4	16	0.004166	0.045562
50	128	128	8	16	0.004491	0.047817
50	128	128	16	16	0.004091	0.044412
50	256	64	4	2	0.007412	0.061596
50	256	64	8	2	0.005732	0.053764
50	256	64	16	2	0.006219	0.056458
50	256	128	4	2	0.004718	0.047602
50	256	128	8	2	0.004907	0.050799
50	256	128	16	2	0.006413	0.058012
50	256	64	4	4	0.006821	0.059621
50	256	64	8	4	0.006136	0.055580
50	256	64	16	4	0.007815	0.061784
50	256	128	4	4	0.005898	0.055383
50	256	128	8	4	0.004866	0.049608
50	256	128	16	4	0.005335	0.052847
50	256	64	4	8	0.007130	0.059261
50	256	64	8	8	0.008550	0.065350
50	256	64	16	8	0.006487	0.057708
50	256	128	4	8	0.005169	0.050707
50	256	128	8	8	0.004891	0.049795
50	256	128	16	8	0.006661	0.056748

Continued on next page

Epochs	Batch Size	Embed Dims	NB Heads	Patch Len	Val MSE	Val MAE
50	256	64	4	16	0.005815	0.054220
50	256	64	8	16	0.006675	0.058653
50	256	64	16	16	0.006756	0.059121
50	256	128	4	16	0.006505	0.056513
50	256	128	8	16	0.006401	0.056052
50	256	128	16	16	0.006438	0.058013
50	512	64	4	2	0.008760	0.067329
50	512	64	8	2	0.009023	0.066034
50	512	64	16	2	0.008816	0.065381
50	512	128	4	2	0.005835	0.053774
50	512	128	8	2	0.006330	0.059739
50	512	128	16	2	0.008062	0.063963

Table B.4: Hyperparameter tuning results for the Customized PatchTST model

B.1.5 PatchTST model

Epochs	Batch Size	Embed Dims	NB Heads	Patch Len	Val MSE	Val MAE
50	128	64	4	4	0.052292	0.186301
50	128	64	8	4	0.052318	0.180447
50	128	64	4	8	0.051700	0.185856
50	128	64	8	8	0.051289	0.184532
50	256	64	4	4	0.075479	0.227473
50	256	64	8	4	0.076349	0.229757
50	256	64	4	8	0.060624	0.205952
50	256	64	8	8	0.057463	0.195235
50	512	64	4	4	0.076720	0.230387
50	512	64	8	4	0.077367	0.233348
50	512	64	4	8	0.070898	0.226341
50	512	64	8	8	0.068396	0.218516
50	1024	64	4	4	0.078930	0.228741
50	1024	64	8	4	0.078477	0.231129
50	1024	64	4	8	0.076609	0.232774
50	1024	64	8	8	0.074788	0.226565

Table B.5: Hyperparameter tuning results for the PatchTST model

B.1.6 Informer model

Epochs	Batch Size	Embed Dims	NB Heads	TOP-K	Val MSE	Val MAE
50	128	64	4	2	0.005777	0.051240
50	128	64	4	4	0.004958	0.049834
50	128	64	4	8	0.005548	0.053524

Continued on next page

Epochs	Batch Size	Embed Dims	NB Heads	TOP-K	Val MSE	Val MAE
50	128	64	4	16	0.004278	0.045901
50	128	64	8	2	0.004784	0.047207
50	128	64	8	4	0.004599	0.048533
50	128	64	8	8	0.005324	0.052192
50	128	64	8	16	0.004507	0.046693
50	128	64	16	2	0.004520	0.048066
50	128	64	16	4	0.004835	0.050274
50	128	64	16	8	0.005322	0.052695
50	128	64	16	16	0.006006	0.056856
50	128	128	4	2	0.004860	0.051468
50	128	128	4	4	0.003937	0.043095
50	128	128	4	8	0.003883	0.043374
50	128	128	4	16	0.004808	0.047303
50	128	128	8	2	0.004053	0.043693
50	128	128	8	4	0.004523	0.048474
50	128	128	8	8	0.004955	0.048839
50	128	128	8	16	0.004507	0.048151
50	128	128	16	2	0.004423	0.046931
50	128	128	16	4	0.004509	0.047946
50	128	128	16	8	0.003800	0.041697
50	128	128	16	16	0.003901	0.043715
50	256	64	4	2	0.004948	0.051149
50	256	64	4	4	0.006739	0.057694
50	256	64	4	8	0.006879	0.058218
50	256	64	4	16	0.006924	0.059443
50	256	64	8	2	0.005759	0.055248
50	256	64	8	4	0.004862	0.049653
50	256	64	8	8	0.006832	0.057646
50	256	64	8	16	0.006011	0.055861
50	256	64	16	2	0.005868	0.054324
50	256	64	16	4	0.006582	0.056378
50	256	64	16	8	0.006431	0.056175
50	256	64	16	16	0.006490	0.057704
50	256	128	4	2	0.004582	0.047125
50	256	128	4	4	0.004551	0.046481
50	256	128	4	8	0.004185	0.045418
50	256	128	4	16	0.004554	0.049928
50	256	128	8	2	0.005278	0.049666
50	256	128	8	4	0.005427	0.054112
50	256	128	8	8	0.004272	0.044903
50	256	128	8	16	0.005389	0.052953
50	256	128	16	2	0.005193	0.051392
50	256	128	16	4	0.004615	0.049955
50	256	128	16	8	0.005016	0.050729
50	256	128	16	16	0.003996	0.043198

Continued on next page

Epochs	Batch Size	Embed Dims	NB Heads	TOP-K	Val MSE	Val MAE
50	512	64	4	2	0.008750	0.068172
50	512	64	4	4	0.007760	0.063138
50	512	64	4	8	0.008343	0.064216
50	512	64	4	16	0.008574	0.064941
50	512	64	8	2	0.008003	0.063921
50	512	64	8	4	0.005462	0.053656
50	512	64	8	8	0.008392	0.065625
50	512	64	8	16	0.006466	0.054236
50	512	64	16	2	0.008264	0.065233
50	512	64	16	4	0.007098	0.057937
50	512	64	16	8	0.007798	0.063587
50	512	64	16	16	0.008210	0.066267
50	512	128	4	2	0.004681	0.050289
50	512	128	4	4	0.006740	0.057173
50	512	128	4	8	0.005343	0.051870
50	512	128	4	16	0.005364	0.053690
50	512	128	8	2	0.005372	0.050015
50	512	128	8	4	0.006016	0.056329
50	512	128	8	8	0.004284	0.046015
50	512	128	8	16	0.006843	0.059557
50	512	128	16	2	0.003918	0.043692
50	512	128	16	4	0.005125	0.050467
50	512	128	16	8	0.006662	0.059284
50	512	128	16	16	0.005479	0.054048
50	1024	64	4	2	0.010051	0.069788
50	1024	64	4	4	0.011642	0.075615
50	1024	64	4	8	0.009926	0.070596
50	1024	64	4	16	0.009999	0.069041
50	1024	64	8	2	0.009965	0.070952
50	1024	64	8	4	0.009297	0.071173
50	1024	64	8	8	0.009481	0.069527
50	1024	64	8	16	0.009896	0.068786
50	1024	64	16	2	0.009999	0.072491
50	1024	64	16	4	0.010320	0.071783
50	1024	64	16	8	0.008901	0.067211
50	1024	64	16	16	0.009526	0.068801
50	1024	128	4	2	0.007720	0.062927
50	1024	128	4	4	0.008411	0.066207
50	1024	128	4	8	0.007204	0.060501
50	1024	128	4	16	0.008653	0.067370
50	1024	128	8	2	0.007682	0.062292
50	1024	128	8	4	0.007309	0.062369
50	1024	128	8	8	0.007017	0.061715
50	1024	128	8	16	0.006978	0.063092
50	1024	128	16	2	0.008204	0.064381

Continued on next page

Epochs	Batch Size	Embed Dims	NB Heads	TOP-K	Val MSE	Val MAE
50	1024	128	16	4	0.006945	0.060834
50	1024	128	16	8	0.006829	0.057997
50	1024	128	16	16	0.007466	0.060394

Table B.6: Hyperparameter tuning results for the Informer model

B.2 Hyperparameter Tuning of Detection Models

B.2.1 BiLSTM Autoencoder

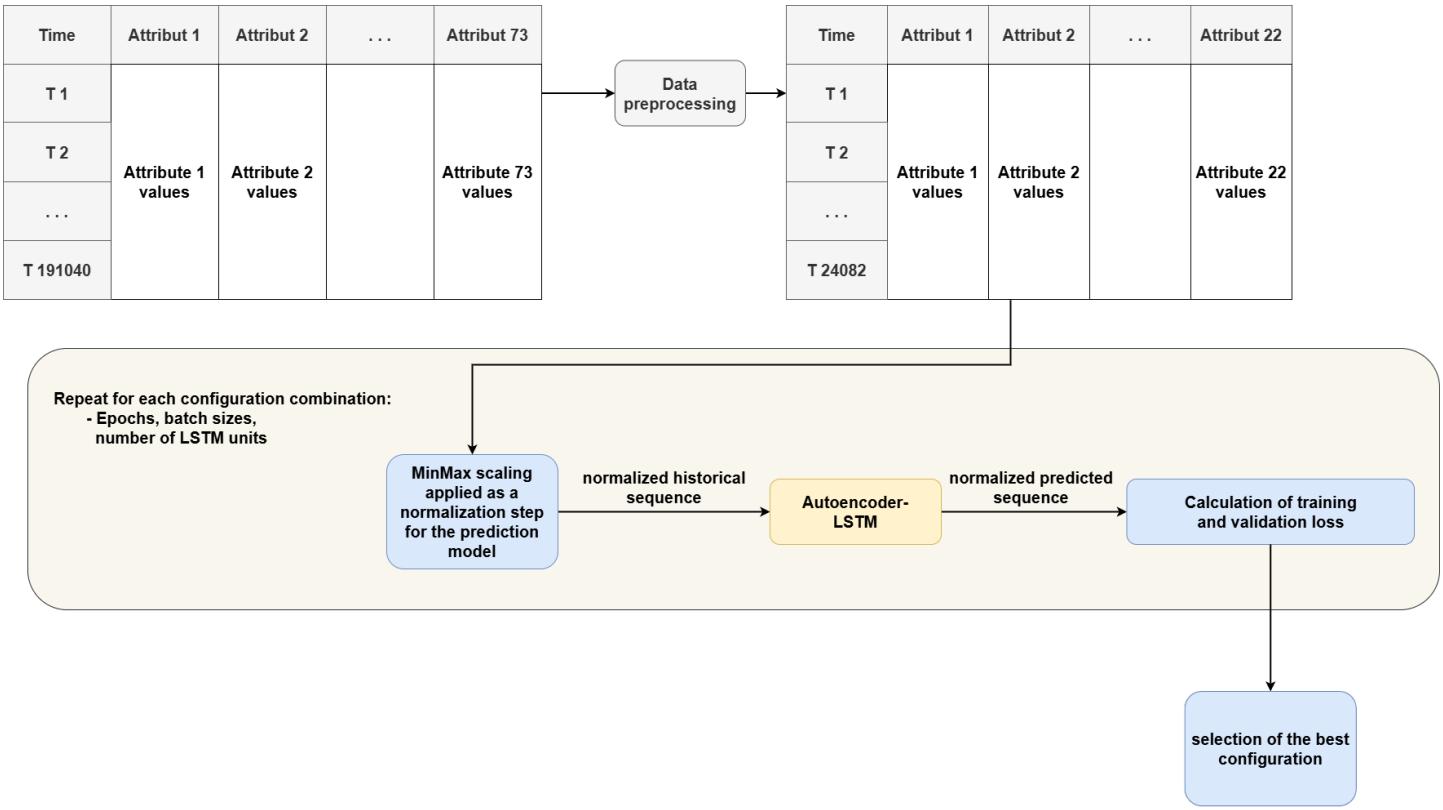


Figure B.2: Fine-tuning pipeline for the anomaly detection model based on an BiLSTM Autoencoder

The diagram above represents the fine-tuning process for an anomaly detection model. It uses an BiLSTM autoencoder to learn the reconstruction of normalized sequences. The performance is evaluated using a loss function on training and validation data, allowing the selection of the best configuration.

Index	Epochs	Batch Size	LSTM Units	Train Loss	Val Loss
0	50	64	64	0.002576	0.002059
1	50	64	128	0.003624	0.001833
2	50	64	256	0.002261	0.001736
3	50	128	64	0.003050	0.003058
4	50	128	128	0.001814	**0.001532**
5	50	128	256	0.001989	0.001695
6	50	256	64	0.003080	0.003139
7	50	256	128	0.002118	0.001992
8	50	256	256	0.001727	0.001686
9	50	512	64	0.004868	0.004047
10	50	512	128	0.004095	0.002788
11	50	512	256	0.002569	0.002320
12	50	1024	64	0.009330	0.010072
13	50	1024	128	0.004693	0.005289
14	50	1024	256	0.005327	0.005554

Table B.7: Hyperparameter tuning results for the BiLSTM Autoencoder model